



Dsmanual this is the manual of data structure and algorithm

Digital signal analysis and processing (Pokhara University)



Scan to open on Studocu

Chapter 1: Concept of Data Structure

1.1 Introduction

Data type is a collection of values and a set of operations on those values.

Data structure is a particular way of storing and organizing data in a computer so that it can be used efficiently. There are different ways to organize the data such as organization of data into the stack, queue, list, tree, graph and so on.

Different types of data structures are used for different types of applications, and some are highly specialized to specific tasks. Data structures are used in almost every program or software system. Specific data structures are essential for many efficient algorithms, and make possible the management of huge amounts of data, such as large databases and internet indexing services. Some formal design methods and programming languages emphasize data structures, rather than algorithms, as the key organizing factor in software design.

Classification of Data Structure

Data structures are normally divided into two broad categories.

- i) Primitive data structures
- ii) Non primitive data structures

Primitive data structures

These are basic structures and are directly operated upon by the machine instructions. Integer, floating point numbers, character constants, string constants and pointers are examples.

Non primitive data structures

These are more sophisticated data structures. These are derived from the primitive data structures. Arrays, lists and files are examples.

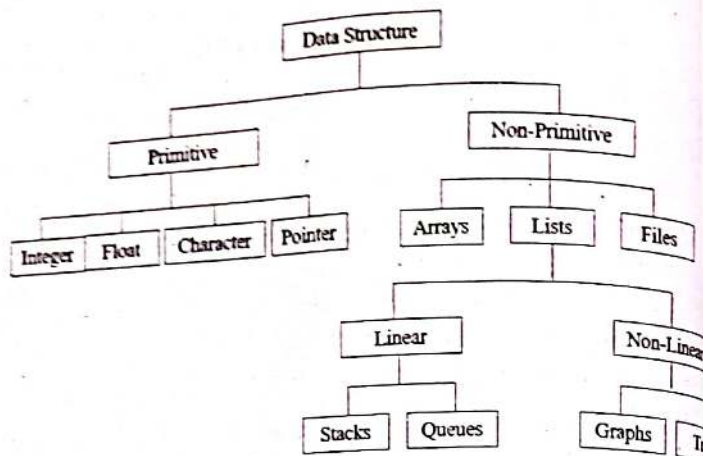


Figure 1.1: Classification of data structure

Characteristics of Data Structure

Data structure	Advantage	Disadvantage
Array	Quick inserts Fast access if index known	Slow search Slow delete Fixed size
Stack	Last-in, first-out access	Slow access to other items
Queue	First-in, first-out access	Slow access to other items
List	Quick insert Quick delete	Slow search
Binary Tree	Quick search Quick insert Quick delete (If the tree remains balanced)	Deletion algorithm is complex

Hash Table	Very fast access if key is known Quick insert	Slow delete Access slow if key is not known Inefficient memory usage
Heap	Quick insert Quick delete Access to largest item	Slow access to other items
Graph	Best models real-world situations	Some algorithms are slow and very complex

1.2 Abstract Data Types

Data type is a collection of values and a set of operations on those values. Those collection and that operations form a mathematical construct that may be implemented using a particular hardware or software data structure and ADT is a basic mathematical concept that defines the data type. ADT focuses on what it does and ignoring how it does its job. A stack or a queue is an example of an ADT.

An ADT consists of two parts that are

- Value definition and
- Operator definition

The value definition defines the collection of values for the ADT and consists of two parts: definition clause and condition clause.

In definition clause permissible data values are defined and condition clause is used to specify any conditions that may be on the data values.

Immediately after the value definition, operator definition is defined which describe all possible operators that may be implemented on those data defined in definition clause. Each operator is defined as a function with three parts:

- Header
- Pre-condition
- Post-condition

The header is just like C function header which consists of header name, arguments and appropriate data type.

The optional pre-condition specifies any restrictions that must be satisfied before the operations can be applied.

The post condition specifies the actual operations done by the operator.

To illustrate the concept of an ADT, consider the ADT RATIONAL, which corresponds to the mathematical concept of a rational number. A rational number is a number that can be expressed as the quotient of two integers. The operations on rational numbers that we define are the creation of a rational number from two integers, addition, multiplication, and testing for equality. The following is an initial specification of this ADT:

```
/*value definition*/
abstract typedef <integer, integer> RATIONAL;
condition RATIONAL[1] != 0;
/*operator definition*/
abstract RATIONAL makerational(a,b)
int a, b;
precondition b != 0;
postcondition makerational[0] == a;
                makerational[1] == b;
abstract RATIONAL add(a,b) /*written a+b */
RATIONAL a,b;
Postcondition add[1] == a[1] * b[1];
                add[0] == a[0]*b[1] + b[0]*a[1];
abstract RATIONAL mult(a,b) /*written a * b */
```

```
RATIONAL a,b;
Postcondition mult[0] == a[0] * b[0];
                mult[1] == a[1]*b[1];
abstract equal(a,b) /*written a == b */
RATIONAL a,b;
Postcondition equal == (a[0] * b[1]==b[0]*a[1]);
```

Explain how a class acts as an ADT?

In OOP, class implements the concept of ADT by defining both the set of values of a given type and the set of operations that can operate on those values i.e. class can be thought as a description of the data in the class and list of operations that operate on this data and instructions on how to use these operations. What is excluded is the details of how the methods carry out their tasks. End users tell what methods to call, how to call them, and the results that should be expected, but not HOW they work.

Example

```
class Rational
{
    long numerator;
    long denominator;
    public:
        Rational add(Rational);
        Rational mult(Rational);
}
```

Here, in above class template, a set of values that are numerator and denominator and the set of operations that are add and mult are declared without giving the definition of the function.

1.3 Algorithms

An algorithm is any well-defined computational procedure that takes some value, or set of values, as input and produces some value, or set of values, as outputs.

An algorithm is thus a sequence of computational steps that transform the input into the output.

Analyzing Algorithms

Analyzing an algorithm generally means predicting the resources that the algorithm requires. Some of the resources that are of concern are: memory, communication bandwidth, or computer hardware. Most important is computational time that we want to measure.

The space complexity of a program is the amount of memory it needs to completion.

The time complexity of a program is the amount of computer time it needs to run to completion.

Chapter 2: The Stack and Queue

2.1 Stack

The stack is an ordered collection of items into which items may be inserted and from which items may be deleted from one end which is known as top of the stack i.e. a single end of the stack is designated as the stack top.

So, the above definition specifies that-

- New items are inserted on the top of the stack and
- Deleted from the top of the stack.

This means that data can be added or removed only from top of the stack. So the stack is called a last-in-first-out (LIFO) list.

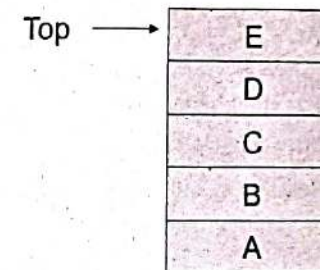


Figure 2.1: Stack

2.2 Stack as an ADT

An ADT is the mathematical model that contains set of values and functions that operates on those values without specifying the detail of those functions. The specification of stack ADT is given below.

Abstract Data Type Stack

{

Instances: ordered finite collections of zero or more elements from a given base type; the right extreme of the sequence is the top.

Operations

CreateStack

Input: nothing

Output: a new stack data object containing the empty sequence

DeleteStack

Input: an existing stack data object

Output: nothing

Effect: destroys the object given in input

EmptyStack

Input: a stack data object

Output: boolean value; TRUE if the stack contains the empty sequence, FALSE otherwise.

StackSize

Input: a stack data object

Output: an integer value identifying the length of the sequence currently stored in the input data object.

StackTop

Input: a stack data object

Output: the element in position in the top position of the sequence stored in the stack.

PrintStack

Input: a stack data object

Output: prints on the screen the sequence of elements currently stored in the input data object.

Push

Input: a stack data object; an element of the base type

Output: a boolean indicating whether the operation was successful.

Effect: the element is added to the sequence in the stack, in the top position

Pop

Input: a stack data object

Output: a boolean indicating whether the operation was successful.

Effect: the element in the top position in the sequence in the stack is removed.

}

In the above ADT definition

1. StackSize used to compute the size of a stack (i.e., the number of elements present in a given stack)

- Preconditions: receives a stack as input
- Postconditions: produces a number representing the count of the elements in the stack

2. EmptyStack used to verify whether a given stack is empty or not

- Preconditions: receives a stack as input
- Postconditions: produces true (i.e., 1 in C) if the stack is empty, false (i.e., 0 in C) otherwise

3. StackTop returns the information which is currently stored as top element in the stack.

- Preconditions: receives a stack s
- Postconditions: if the stack is not empty, then it returns the first element in the stack s, otherwise it gives an error

4. Push used to insert a new element as the top of the stack.

- Preconditions: receives as input a stack s, and an element x

- Postconditions: if x is of the correct type (i.e., it has the same type as all the other elements in s), and then the stack is modified introducing the element x in the first position of the stack; otherwise it returns an error

5. Pop removes the top element from the stack.

- Preconditions: receives as input a stack s
- Postconditions: if the stack is not empty, then the stack is modified by removing the element in first position in s ; otherwise it returns an error.

2.3 Operations on Stack

The possible operations on stack are

1. Create empty stack.
2. Isfull: To check whether the stack is full or not.
3. Isempty: To check whether the stack is empty or not.
4. Push operation: To push element in the stack (if stack is not full in case of implementing stack using array).
5. Pop operation: To access and remove the top element of the stack.
6. Print the entire stack.

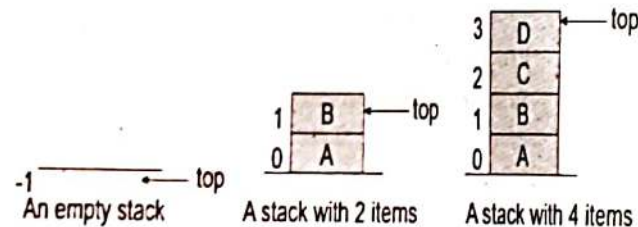


Figure 2.2: Stack with elements

Algorithm to check whether the stack is full or not

In stack to check whether the stack is full or not, we use Isfull function. This function returns true if the stack is full and returns false otherwise. This function takes pointer to stack as an argument.

Algorithm

1. Is full (stack s)
2. If the top of the stack is equal to $(\text{maxsize} - 1)$ return true
3. Otherwise return false

C Function

```
int full (stack *s)
{
    If (s->top == maxsize-1)    // Stack is Full
        return(1);
    else
        return(0);
}
```

Algorithm to Check whether stack is empty or not

To check the stack is empty or not, we use empty function. This function takes stack variable as argument and returns true if the stack is empty and false otherwise.

Algorithm

1. Isempty (stack s)
2. If the top of the stack is equal to -1 return true
3. Otherwise return false

C Function

```
int empty (stack *s)
{
    If (s->top == -1)    //Stack is Empty
        return (1);
    else
        return (0);
}
```



```
return (0);
```

Push operation on stack

This function is used to insert an element into the top of the stack.

Algorithm

1. Check whether the stack is full or not
2. If true
 Display stack is full
3. Else
 - a. Increment the top of the stack
 - b. Push an element into the top

C Function

```
Void push (stack *s, int x)
```

```
{  
    If (s->top == maxsize -1)  
    {  
        Printf("Stack is full");  
        Exit(0);  
    }  
    Else  
    {  
        s->top++;  
        s->n[s->top]=x;  
    }  
}
```

Pop operation on stack

This function is used to remove the top element from the stack.

Algorithm

1. Check whether the stack is empty or not
2. If true
 - a. Display stack is empty
 - b. Exit
3. Else
 - a. Remove top element from the stack
 - b. Decrease top by 1
 - c. Return top element

C Function

```
Int pop (stack *s)
```

```
{  
    int x;  
    if(s->top == -1)  
    {  
        Printf("Stack is empty");  
        exit(0);  
    }  
    else  
    {  
        x = s->n[s->top];  
        s->top--;  
    }  
    return x;  
}
```

Stack implementation

Stack can be implemented as

- Array implementation (Static implementation)
- Linked list implementation using pointer (Dynamic implementation)

Program 1: Array Implementation of Stack.

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
#include<stdlib.h>
#define N 5
struct stack
{
    int n[N];
    int top;
};

void push(struct stack *s, int val);
void pop(struct stack *s);
void show(struct stack *s);
void tos(struct stack *s);
void main()
{
    int c, val;
    struct stack s;
    s.top=-1;
    do
    {
        printf("\n\t\t STACK IMPLEMENTATION\n\t\t USING ARRAY");
```

```
printf("\n\n Enter 1->Push\t2->Pop\t3->Show\n\t4->Top of stack\t5->Exit\n");
```

```
scanf("%d",&c);
```

```
switch(c)
```

```
{
```

```
case 1:
```

```
{
```

```
printf("\n Enter number to\n push\n");
```

```
scanf("%d",&val);
```

```
push(&s,val);
```

```
break;
```

```
}
```

```
case 2:
```

```
{
```

```
pop(&s);
```

```
break;
```

```
}
```

```
case 3:
```

```
{
```

```
show(&s);
```

```
break;
```

```
}
```

```
case 4:
```

```
{
```

```
tos(&s);
```

```
break;
```

```

    }
    case 5:
        exit(1);
    default:
        printf("\n Wrong option");
        getch();
        break;
    }
} while(c!=5);
}

```

```

void push(struct stack *s, int val)
{
    if(s->top==N-1)
    {
        printf("\n Stack is full");
        getch();
        exit(1);
    }
    else
    {
        s->top++;
        s->n[s->top]=val;
        printf("\n %d is pushed", s->n[s->top]);
        getch();
    }
}

```

```

void pop(struct stack *s)
{
    if(s->top== -1)
    {
        printf("\n Stack is empty, underflow");
        getch();
        exit(1);
    }
    else
    {
        printf("\n Pop value is %d", s->n[s->top]);
        s->top--;
        getch();
    }
}

```

```

void show(struct stack *s)
{
    if(s->top== -1)
    {
        printf("\n Stack is empty, Press any key..");
        getch();
        exit(1);
    }
    for(int i = s->top; i>=0; i--)
    {
        printf("\n%d", s->n[i]);
    }
}

```



```

    getch();
}
void tos(struct stack *s)
{
    if(s->top==1)
    {
        printf("\n Stack is empty, underflow");
        getch();
        exit(1);
    }
    else
    {
        printf("\n Top value is %d", s->n[s->top]);
        getch();
    }
}

```

2.4 Expression

Any arithmetic expression can be represented in three different way : Infix, Prefix and Postfix.

1. Infix expression

It is the traditional way of writing arithmetic expression where an operator is placed in between its two operands that means <operand1 operator operand2>

Example: $a+b$, $a-b$, a/b , $a\%b$, etc.

2. Prefix expression

In prefix notation, an operator is placed before its two operands. It is also known as Polish notation. That means <operator operand1 operand2>

Example: $+ab$, $-ab$, $/ab$, $\%ab$, etc.

3. Postfix expression

In postfix notation, an operator is placed after the operand. It is also known as reverse Polish notation. That means <operand1 operand2 operator>

Example: $ab+$, $ab-$, $ab/$, $ab\%$, etc.

Infix	Postfix
$A + B - C$	$AB + C -$
$(A + B) \times (C - D)$	$AB + CD - \times$
$A \$ B \times C - D + E / F / (G + H)$	$AB \$ C \times D - EF / GH + / +$
$((A + B) \times C - (D - E)) \$ (F +$	$AB + C \times DE - - FG + \$$
$A - B / (C \times D \$ E)$	$ABCDE \$ \times / -$

2.5 Applications of Stack

- Reversing string
- Balancing symbols
- Infix expression to postfix conversion
- Infix expression to prefix conversion
- Evaluation of a postfix expression

2.5.1 Reversing string

Stack is used to reverse a given string.

Algorithm

1. Make an empty stack S
2. For each character ch in the input string
 - a. Push ch in S
3. While S is not empty

- ch = Pop a character from S
- Print ch

2.5.2 Balancing symbols

Stack is used to check whether the given expression is valid or not by checking the number of opening and closing braces used in the given expression.

Algorithm

- Scan one character at a time from left to right from the given expression.
- If the scan character is [or { or (push it into the stack.
- If scan characters are other than braces leave it and scan remaining characters.
- If the scanned character is) or } or] pop the matching (or { or [character from the top of the stack.
- After all character scanned,
 - If the stack is empty, the given expression is valid.
 - Else given expression is not valid.

Example 1. Check whether the given expression is balanced or not: $\{x+(y-[a+b])\times c-[(d+e)]\}$.

Solution:

Step	Scan char	Opstack
1	{	{
2	X	{
3	+	{
4	(	{{
5	Y	{{
6	-	{{

7	[	{{{
8	A	{{{
9	+	{{{
10	B	{{{
11	]	{{
12	)	{
13	x	{
14	C	{
15	-	{
16	[	{{
17	(	{{{
18	D	{{{
19	+	{{{
20	E	{{{
21	)	{{
22	]	{
23	}	Null

Here, in this example after the scanning of all characters, the stack is empty. So the expression is balanced.

2.5.3 Infix Expression to Postfix Conversion

Algorithm

- Scan one character at a time from left to right.
- Repeat while there is data.
 - If '(' push it into the stack

- b. If operand, Append it into postfix string
- c. If operator
 - i. If stack is empty push it onto opstack
 - ii. If not repeat till precedence of top of stack (tos) $\geq$ scan character
 - iii. Pop tos and append into postfix string
 - iv. Push scan character into opstack
- d. If ')' is found
 - i. Pop and append to postfix until matching '(' is found and ignore both.
3. Pop and append to postfix until stack is empty.
4. Return.

Examples:

1. Convert following infix expression to postfix expression:
 $(A+B \times C - D/E)$.

Solution:

S.N.	Scan character	Postfix string	Opstack
1	(		(
2	A	A	(
3	+	A	(+
4	B	AB	(+
5	$\times$	AB	(+ $\times$
6	C	ABC	(+ $\times$
7	-	ABC $\times$	(-)
8	D	ABC $\times$ +D	(-)
9	/	ABC $\times$ +D	(-/
10	E	ABC $\times$ +DE	(-/

11	)	ABC $\times$ +DE/-	NULL
12	NULL	ABC $\times$ +DE/-	NULL

So, the resulting postfix expression is: ABC $\times$ +DE/-

2. Convert following infix expression to postfix expression:
 $A+(B \times C - (D/ESF) \times G) \times H$.

Solution:

SN	Scan character	Postfix string	Opstack
1	A	A	
2	+	A	+
3	(	A	+(
4	B	AB	+(
5	$\times$	AB	+($\times$
6	C	ABC	+($\times$
7	-	ABC $\times$	+(-
8	(	ABC $\times$	+(-(
9	D	ABC $\times$ D	+(-(
10	/	ABC $\times$ D	+(-(/
11	E	ABC $\times$ DE	+(-(/
12	\$	ABC $\times$ DE	+(-(/ \$
13	F	ABC $\times$ DEF	+(-(/ \$
14	)	ABC $\times$ DEFS/	+(-
15	$\times$	ABC $\times$ DEFS/	+(- $\times$

16	G	ABC×DEFS/G	+(-×
17	)	ABC×DEFS/G×-	+
18	×	ABC×DEFS/G×-	++
19	H	ABC×DEFS/G×-H	++
20	NULL	ABC×DEFS/G×-H×+	NULL

So, the resulting postfix expression is:

ABC×DEFS/G×-H×+

3. Convert following infix expression to postfix expression:
 $A+B-(C \times D/E+F)-G \times H$. [2073 Bhadra]

Solution:

SN	Scan character	Postfix string	Opstack
1	A	A	
2	+	A	+
3	B	AB	+
4	-	AB+	-
5	(	AB+	-(
6	C	AB+C	-(
7	×	AB+C	-(×
8	D	AB+CD	-(×
9	/	AB+CD×	-(/
10	E	AB+CD×E	-(/
11	+	AB+CD×E/	-(+
12	F	AB+CD×E/F	-(+
13	)	AB+CD×E/F+	-

14	-	AB+CD×E/F+ -	-
15	G	AB+CD×E/F+ -G	-
16	×	AB+CD×E/F+ -G	-×
17	H	AB+CD×E/F+ -GH	-×
18	NULL	AB+CD×E/F+ -GH×-	NULL

So the resulting postfix expression is:

AB+CD×E/F+ -GH×-

4. Convert following infix expression to postfix expression:
 $a^b \times c - d - e - f / e / (g - h)$. [2076 Bhadra]

Solution:

SN	Scan character	Postfix string	Opstack
1	A	A	
2	^	A	^
3	B	ab	^
4	×	ab^	×
5	C	ab^c	×
6	-	ab^c×	-
7	D	ab^c×d	-
8	-	ab^c×d-	-
9	E	ab^c×d-e	-
10	-	ab^c×d-e-	-
11	F	ab^c×d-e-f	-
12	/	ab^c×d-e-f	-/

13	E	$ab^c \times d - e - fe$	-/
14	/	$ab^c \times d - e - fe/$	-/
15	(	$ab^c \times d - e - fe/$	-(/
16	G	$ab^c \times d - e - fe/g$	-(/
17	-	$ab^c \times d - e - fe/g$	-(/-
18	H	$ab^c \times d - e - fe/gh$	-(/-
19	)	$ab^c \times d - e - fe/gh-$	-/
20	NULL	$ab^c \times d - e - fe/gh-/-$	NULL

So, the resulting postfix expression is:

$ab^c \times d - e - fe/gh-/-$

2.5.4 Infix Expression to Prefix Conversion

Algorithm

1. Reverse the given input string.
2. Scan one character at a time from left to right.
3. If it is operand, add it to the postfix string.
4. If it is closing parenthesis, push it to opstack.
5. If it is an operator, then
 - a. If stack is empty, push operator on stack.
 - b. If the top of stack is closing parenthesis, push operator on stack.
 - c. If it has same or higher priority than the top of stack, push operator on postfix string.
 - d. Else pop the operator from the stack and add it to postfix string, and push the scanned operator onto stack.
6. If it is an opening parenthesis, pop operator from stack and add them to postfix string until a closing parenthesis is encountered. Pop and discard the closing parenthesis.
7. If there is more input go to step 2.

8. If there is no more input, pop the remaining operators and add them to postfix string.
9. Reverse the postfix string to get required Prefix expression.

Examples:

1. Convert following infix expression to prefix expression: $(A+B \times C)$.

Solution:

Reversing the given expression $(A+B \times C)$ we get, $) C \times B + A ($

S.N	Scan character	Postfix string	Opstack
1	)		)
2	C	C	)
3	$\times$	C	) $\times$
4	B	CB	) $\times$
5	+	CB $\times$	) $+$
6	A	CB $\times$ A	) $+$
7	(	CB $\times$ A $+$	

So, the postfix expression is $CB \times A +$

By reversing this postfix expression, we get required prefix expression: $+A \times BC$

2.5.5 Evaluation of a Postfix Expression

Algorithm

1. Scan one character at a time from left to right.
2. If scan character is an operand, push it onto the stack.
3. If an operator, pop 2 operand from the stack, the first one popped is called operand2 and the second one is called operand1.
4. Perform $result = operand1 \text{ operator } operand2$

5. Push result onto the stack.
6. Repeat the above steps till the end of the input.

Example:

Evaluate the following postfix expression: $AB+C \times DE - FG + \$$.
Where $A=1, B=2, C=3, D=4, E=3, F=2$, and $G=1$.

Solution:

SN	Scan character	Operand1	Operand2	Opstack
1	A(1)			1
2	B(2)			1,2
3	+	1	2	3
4	C(3)			3,3
5	$\times$	3	3	9
6	D(4)			9,4
7	E(3)			9,4,3
8	-	4	3	9,1
9	-	9	1	8
10	F(2)			8,2
11	G(1)			8,2,1
12	+	2	1	8,3
13	\$	8	3	512

Hence the result of the above postfix expression is 512.

2.6 Queue

A queue is an ordered collection of items in which all insertions are made at one end (the rear), and all deletions are made at the other end (the front).

2.6.1 Enqueue and Dequeue Operations

Enqueue – Inserts an item at the rear of the queue. Other names – Add, Insert.

Dequeue – Deletes an item from the front of the queue. Other names – Delete, Remove, Serve.

Since it is always the first item to be put into the queue that is the first item to be removed, a queue is a first-in, first-out (FIFO) list. Front and rear are also called head and tail respectively.

2.7 Implementation of Queue

Queue can be implemented in two ways:

i) Array implementation

- A linear array with two indices always increasing.
- A circular array with front and rear indices and one position left vacant.

ii) Linked list

- A linked list with pointers to the front and rear nodes.

2.8 Linear Queue

For all type of array implementation, we set up an array to hold the items in the queue. In linear implementation, we use two indices to keep track the front and the rear of the queue. To enqueue an item, we increase the rear by one and put the item in that position. To dequeue an item, we take it from the position at the front and then increase front by one.

Enqueue and dequeue operations are shown below in the queue of size 5.

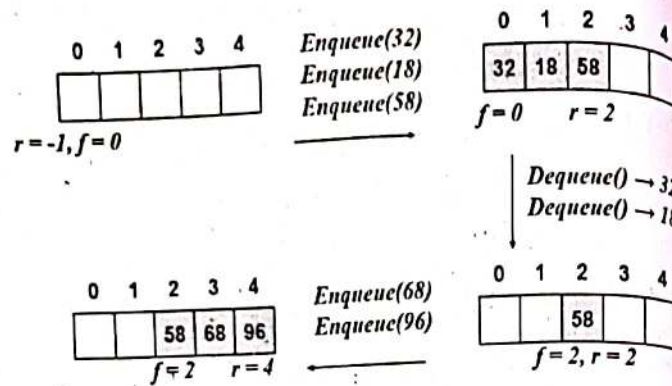


Figure 2.3: Enqueue and dequeue operations in a linear queue

Algorithm

1. Declare and initialize necessary variables: $\text{front} = 0, \text{rear} = -1, \text{maxsize}, \text{item}, \text{queue}[\text{maxsize}]$.
2. For enqueue operation:
 - If $\text{rear} \geq \text{maxsize} - 1$
 - Print "queue is full"
 - Else
 - Read item from user
 - $\text{rear} = \text{rear} + 1$
 - $\text{queue}[\text{rear}] = \text{item}$
3. For next enqueue operation, repeat step 2.
4. For dequeue operation:
 - If $\text{front} > \text{rear}$
 - Print "queue is empty"
 - Else
 - $\text{item} = \text{queue}[\text{front}]$
 - $\text{front} = \text{front} + 1$
 - display item.
5. For next dequeue operation, repeat step 4.

Program 2: Linear Queue Implementation.

```
#include <stdio.h>
#include <conio.h>
#include <process.h>
#include <stdlib.h>

#define N 10 //Size of queue is 10

//Definition of structure
struct queue
{
    int n[N]; //Data holder
    int front, rear; //Front and Rear pointer
};

//Function declaration
void enqueue(struct queue *);
void dequeue(struct queue *);
void show(struct queue *);
void front(struct queue *);
void makeempty(struct queue *);

int main()
{
    int c;
    struct queue q; //variable of type queue is declared (q)
    makeempty(&q); //Initialization of the queue
    do
    {
        printf("\n\t\t LINEAR QUEUE IMPLEMENTATION\n\t\t USING ARRAY");
```

```

printf("\n\n1->Enqueue\n2->Dequeue\n3->Show all\n4->Front\n5->Make empty\n6->Exit\n");
printf("\n\nEnter your choice: ");
scanf("%d",&c);
switch(c)
{
    case 1: enqueue(&q); //call function enqueue
            break;
    case 2: dequeue(&q); //call function dequeue
            break;
    case 3: show(&q); //call function show
            break;
    case 4: front(&q); //call function front
            break;
    case 5: makeempty(&q); //call makeempty
            break;
    case 6: exit(1); //call function exit
    default: printf("\n Wrong option");
            getch();
            break;
}
} while(c!=6);
}

void enqueue(struct queue *q)
{
    int val;
    if(q->rear==N-1) //check for overflow
        printf("\n Queue is full");
    else

```

```

{
    printf("\nEnter the number to enqueue\n");
    scanf("%d",&val);
    q->rear++; //rear is incremented
    q->n[q->rear]=val; //value is enqueued
    printf("\n %d is enqueued", val);
}
getch();
}

void dequeue(struct queue *q)
{
    if(q->rear<q->front) //check for underflow
        printf("\n Queue is empty");
    else
    {
        printf("\n Dequeued value is %d", q->n[q->front]);
        q->front++; //front is incremented
    }
    getch();
}

void show(struct queue *q)
{
    int i;
    if(q->rear<q->front) //check for empty
        printf("\n Queue is empty, Press any key..");
    else
    {

```

```

printf("\nThe numbers in the queue are:\n");
for(i=q->front; i<=q->rear; i++)
    printf("%d", q->n[i]);
}
getch();
}

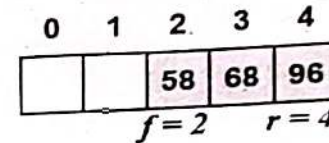
void front(struct queue *q)
{
    if(q->rear<q->front)    //check for empty
    {
        printf("\n Queue is empty");
        getch();
    }
    else    //if not empty, print the value at front
    {
        printf("\n The value at front is %d", q->n[q->front]);
        getch();
    }
}

void makeempty(struct queue *q) //makes the queue empty
{
    q->front=0;    //reinitialize front to 0 and
    q->rear=-1;    //rear to -1
}

```

Problems with linear queue:

Both the rear and front indices are increased but never decreased. As items are removed from the queue, the storage space at the beginning of the array is discarded and never used again.



This queue is considered full, even though the space at beginning is vacant.

2.9 Circular Queue

In circular queue, array is viewed as a circle rather than a straight line. Imagine the first element of the array (i.e., the element at index 0) as immediately following its last element. If incrementing either rear or front causes it to go past the array, reset its value to zero. With this approach, we can always insert an item unless the array is fully occupied.

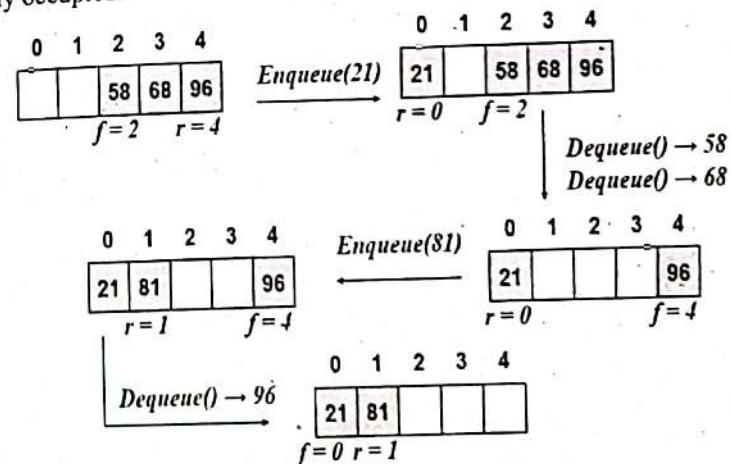
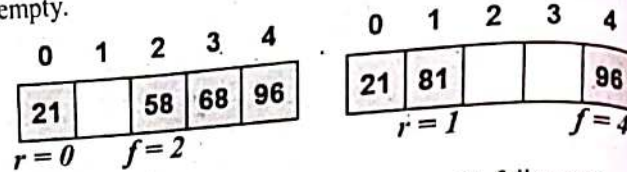


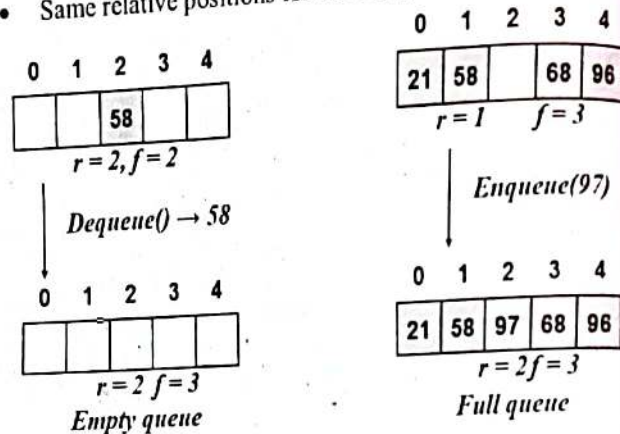
Figure 2.4 : Enqueue and dequeue operations in a circular queue.

Checking Boundary Conditions

- The condition $rear < front$ does not hold for empty queue. In figure below, for both cases, $rear < front$ but the queue is not empty.



- Same relative positions for an empty and a full queue.



To resolve this problem, leave one empty position in the array so that the queue is considered full when the rear index has moved within two positions ahead of front.

For convenience, initialize both rear and front to same value for empty queue.

We are using a circular array with front and rear indices and one position left vacant.

Algorithm

- Declare and initialize necessary variables: $front = 0$, $rear = 0$, $maxsize$, $item$, $queue[maxsize]$.
- For enqueue operation :
If $front = (rear+1) \% maxsize$

Print "queue is full"

Else

Read item from user

$queue[rear] = item$

$rear = (rear + 1) \% maxsize$

- For next enqueue operation, repeat step 2.

- For dequeue operation :

If $front = rear$

Print "queue is empty"

Else

$item = queue[front]$

$front = (front + 1) \% maxsize$

display item.

- For next dequeue operation, repeat step 4.

Program 3: Circular Queue Implementation.

```
#include<stdio.h>
#include<conio.h>
#define n 5
struct queue
{
    int que[n];
    int front;
    int rear;
};
int d,choice,x,flag=0;

int menu(void)
```

```

printf("\n\n ***MENU***\n");
printf("\n1.Insert\n");
printf("\n2.Delete\n");
printf("\n3.Display\n");
printf("\n4.Exit\n\n");
printf("\n\nEnter your choice:");
scanf("%d",&choice);
return 0;
}

void insert(struct queue *cq)
{
    if(cq->front==(cq->rear+1)%n)
    {
        printf("\nQueue is full");
        getch();
    }
    else
    {
        printf("\n\nEnter the element to insert:");
        scanf("%d",&x);
        cq->que[cq->rear]=x;
        cq->rear=(cq->rear+1)%n;
    }
}

void del(struct queue *cq)
{

```

```

int d;
if(cq->rear==cq->front)
{
    cq->rear=0;
    cq->front=0;
    printf("Queue is empty");
}
else
{
    d=cq->que[cq->front];
    cq->front=(cq->front+1)%n;
    printf("Deleted element is %d",d);
}
getch();
}

void display(struct queue *cq)
{
    int i;
    i=cq->front;
    if(cq->front==cq->rear)
        printf("\nNo element in queue");
    else
    {
        printf("Elements are:\n");
        for(i!=cq->rear;i=(i+1)%n)
            printf("%d\t",cq->que[i]);
    }
}

```

```

    getch();
}

int main()
{
    struct queue cirque;
    cirque.front=cirque.rear=0;
    do
    {
        menu();
        switch(choice)
        {
            case 1: insert(&cirque);
                    break;
            case 2: del(&cirque);
                    break;
            case 3: display(&cirque);
                    break;
            case 4: flag=1;
                    break;
        }
    } while(flag==0);
}

```

2.10 Queue as an Abstract Data Type

The representation of a queue as an abstract data type is straightforward. We use eltype to denote the type of the queue element and parameterize the queue type with eltype.

/*value definition*/

```
abstract typedef <<eltype>> QUEUE(eltype);
```

```

/*check for empty*/
abstract empty (q)
QUEUE(eltype) q;
postcondition      empty == (len(q) == 0);
/*Remove item*/
abstract eltype remove(q)
QUEUE(eltype) q;
precondition      empty(q) == FALSE;
postcondition      remove == first(q');
                  q == sub(q', 1, len(q') - 1);

/*insert item*/
abstract insert(q, item)
QUEUE(eltype) q;
eltype item;
postcondition      q == q' + <item>;

```

2.11 Priority Queues

In stacks and queues, the elements are ordered are based on the sequence in which they have been inserted. A priority queue is a data structure in which the intrinsic ordering of the elements determines the results of its basic operations.

Types of priority queues:

- **Ascending (minimum) priority queue**
 - A collection of items into which items can be inserted arbitrarily but only the smallest item can be removed.
- **Descending (maximum) priority queue**
 - A collection of items into which items can be inserted arbitrarily but only the largest item can be removed.

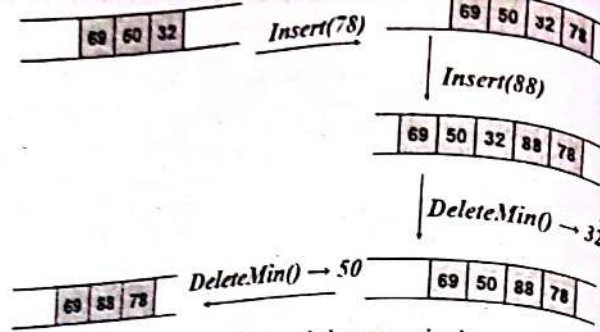


Figure 2.5: A minimum priority queue

Applications of priority queue:

In a time-sharing computer system, a large number of tasks may be waiting for the CPU. Some of these tasks have higher priority than others. The set of tasks waiting for the CPU forms a priority queue.

2.12 Double-Ended Queue (Deque)

A deque (double-ended queue) is a data structure consisting of a list of items, on which the following operations are possible:

Push(D, x) – Insert item x on the front of deque D.

Pop(D) – Remove the front item from deque D and return it.

Inject(D, x) – Insert item x on the rear end of deque D.

Eject(D) – Remove the rear item from deque D and return it.

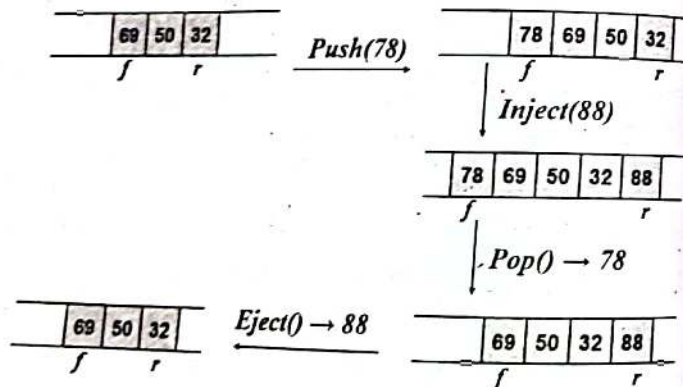


Figure 2.6: Double-ended queue

Chapter 3: List

3.1 Introduction

A list is a data structure that implements an ordered collection of values. Each value in the list is called an item or an element of the list. It differs from stack and queue in such a way that insertions and deletions can be made at any positions in the list.

3.2 Array Implementation of Lists

A structure is used for representing nodes of the list. Structure consists of two data items: information (info) and next. Information consists of the data on a particular node, and the next consists of the value for the next node being represented. Value of next is NULL (or -1) for the last node.

/* Declaring the list */

Define max 10

struct nodetype

{

int info, next;

};

struct nodetype node [max];

Example 1: This is a contiguous list. Here, the first node is at location 0 of an array. Next field of first node contains address of second node and so on.

	Info	Next
0	A	1
1	B	2
2	C	3
3	D	4
4		

Figure 3.1 : Array of nodes

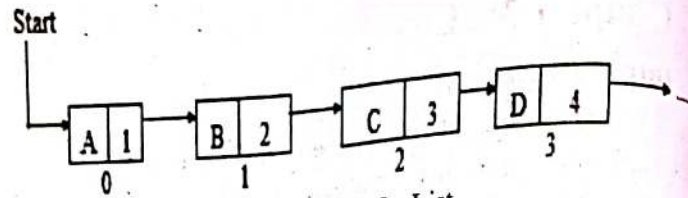


Figure 3.2 : List

Example 2: Here four lists are shown. List 1 has start value 1 i.e. list starts from node [1]. Every next value on the successive node gives the address of next node. If the next consist NULL value, then the list ends.

	Info	Next
0	20	-1
1	32	2
2	30	7
3	40	5
4	90	3
5	75	0
6	54	-1
7	39	6
8	70	9
9	83	0
10	95	4
11	45	-1
12	59	9
13	29	6
14	10	2
15	25	..

Figure 3.3: Array of nodes containing four lists

The elements of four lists are shown below:

List 1: (Start = 1)

Elements are: 32, 30, 39, 54.

List 2: (Start = 10)

Elements are: 95, 90, 40, 75, 20.

List 3: (Start = 8)

Elements are: 70, 83, 20.

List 4: (Start = 13)

Elements are: 29, 54.

3.3 List Initialization

Initialization of list elements can be done as follows:

/ Initialization of list */*

avail=0

for (i=0; i< max -1; i++)

node [i].next = i+1;

node [max -1].next = -1;

The global variable avail is used to point the available list. Conceptually available is a pool of node. In this case its value 0 means the very first node of the available list. All node are ordered sequentially by node[i] pointing to the node[i+1]. The next field of last node is set to -1 (NULL) value.

The ordering of the node is explicit and hence can be ordered in any other way as well. We can also point node [0] to node [8], node [8] to node [13] and so on.

3.4 Operations of List

getnode: creates the new node

freenode: destroys the node

insafter: Insert an item into the list after a node

delafter: delete an item into the list after a node
 When a node is required for particular list, it is obtained from available list. When the node is not required, it has to be returned to the available list.

getnode

getnode is a function that removes a node from the available list. For insertion of new elements, a node is required which is provided by the following getnode function:

/* getting the available node from array*/

```
int getnode (void)
{
    int p;
    if (avail == -1) /* empty array*/
    {
        printf ("underflow");
        exit (1);
    }
    p = avail;
    avail = node[avail].next;
    return p;
}
```

freenode

When the node is not required, it has to be returned to the available list. Following freenode function is used to return the node to available list.

```
void freenode (int p)
{
    node [p].next = avail;
    avail = p;
    return;
}
```

insafter

Let p be any node (ensure p is not null) and q be the new node having its value x and is required to insert after node p. The insafter function is used to insert the node q after any node p.

void insafter (int p, int x)

```
{
    int q;
    if (p == -1)
    {
        printf ("Invalid operation");
        exit (1);
    }
    q = getnode ()
    node [q].info = x;
    node [q].next = node [p].next;
    node [p].next = q;
}
```

delafter

The function delafter deletes the node after any node p. Its content is stored in x.

void delafter (int p, int *x)

```
{
    int q;
    if ((p == -1) || (node [p].next == -1))
    {
        printf ("Invalid deletion");
        exit (1);
    }
}
```



```

    }
    q = node [p].next;
    *x = node [q].info;
    node [p].next = node [q].next;
    freenode [q];
}

```

3.5 Static vs Dynamic List

Static List	Dynamic List
1. Static list is an array implementation.	1. Dynamic list is linked list implementation.
2. Size of list is fixed	2. Size of list is dynamic
3. It is suitable if less nodes are available in a list i.e. for smaller list.	3. It is suitable if large number of nodes are available in a list i.e. for larger list.
4. Programming implementation is easy.	4. Programming implementation is complex.
5. It consume less time for computation.	5. It consume more time for computation.

3.6 Queue as a List in C

Queue can be implemented as a linear list. It can be illustrated in both static as well as dynamic ways. The static implementation uses array whereas the dynamic implementation uses linked list.

Static (Array) Implementation of Queue as List

First, we define a structure representing each node containing info and next (pointer to the next node).

```

/* List definition */
struct nodetype
{

```

```

    int info, next;

```

```

};
struct nodetype node [max];
Initialization of list elements can be done as follows:
/* Initialization of list */

```

```

avail = 0
for ( i=0; i< max -1; i++)
    node [i].next = i+1;
node [max -1].next = -1;

```

For insertion of new elements, a node is required which is provided by the getnode function.

```

/* getnode function */

```

```

int getnode ( )
{

```

```

    int p;
    if (avail == -1)
    {
        printf ("Overflow");
        exit (1);
    }

```

```

    p= avail;
    avail = node[ avail].next;
    return p;
}

```

If node is not required in a list, freenode function is used to return the node to available list.

```

/* freenode function */

```

```

void freenode (int p)
{
    node [p].next = avail;
    avail = p;
    return;
}

```

/* queue implementation */

struct queue

```

{
    int front = -1, rear = -1;
};

```

To check if the queue is empty, we use the function:

/* empty check */

int empty (struct queue *p)

```

{
    if (p -> front == -1)
        return 1;
    else
        return 0;
}

```

To insert an element into a queue (enqueue), we use:

/* Enqueue operation */

void insert (struct queue *pq, int x)

```

{
    int p;
    p = getnode ();
    node[p].info = x;
    node[p].next = -1;
}

```

```

if (pq -> rear == -1)
    pq -> front = p;

```

else

```

    node [ pq -> rear ].next = p;
    pq -> rear = p;

```

To remove the first element (dequeue) and return it, we use the following function:

/* Dequeue operation */

int remove (struct queue *pq)

```

{
    int p, x;
    if (empty(pq))
    {
        printf ("underflow");
        exit(1);
    }
    p = pq -> front;
    x = node[p].info;
    pq -> front = node[p].next;
    if (pq -> front == -1)
    {
        pq -> rear = -1;
    }
    freenode(p);
    return x;
}

```

Question:

Explain array implementation of list with its limitations. [2069 Bhadra]

Answer:

The representation of a list as an array is classified as a static list. A structure is used to represent each element of the list. The structure contains two data items, info and next where info is the data and next is the pointer to the next node of the list. A pointer to a node is represented by the relative position of the node within the array. A static list is an array of this structure type.

However, there are many limitations of implementing a list using an array. Some of them are as follows:

- 1) Memory allocation for array of nodes is always done at the start of execution.
- 2) Another disadvantage is that no matter how many nodes we end up using, the allocated memory for the nodes remain allocated for the program throughout the execution of the program.
- 3) Even after the deletion of a node the space previously occupied that node is still reserved for the program. So, lots of memory is wasted.
- 4) We may have to shift all the elements of the array when we either insert an element or delete an element from the array.

Chapter 4: Linked Lists

4.1 Introduction

A list is a sequence of related data items. It can be an ordered collection of integer items like 3,40,75,98,100, or it can be a collection of structures like;

```
struct student {  
    char name[20];  
    char class[10];  
    int grade;  
};
```

with each structure containing a set of related data items.

Stacks and queues are special type of lists. We call them restricted list because insertions and deletions are restricted to occur in these structures only at the ends of a list.

A linked list is a series of data items with each item containing a pointer giving a location of the next item in the list. A linked list is a collection of nodes, where each node consists of two parts:

information: the actual element to be stored in the list.

link: one or two links that points to next and previous node in the list, also known as pointer.

The entire link list is accessed by an external pointer (say start) that points to the first node in the linked list.

Types of Linked List

- i. Singly linked list
 - Linear
 - Circular
- ii. Doubly linked list
 - Linear

- Circular

Array vs List

- The number of elements of an array is fixed when the array is created. In contrast, the number of elements in a list can be varied by adding or removing elements.
- The index of a component of an array is unique and fixed. In contrast, the position of an element in a list can vary over time, e.g. if we remove the first element of a list then the element which was previously second in the list now becomes the first.
- All components of an array can be accessed easily in constant time, but there is no statement that the same is true for the elements of a list.

Advantages of Linked List over Contiguous List

- A linked list is a dynamic data structure. Therefore, the primary advantage of linked lists over arrays is that linked lists can grow or shrink in size during the execution of a program. A linked list can be made just as long as required.
- Another advantage is that a linked list doesn't waste memory space. It uses the memory that is just needed for the list at any point of time. This is because it is not necessary to specify the number of nodes to be used in the list.
- The third and the more important advantage is that the linked lists provide flexibility in allowing the items to be rearranged efficiently; it is easier to insert or delete items by rearranging the links i.e. the items can be inserted randomly at any position without too much difficulty.

Limitations of Linked Lists

- Whenever we deal with a fixed length list, it would be better to use an array rather than a linked list.

- We must also note that a linked list will use more storage than an array with the same number of items. This is because each item has an additional link field.

4.2 Basic Operations in Linked List

Insertion

This operation creates a new node and inserts it into the list. A new node may be inserted:

- at the beginning of the list (insertHead)
- at the end of the list (insertTail)
- at the specified position (insertAfter, insertBefore, insertAtNthPosition)

If the list is empty, the node to be inserted will be the first node.

Deletion

This operation deletes a node (i.e. an item) from the linked list. A node may be deleted from the

- beginning of the list (deleteHead)
- end of the list (deleteTail)
- specified position (deleteAfter, deleteAtNthPos, etc.)

Traversing

The process of visiting all the nodes of the linked list from one end to the other is termed as traversing the list.

Searching

This operation checks if a particular item is present in the list.

Display

This operation is used to print the content of the list.

4.3 Singly Linked List

Singly linked list contains nodes with two fields; Information field and address or link field.

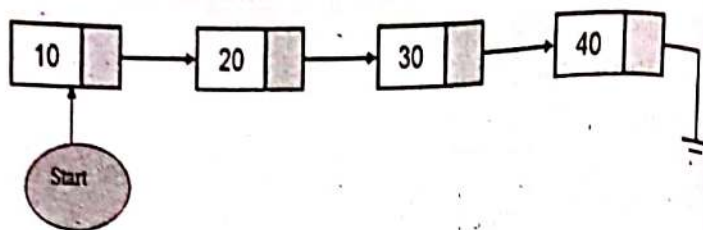


Figure 4.1: A singly linked list

- The next address field of the last node in the list contains a special value, known as null, which is not a valid address. This null pointer is used to represent the end of a list.
- The list with no nodes on it is called the empty list or null list.
- The nodes in a linked list are not stored contiguously in the computer's memory.
- We don't have to shift any element while inserting/deleting in the list.
- Memory for each node can be allocated dynamically whenever the need arises.
- The number of items in the list is only limited by the computer's memory.

1. Creation of Linked List

Linked list can be created as:

```
struct list
{
    int item;
    struct list *next;
};
typedef struct list node;
```

```
node *ptr, *start;
```

```
ptr = (node *) malloc (sizeof(node));
```

The struct declaration merely describes the format of the nodes and does not allocate storage. Here, the type of the information to be stored is 'int'. next is a pointer to the parent structure type. This type of structure is called self-referential structures. Storage space for a node is created only when the function malloc() is called in the statement.

```
ptr = (node *) malloc (sizeof(node)); ..... (1)
```

```
start = ptr; ..... (2)
```

This statement (1) obtains a piece of memory that is sufficient to store a node and (2) assigns its address to the pointer variable start. This start pointer indicates the beginning of the linked list.

2. Insertion

To insert a node in a list:

- allocate required memory to a node
- assign the data in its information field
- adjust the pointers to link the node at the required place

i) For inserting a new node at the beginning of the list

Algorithm

- Allocate memory for the new node.
- Assign value.
- If the list is currently empty, assign NULL to the link field of the new node. Otherwise, make the link field of the new node to point to the starting node of the linked list.
- Then, set the external pointer (which is pointing to the starting node) to point to the new node.

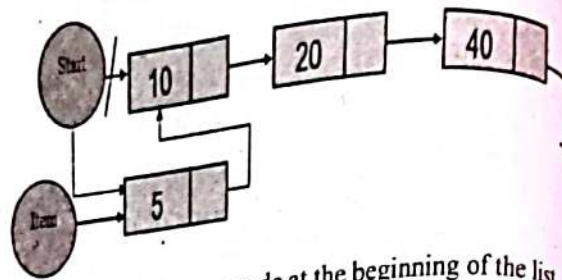


Figure 4.2: Inserting a new node at the beginning of the list

C Function

Void f_insert()

```
{
    node *item;
    item=(node *)malloc(sizeof(node));
    printf("Enter data to insert");
    scanf("%d",&item->info);
    if(start==NULL)
    {
        item->next=NULL;
        start=item;
    }
    else
    {
        item->next=start;
        start=item;
    }
}
```

ii) For inserting a new node at the end of the list

Algorithm

- Allocate memory for the new node.
- Assign value.

- Assign NULL to the link field of the new node.
- Traverse the list until last node is reached. Then, insert the new node after the last node by storing the address of the newnode (item) to link field of the last node.

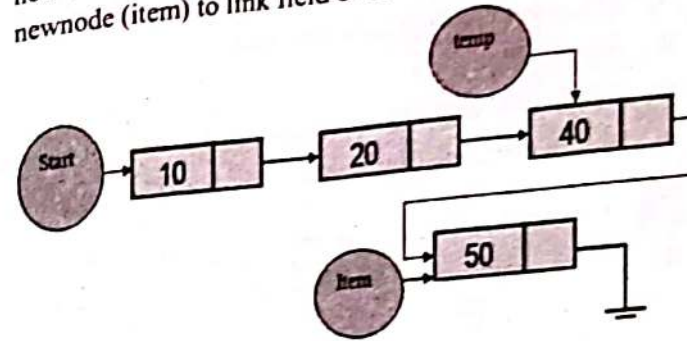


Figure 4.3: Inserting a new node at the end of the list

C Function

void e_insert()

```
{
    node *item,*temp;
    item=(node *)malloc(sizeof(node));
    printf("\nEnter data to insert:");
    scanf("%d",&item->info);
    item->next=NULL;
    if(start==NULL)
    start=item;
    else
    {
        temp=start;
        while(temp->next!=NULL)
            temp=temp->next;
    }
}
```


temp->next=item;

iii) For inserting a new node at specific position

Algorithm

- Allocate memory for the new node.
- Assign value.
- Assign next of the given node to the link field of the new node.
- Assign newnode (item) to the link field of the given node

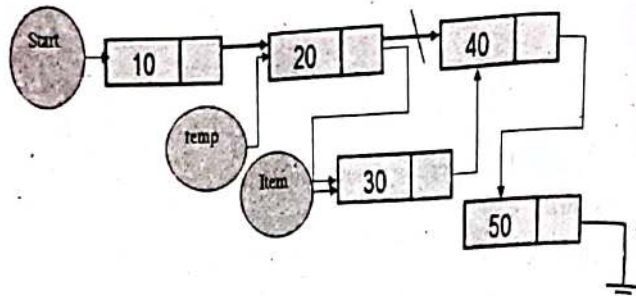


Figure 4.4: Inserting a new node at specific position

C Function

void n_insert()

{

```
int n, count=1;
node *ptr, *temp;
item=(node *)malloc(sizeof(node));
printf("\nEnter the position");
scanf("%d",&n);
printf("enter the data to insert");
scanf("%d",&item->info);
temp=start;
```

while(count<n)

{

temp=temp->next;

count++;

}

item->next=temp->next;

temp->next=item;

3. Deletions

To delete a node from a list:

- Find out the node to be deleted
- Adjust the pointers
- Free the node to be deleted

i) For deleting the first node

Algorithm

- If the list is empty, it is underflow.
- Else,
 - move the start pointer to the second node
 - free the first node

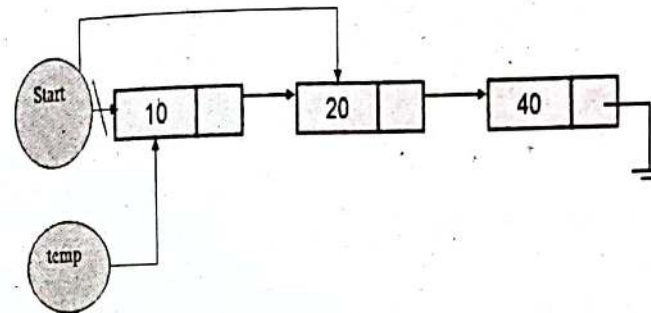


Figure 4.5: Deleting the first node

C Function

void f_del()

```
{
    node *ptr;
    if(start==NULL)
    {
        printf("\n Empty list");
        getch();
    }
    else if(start->next==NULL)
    {
        printf("\n Deleted element is %d", start->info);
        getch();
        free(start);
        start=NULL;
    }
    else
    {
        ptr=start;
        start=start->next;
        printf("\n Deleted element is %d", ptr->info);
        getch();
        free(ptr);
    }
}
```

ii) For deleting the last node

Algorithm

1. If the list is empty, it is underflow.

2. Else, traverse until the last node is encountered.
- Also keep track of the node, which is before the current node.
3. make the second last node to point to NULL and free the last node(i.e., ptr).

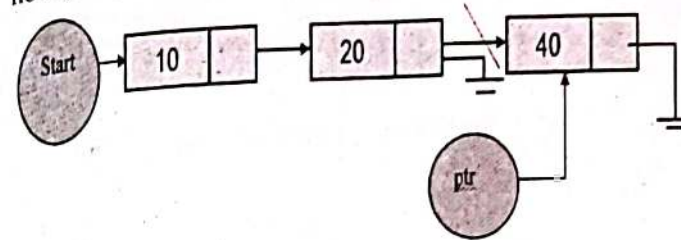


Figure 4.6: Deleting the last node

C Function

void e_del()

```
{
    node *ptr, *loc;
    if(start==NULL)
    {
        printf("\n Empty list");
        getch();
    }
    else if(start->next==NULL)
    {
        printf("\n Deleted item is %d", start->info);
        free(start);
        start=NULL;
    }
    else

```

```

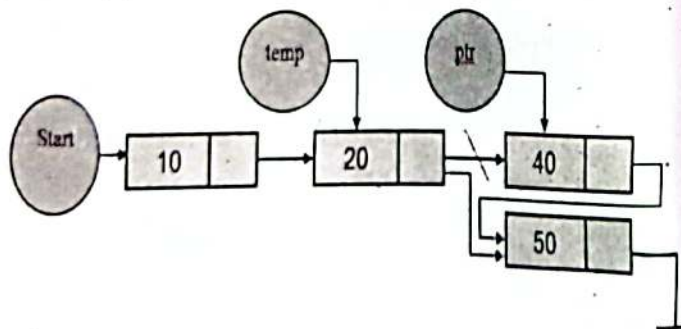
ptr=start;
while(ptr->next!=NULL)
{
    loc=ptr;
    ptr=ptr->next;
}
loc->next=NULL;
printf("\nDeleted element is %d",ptr->info);
getch();
free(ptr);
}

```

iii) For deleting a node at the specified position

Algorithm

1. If the list is empty, it is underflow
2. Else, traverse until the specified node is encountered (ptr)
 - Also keep track of the node, which is before the specified node.
3. Make the next of temp to point to next of ptr and free the node pointed by ptr.



C Function
void n_del()

```

{
    node *temp, *ptr;
    int n, count=1;
    if(start==NULL)
    {
        printf("\nEmpty list");
        getch();
    }
    else
    {
        printf("\n Enter a position");
        scanf("%d",&n);
        ptr=start;
        while(count<n)
        {
            count++;
            temp=ptr;
            ptr=ptr->next;
        }
        temp->next=ptr->next;
        printf("\nDeleted item is %d",ptr->info);
        getch();
        free(ptr);
    }
}

```


4. Traversing

Traversing a list and displaying information stored

As has been stated, list traversal is the process of visiting all the nodes in the list. Traversal is generally needed to print the items stored in each node of the list. The following function, traverse the given list and prints the item stored in each of them.

Algorithm

1. If the list is empty, print no element exist.
2. Else, traverse until the temp is not NULL (temp!=NULL)
-While traversing print the info part each time.

C Function

void display()

```
{  
    node *temp;  
    printf("\nElements of linked list are");  
    temp=start;  
    while(temp!=NULL)  
    {  
        printf("%d\t",temp->info);  
        temp=temp->next;  
    }  
    getch();  
}
```

5. Searching

Searching for an item in a list

In order to find an item in a given list, we visit the nodes one after another until we find the item.

Algorithm

1. Start with the first node in the list, as the temp node and keep track of count.
2. Check the info at the current node with the search item
If the items match, print the value with its position (counter).
Otherwise, move to the next node and repeat the process.
3. If no match is found till we reach the end of the list, print search unsuccessful.

C Function

void search()

```
{  
    node *temp;  
    int num,count=0;  
    printf("\n Enter a number to search");  
    scanf("%d",&num);  
    temp=start;  
    do  
    {  
        count++;  
        if(temp->info==num)  
        {  
            printf("\nThis number is at %d  
position",count);  
            getch();  
            temp=NULL;  
            return;  
        }  
    }  
    else
```

4. Traversing

Traversing a list and displaying information stored in nodes in the list. Traversal is the process of visiting all nodes in the list. Traversal is generally needed to print the info stored in each node of the list. The following function, traverse, given list and prints the item stored in each of them.

Algorithm

1. If the list is empty, print no element exist.
2. Else, traverse until the temp is not NULL (temp!=NULL).
-While traversing print the info part each time.

C Function

void display()

```
{
    node *temp;
    printf("\nElements of linked list are");
    temp=start;
    while(temp!=NULL)
    {
        printf("%d\t",temp->info);
        temp=temp->next;
    }
    getch();
}
```

5. Searching

Searching for an item in a list

In order to find an item in a given list, we visit the nodes one after another until we find the item.

Algorithm

1. Start with the first node in the list, as the temp node and keep track of count.
2. Check the info at the current node with the search item. If the items match, print the value with its position (counter). Otherwise, move to the next node and repeat the process.
3. If no match is found till we reach the end of the list, print search unsuccessful.

C Function

void search()

```
{
    node *temp;
    int num, count=0;
    printf("\n Enter a number to search");
    scanf("%d",&num);
    temp=start;
    do
    {
        count++;
        if(temp->info==num)
        {
            printf("\nThis number is at %d position",count);
            getch();
            temp=NULL;
            return;
        }
    }
    else
```

```

        temp=temp->next;
    }while(temp!=NULL);
    printf("\nThis number doesnot exist in list");
    getch();
}

```

6. Destroying

Destroying a list

When the list is not needed any more, we delete all the nodes of the list by releasing the memory occupied by them. Finally, list will be empty.

For this, delete the nodes one by one from the beginning till the last node.

C Function

```

void destroy()
{
    struct node *temp;
    temp = NULL;
    while (start != NULL)
    {
        temp = start; // Store current node, before deleting
        start = start->next; // Move to next node
        free(temp); // Release memory occupied by temp
    }
}

```

4.4 Implementation of Stack using Linked List

Stack is a collection of items in which addition of new items and deletion of existing items are done from only one end known as top of stack. Since, linked list is also a list of items, concepts of stack can be implemented using linked lists instead of an array.

- The two operations that we apply on a stack are push (inserting item on the top) and pop (deleting item at the top).
- The push operation is similar to adding a new node at the beginning of the list (insertHead()).
- A stack items can be accessed only through its top, and a list items can be accessed from the pointer to the first node.
- Similarly, removing the first node from a linked list (deleteHead()) is similar to popping an item from a stack. In both cases, the only immediately accessible item of a collection is removed from that collection, and the next item becomes immediately accessible one.
- The first node of the list is the top of the stack.

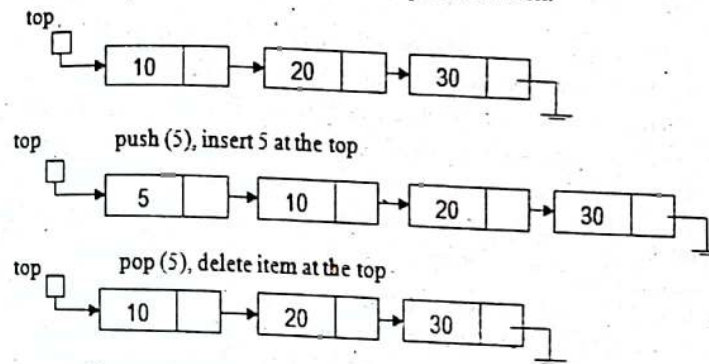


Figure 4.8: Implementation of stack using linked list

4.5 Implementation of Queue using Linked List

A queue is a special type of list that allows insertion at one end, called the front of queue, and deletion at other end, called the rear of queue.

- To implement a queue in a linked list, we make the first node of the list as the front and the last node as the rear. So, we will use two pointer variables – front and rear. 'front' will point to

- the beginning of the list (like start) while 'rear' will point to the last node of the list.
- Initially, when the queue is empty, both rear and front will be NULL.
 - To enqueue a new item, we insert it after the rear node and update rear to point to new node. This function will be similar to insertTail(), because both are the process to add a new item after the last node. In the case of queue, we insert a new item after the node pointed by 'rear'.
 - To dequeue an item, we remove the front node and update front to point to the next node. This function will be similar to deleteHead() of general list or pop() operation on a stack.

4.6 Circular Linked List

If we use linear singly linked list, we have to start from the starting node to visit a particular list. This problem can be solved by using a circular linked list. In such a list, it is possible to reach any other point in the list. A circular linked list is a list, where the next link of the last node contains a pointer to the first node rather than a null pointer. If we begin at a given node and traverse the entire list, we ultimately end up at the starting point.

In a circular list, we don't have a first or a last node. But, following a convention, we assume that the external pointer points to a last node. The node following this node will be called the first node.

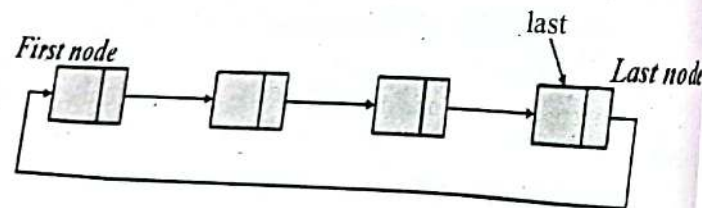


Figure 4.9: Circular linked list with last pointer

This convention allows access to the last and the first node (by referencing last->next).

Also, this provides the advantage of being able to add or remove an element conveniently from either the front or the rear of the list where as in linear linked list, to add a node at the end we have to start right from the beginning to reach the last node.

For an empty circular list, last = NULL.

Operations in Circular Linked List

Most of the operations on circular lists are similar to singly linked lists. However, care has to be taken for non-empty lists, as there is no null pointer to signify the end of the list. For our convenience, we can use start and last pointers to point to first and last node respectively.

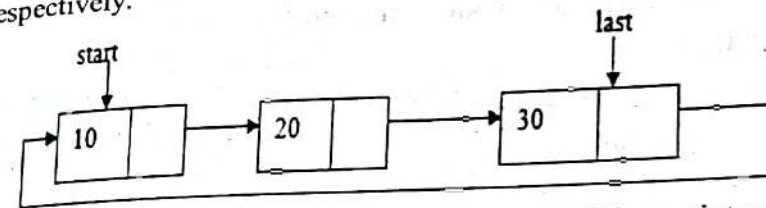


Figure 4.10: Circular linked list with start and last pointers

We declare the structure for the circular linked list in the same way as we declare it for the linear linked lists.

struct list

```
{
    int info;
    struct linked_list *next;
}
```

```
typedef struct list node;
node *start=NULL;
node *last=NULL;
```

1. Insertion

i) For inserting a node at the beginning

Algorithm

1. `ptr=(node *)malloc(sizeof(node))`
2. set `ptr->info=item`
3. if `start==NULL` then
 set `ptr->next=ptr`
 set `start=ptr`
 set `last=ptr`
 else follow step 4 to 6
4. set `ptr->next=start`
5. set `start=ptr`
6. set `last->next=ptr`

ii) For inserting a new node at the end

Algorithm

1. `ptr=(node *)malloc(sizeof(node))`
2. set `ptr->info=item`
3. if `start==NULL`
 set `ptr->next=ptr`
 set `start=ptr`
 set `last=ptr`
 else follow step 4 to 6
4. set `last->next=ptr`
5. set `last=ptr`
6. set `last->next=start`

2. Deletion

i) For deleting the node from the beginning

1. if `start==NULL`

 print "circular list is empty"

 else follow step 2

2. if `start->next==start`

 print "Deleted element is start->info"

`free(start)`

`start=NULL`

`last=NULL`

 else follow step 3 to 7

3. set `ptr = start`

4. set `start = start->next`

5. print "Deleted element is ptr->info"

6. set `last->next=ptr`

7. `free(ptr)`

ii) For deleting the node from the end

Algorithm

1. if `start==NULL`

 print "circular list is empty"

 else follow step 2

2. if `start->next==start`

 print "Deleted element is start->info"

`free(start)`

`start=NULL`

`last=NULL`

 else follow step 3 to 10

3. set `ptr=start`

4. Repeat step 5 and 6 until `ptr->next==start`

5. set `temp=ptr`

6. set ptr=ptr->next
7. print "Deleted item is ptr->info"
8. set temp->next=ptr->next
9. set last=temp
10. free(ptr)

4.7 Doubly Linked List

In both, singly linear list and a circular list, list can be traversed only in forward direction. But in doubly list, we can traverse the list in both directions. This has been made possible by defining two address fields, one of them points to the previous(left) node while the other one points to the next(right) node.

The external pointer will point to the first node, as in singly list. While performing the basic operations, we need to update both(left and right) address fields.

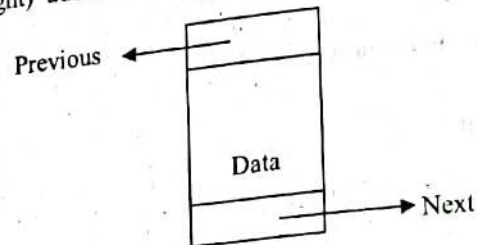


Figure 4.11: Doubly linked list node

We declare the structure for the doubly linked as:

```
struct list
{
    int info;
    struct list *prev;
    struct list *next;
};
typedef struct list node;
```

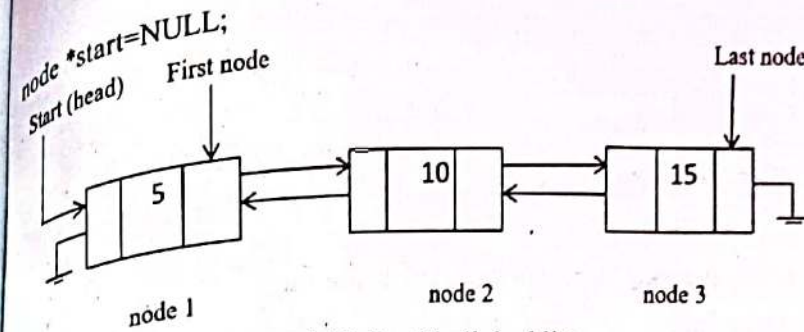


Figure 4.12: Doubly linked list

1. Insertion

i) For inserting a node at the beginning

Algorithm

1. ptr=(node *)malloc(sizeof(node))
2. set ptr->info=item
3. if(start==NULL) then
set ptr->prev = ptr->next = NULL
set start = ptr
else follow step 4 to 7
4. set ptr->prev=NULL
5. set ptr->next=start
6. set start->prev=ptr
7. set start=ptr

ii) For inserting a node at the end

Algorithm

1. ptr=(node *) malloc (sizeof(node))
2. ptr->info=item
3. if(start==NULL) then
set ptr->prev = ptr->next = NULL

set start = ptr
else follow step 4 to 7

4. set ptr->next=NULL
5. ptr->prev=last
6. last->next=ptr
7. last=ptr

2. Deletion

i) For deleting the node from the beginning

Algorithm

1. If the list is empty then display the message "empty list" and exit.
2. Otherwise make the head pointer to the point to the second node and if the second node is not null then make its left link to point to null.
3. Free the first node.

ii) For deleting the node from the end

Algorithm

1. If the list is empty then display the message "empty list" and exit.
2. Otherwise traverse the list till the last node and make the right link of the second last node to point to null.
3. Free the last node.

Advantages and Disadvantages of Doubly Linked List

Advantages:

1. Efficient utilization of memory.
2. Bidirectional traversal helps in the efficient and easy accessibility.

Disadvantages:

1. It uses two pointers which results in more memory space required.

4.8 Circular Doubly Linked List

It is the doubly linked list where the next link of last node points to the first node and previous link of first node points to the last node of the list.

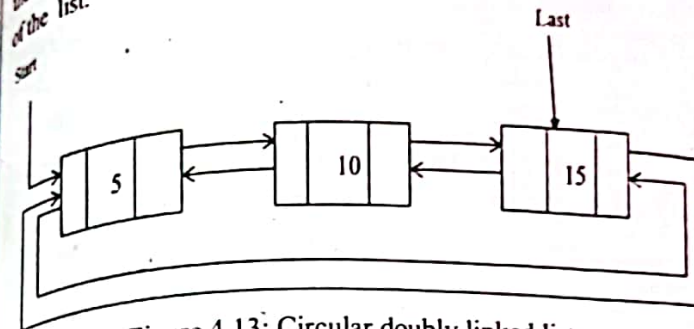


Figure 4.13: Circular doubly linked list

1. Insertion

i) For inserting a node at the beginning

Algorithm

1. Allocate the memory to the new node.
2. Assign value to its info field.
3. Make the left link of the head node to point to the new node and the right link of the right most (last) node to point to the newnode. Similarly, make right link of the newnode to point to the head. Assign head to newnode and set the left link of the newnode to point the last node.

ii) For inserting a node at the end

Algorithm

1. Allocate the memory to the newnode.
2. Assign value to its info field.
3. Make the left field of the new node to point to the last node and right link of the new node to point to the head node. Similarly, make the right link of the last node to point to the

new node. Set left link of the head node to point to the newnode.

2. Deletion

i) For deleting the node from the beginning

Algorithm

1. If the right link of the head node is pointing to itself then free the node and set head pointer to null and exit.
2. Otherwise make the head to point to the second node and left link of second node to point to the head node.
3. Free the first node.

2. For deleting the node from the end

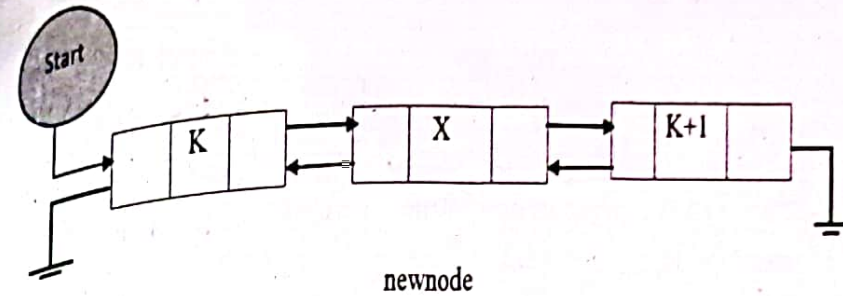
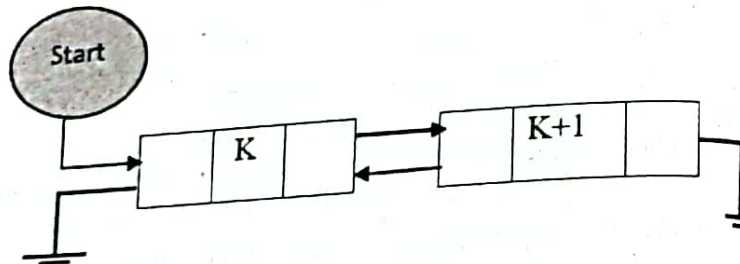
Algorithm

1. If the right link of the head node is pointing to itself then free the node and set head pointer to null and exit.
2. Otherwise make the right link of the second last node to point to the head node. Make the left link of the head node to point to the second last node.
3. Free the last node.

Questions:

1. Write an algorithm to insert an element after K^{th} node in doubly linked list. [2075 Bhadra]

Answer:



Here, Start is an external pointer which points to the first node. We are going to insert 'newnode' of value 'X' after node K.

Algorithm

1. `newnode = (node *) malloc (sizeof(node))`
2. `newnode->info = X`
3. `set newnode->next = K->next`
4. `K->next = newnode`
5. `newnode->prev = K+1->prev`
6. `K+1->prev = newnode`

2. Explain how the addition of polynomial equations is done using linked list. Write an algorithm to insert element in queue using linked list. [2076 Bhadra]

Answer:

Addition of polynomial equations using linked list

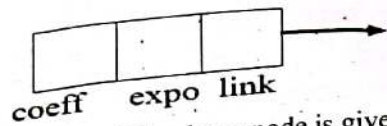
Linked lists are widely used to represent and manipulate polynomials.

Polynomials are expression containing number of terms with non zero coefficients and exponents. Consider the following polynomial,

$$P(X) = a_n x_n^e + a_{n-1} x_{n-1}^e + \dots + a_1 x_1^e + a.$$

Where a_i are non-zero coefficients and e_i are exponents.

In linked list representation of polynomials, each term is considered as a node and such node contains three fields: coefficient field, exponent field, and link field.



The logical representation of the above node is given as:

struct node

```
{
int coeff;
int expo;
struct node *ptr;
};
```

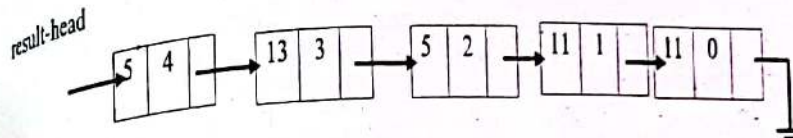
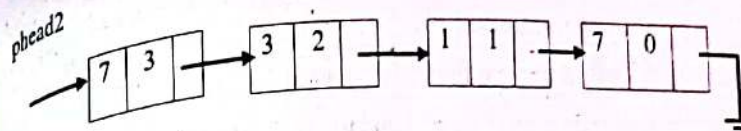
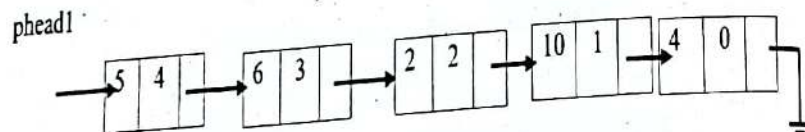
typedef struct node polynode;

Example: Let us consider the addition of two polynomials of single variable; $5x^4+6x^3+2x^2+10x+4$ and $7x^3+3x^2+x+7$.

In general addition:

$$\begin{array}{r} 5x^4+6x^3+2x^2+10x+4 \\ + 7x^3+3x^2+x+7 \\ \hline = 5x^4+13x^3+5x^2+11x+11 \end{array}$$

As a linked list representation:



Algorithm

1. Read the first and second polynomial and store into a polynode pointer variable *phead1 and *phead2 respectively.
2. Let *result-phead be a polynode pointer variable to store resultant polynomial.
3. Repeat step iv while there are terms in both polynomials yet to be processed.
4. Compare the exponent of two polynomials starting from the first node.
 - a) If both exponents are equal, then add the coefficients and store it in the resultant polynomial.
 - b) If the exponent of first polynomial is greater than the corresponding exponent of second polynomial, then copy the coefficient and exponent term of first polynomial into the resultant polynomial and check the next term of first polynomial.
 - c) Else insert the terms from second polynomial into the resultant polynomial and check the next term of second polynomial.

Algorithm to insert an element in queue using linked list
 Linked list can be implemented as a queue. To add an element, we have to add at last and to remove an element, we have to remove from beginning of the linked list.

- Allocate memory for new node.
- Insert the new node at last position of linked list and adjust pointers as follows:

```
item->next = Null;
if(start == Null)
    start = item;
else {
    while (temp->next != Null)
    {
        temp = temp->next;
    }
    temp->next = item;
}
```

Program 1: Linked List Implementation.

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<stdlib.h>
void insertfirst();
void insertlast();
void insertloc();
void deletefirst();
void deletelast();
void deleteloc();
```

```
void traverse();
void search();
struct list
{
    int info;
    struct list *next;
};
typedef struct list node;
node *start;
```

```
void main()
{
    int ch=0;
    start='\0';
    while(ch!=9)
    {
        printf("\n\t\tIMPLEMENTATION OF LINKED LIST");
        printf("\n1->Insert at first\n2->Insert at last\n3->Insert at desired\n\n4->Delete from first\n5->Delete from last\n6->Delete\n\n7->Delete from desired position\n8->Search the\n\n9->Exit\n");
        printf("\nEnter your choice\t");
        scanf("%d",&ch);
        switch(ch)
        {
            case 1:
            {
                insertfirst();
```

```

break;
}
case 2:
{
insertlast();
break;
}
case 3:
{
insertloc();
break;
}
case 4:
{
deletefirst();
break;
}
case 5:
{
deletelast();
break;
}
case 6:
{
deleteloc();
break;
}
case 7:

```

```

{
traverse();
break;
}
case 8:
{
search();
break;
}
case 9:
{
exit(1);
}
default:
{
printf("\nWrong selection");
break;
}
}
}

```

```

void insertfirst()
{
node *item;
item=(node*)malloc(sizeof(node));
printf("\nEnter the data to insert\n");
scanf("%d",&item->info);

```

```

while(count<n-1)
{
    if(temp->next=='\0'&&count+1!=n)
    {
        printf("\nThe location is not available");
        return;
    }
    temp=temp->next;
    count++;
}
item->next=temp->next;
temp->next=item;
}
printf("\nData has been inserted at %d location in the list",n);
getch();
}
void traverse()
{
    node *temp;
    if(start=='\0')
    {
        printf("\nThere are no data present in the list");
    }
    else
    {
        printf("\nThe data are:");
        temp=start;
        while(temp->next!='\0')

```

```

{
    printf("\n%d",temp->info);
    temp=temp->next;
}
printf("\n%d",temp->info);
}
getch();
}
void search()
{
    int n,i=1;
    node *temp;
    start:
    printf("\nEnter the data to search\n");
    scanf("%d",&n);
    if(start=='\0')
    {
        printf("\nThe list is empty");
    }
    else
    {
        temp=start;
        while(temp->info!=n)
        {
            if(temp->next=='\0')
            {
                printf("\nThe value doesn't exists");

```



```

        goto start;
    }
    temp=temp->next;
    i++;
}
printf("\nThe value %d is located at %d position",n,i);
getch();
}

void deletefirst()
{
    node *ptr;
    if(start=='\0')
    {
        printf("\nIt is empty");
    }
    else if(start->next=='\0')
    {
        printf("\nThe deleted number is %d",start->info);
        free(start);
        start='\0';
    }
    else
    {
        ptr=start;
        start=start->next;
        printf("\nThe deleted number is %d",ptr->info);
        free(ptr);
    }
}

```

```

    }
    getch();
}

void deletelast()
{
    node *ptr, *loc;
    if(start=='\0')
    {
        printf("\nIt is empty");
    }
    else if(start->next=='\0')
    {
        printf("\nThe deleted number is %d",start->info);
        getch();
        free(start);
        start='\0';
    }
    else
    {
        ptr=start;
        while(ptr->next!='\0')
        {
            loc=ptr;
            ptr=ptr->next;
        }
        loc->next='\0';
        printf("\nThe deleted number is %d",ptr->info);
    }
}

```

```

        free(ptr);
    }
    getch();
}
void deleteloc()
{
    node *ptr, *loc;
    int ch, n=1;
    if(start=='\0')
    {
        printf("\nIt is empty");
    }
    else if(start->next=='\0')
    {
        printf("\nIt has only one element");
    }
    else
    {
        traverse();
        printf("\nEnter the location at which you want to delete\n");
        scanf("%d", &ch);
        ptr=start;
        while(n<ch)
        {
            if(ptr->next=='\0')
            {
                printf("\nThe location doesn't exists");
                return;
            }
        }
    }
}

```

```

    }
    loc=ptr;
    ptr=ptr->next;
    n++;
}
loc->next=ptr->next;
free(ptr);
printf("\nThe data has been deleted");
}
getch();
}

```

Chapter 5: Recursion

5.1 Introduction

Recursion is a method to define something in terms of itself. Recursion makes algorithms and its implementation more compact and simple. Every recursive process consists of two parts:

- A smallest, base case that is processed without recursion; and
- A general method that reduces a particular case to one or more of the smaller cases, thereby making progress toward eventually reducing the problem all the way to the base case.

A function that calls itself is called recursive function.

Iteration vs Recursion

Iteration

- As long as the condition is true, the loop body is executed.
- When the loop body has been executed for the last time, the loop completely terminates.

Recursion

- As long as the recursion condition is true, the method is called again.
- When the base case has been reached, no further recursion occurs. However, all recursive calls then unfold backwards, possibly leading to the execution of further code.

Applications of Recursion

Recursion is useful for the definition of mathematical functions. Also used in game and puzzle solving.

5.2 Factorial of a Number

For a positive integer n , the factorial of n is defined as the product of all integers between n and 1.

For example, 5 factorial equals $5 \times 4 \times 3 \times 2 \times 1 = 120$ and 3 factorial equals $3 \times 2 \times 1 = 6$. 0 factorial is defined as 1. In mathematics, n factorial is denoted by $n!$.

Here, $n! = 1$, if $n = 0$.

$n! = n \times (n-1) \times (n-2) \times \dots \times 1$, if $n > 0$.

Hence,

$$0! = 1$$

$$1! = 1$$

$$2! = 2 \times 1$$

$$3! = 3 \times 2 \times 1$$

$$4! = 4 \times 3 \times 2 \times 1$$

We know, $4! = 4 \times 3 \times 2 \times 1$

But $3 \times 2 \times 1$ is $3!$, so we can write $4! = 4 \times 3!$

In fact, for any $n > 0$, we see that $n!$ equals $n \times (n-1)!$

5.3 Fibonacci Numbers

$\text{fib}(n) = n$ if $n = 0$ or $n = 1$.

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ if $n \geq 2$.

Example: Evaluate $\text{fib}(5)$.

$$\begin{aligned}\text{Solution: fib}(5) &= \text{fib}(4) + \text{fib}(3) \\ &= (\text{fib}(3) + \text{fib}(2)) + (\text{fib}(2) + \text{fib}(1)) \\ &= ((\text{fib}(2) + \text{fib}(1)) + (\text{fib}(1) + \text{fib}(0))) + \\ &\quad ((\text{fib}(1) + \text{fib}(0)) + 1) \\ &= (((\text{fib}(1) + \text{fib}(0)) + 1) + (1+0)) + ((1+0) + 1) \\ &= (((1+0) + 1) + (1)) + ((1) + 1) \\ &= (((1) + 1) + (1)) + ((1) + 1) \\ &= 5\end{aligned}$$

5.4 Iterative Algorithm for Function

Algorithm to evaluate the product of all integers between n and 1.

prod = 1;

for (i = n; i > 0; i--)

prod *= i;

prod is the required result.

This type of algorithm is called iterative because it calls for the explicit repetition of some process until a certain condition is met.

5.5 Recursive Function and Use of Stack

C allows to write functions that call themselves. Such functions are called recursive. Consider the following program which calculates the factorial of a given number n.

```
int fact(int n)
{
    int x, y, prod;
    if (n == 0) /* base case */
        prod = 1;
    else {
        x = n-1;
        y = fact(x); /* recursive call */
        prod = n * y;
    }
    return prod;
}
```

The recursive description suggests the use of a stack to keep the successive generations of local variables and parameters. This stack is maintained by the C system and is invisible to the user (internal stack). Each time a function is called, a new allocation of its variables are pushed on top of the stack. When a function returns, the stack is popped, the top allocation is freed and the previous allocation becomes the current stack top.

Illustration of above recursive function is shown below:

n	x	y	prod

Initially

2	1	*	*
3	2	*	*
4	3	*	*
n	x	y	prod

fact(2)

1	0	1	1
2	1	*	*
3	2	*	*
4	3	*	*
n	x	y	prod

fact(1)

y = fact(0)
prod = n * y

4	3	*	*
n	x	y	prod

fact(4)

1	0	*	*
2	1	*	*
3	2	*	*
4	3	*	*
n	x	y	prod

fact(1)

2	1	1	2
3	2	*	*
4	3	*	*
n	x	y	prod

fact(2)

y = fact(1)
prod = n * y

3	2	*	*
4	3	*	*
n	x	y	prod

fact(3)

0	*	*	1
1	0	*	*
2	1	*	*
3	2	*	*
4	3	*	*
n	x	y	prod

fact(0)

3	2	2	6
4	3	*	*
n	x	y	prod

fact(3)

y = fact(2)
prod = n * y

4	3	6	24
n	x	y	prod

fact(4)

y = fact(3)
*prod = n * y*

This factorial function can be rewritten as follows:

```
int fact(int n)
{
    if (n == 0)
        return 1;
    else
        return n * fact(n-1);
}
```

Some other examples of recursive function are:

The PrintBinary Function

```
void PrintBinary(int n)
{
    if (n == 0)
        return;
    PrintBinary(n/2);
    printf("%d", n%2);
}
```

The Fibonacci Function

```
int fib(int n)
{
    int a, b, sum;
```

```
if (n == 0 || n == 1)
    return n;
```

```
else
{
    a = fib(n - 1);
    b = fib(n - 2);
    sum = a + b;
    return sum;
}
```

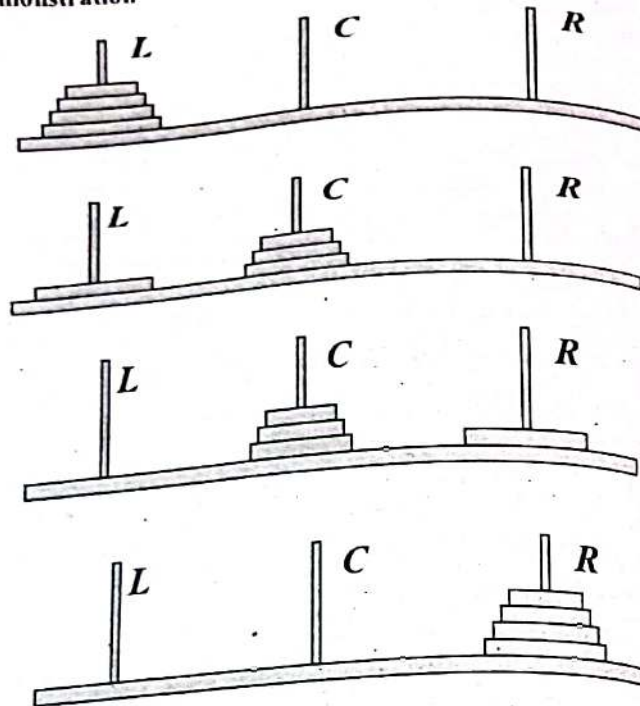
5.6 The Towers of Hanoi (TOH) Problem

The Towers of Hanoi is a mathematical game or puzzle. There are three pegs L, C, and R. Disks of different diameters are placed on peg L so that a larger disk is always below a smaller disk. The objective is to move the disks to peg R, using C as auxiliary. Only the top disk on any peg may be moved to any other peg. A larger disk may never rest on a smaller one.

The idea that gives a solution is to concentrate our attention not on the first step (which must be to move the top disk somewhere), but rather on the hardest step: moving the bottom disk.

There is no way to reach the bottom disk until all the disks above the bottom have been moved, and, furthermore they must all be on peg C so that we can move the bottom disk from peg L to R. The minimal number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$, where n is the number of disks.

Demonstration



Algorithm for Towers of Hanoi

(Move n disks from L to R using C as auxiliary).

1. If $n = 1$, move the single disk from L to R and stop.
2. Move the top $n - 1$ disks recursively from L to C, using R as auxiliary.
3. Move the n th disk from L to R.
4. Move the $n - 1$ disks recursively from C to R, using L as auxiliary.

5.7 Recursion Tree

Recursion tree method is a pictorial representation of an iteration method which is in the form of a tree where at each level nodes are expanded.

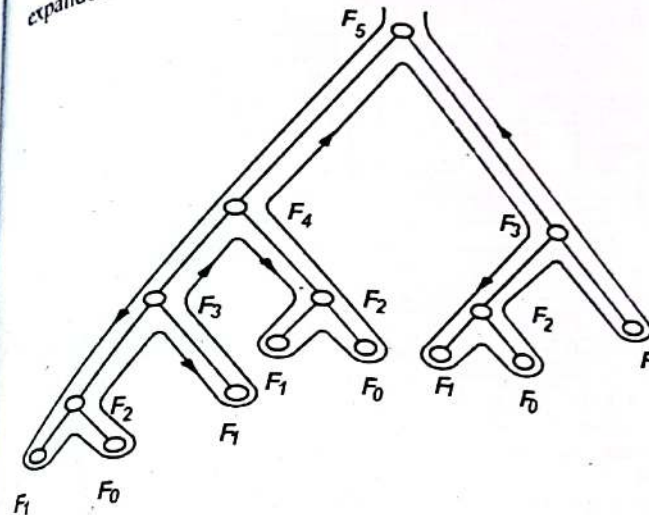


Figure 5.1: Recursion tree for fib(5)

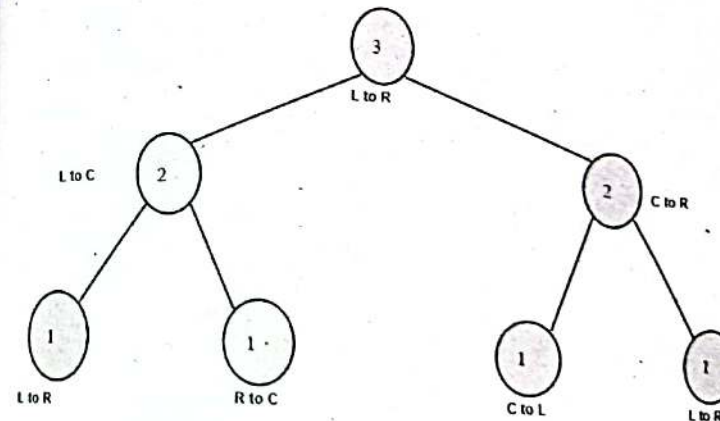


Figure 5.2 : Recursion tree for TOH ($n=3$)

5.8 Recursion and Efficiency

Some recursive solutions are so inefficient that they should not be used. Factors that contribute to the inefficiency of some recursive solutions are as follows:

- Overhead associated with method calls.
- It consumes more storage space, all the automatic (local) variables are stored on the stack.
- If the condition is not checked during recursion, computer may run out of memory.
- If proper care is not taken, recursion may result in non-terminating iterations.

Question: Discuss advantages and disadvantages of recursion. Generate a recursion tree for TOH problem consisting 3 discs [2075 Bhadra]

Answer:

Advantages:

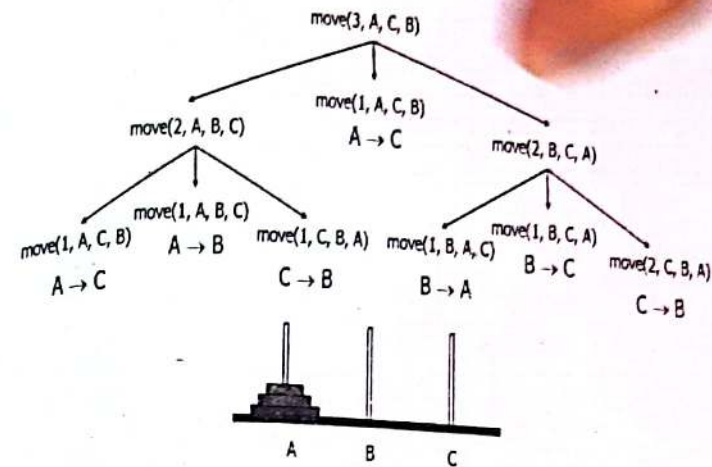
- Reduce unnecessary calling of function.
- Adds clarity and reduces the time needed to write and debug code.
- Better at tree traversal.

Disadvantages:

- Less efficient.
- Not all problems have recursive solution.
- Makes code smaller to understand but complex for computer.

Recursion tree for TOH problem consisting 3 discs

Let, there are 3 pegs; A, B and C. Recursion tree for 3 discs is shown in the figure below:



Program 1: Generation of Fibonacci Series.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int n,i,a;
    int fib(int n);
    printf("\nHow many numbers of the series you want to see\n");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        a=fib(i);
        printf("\n%d",a);
    }
}
```

```

        getch();
    }
    int fib(int n)
    {
        int a, b, sum;
        if(n==0||n==1)
            return n;
        else
        {
            a=fib(n-1);
            b=fib(n-2);
            sum=a+b;
            return sum;
        }
    }
}

```

Program 2: Implementation of TOH Problem.

```

#include<stdio.h>
#include<conio.h>
void main()
{
    int n;
    void transfer(int n, char from, char to, char temp);
    printf("\nEnter No. of discs\n");
    scanf("%d",&n);
    transfer(n,'L','R','C');
    getch();
}

```

```

void transfer(int n, char from, char to, char temp)
{
    if(n==0)
        return;
    else
    {
        transfer(n-1,from,temp,to);
        printf("\nMove disk %d from %c to %c",n,from,to);
        transfer(n-1,temp,to,from);
    }
}

```

Chapter 6: Trees

6.1 Introduction

Until now we have studied linear data structures. But there are many applications where only the linear data structures are not sufficient. For such situations, we use non-linear data structures like trees and graphs.

In computer science, a tree is an abstract model of a hierarchical structure.

- It is used to represent a hierarchical relationship existing among several data items.
- A tree consists of nodes with a parent-child relationship.
- It is a non-linear data structure.
- Searching as fast as in ordered array.
- Insertion and deletion as fast as in linked list.

Definition of a Tree

It is a finite set of one or more data items (nodes) such that

- There is a special data item called the root of the tree.
- Its remaining data items are partitioned into number of disjoint subsets, each of which is itself a tree again, called subtrees.

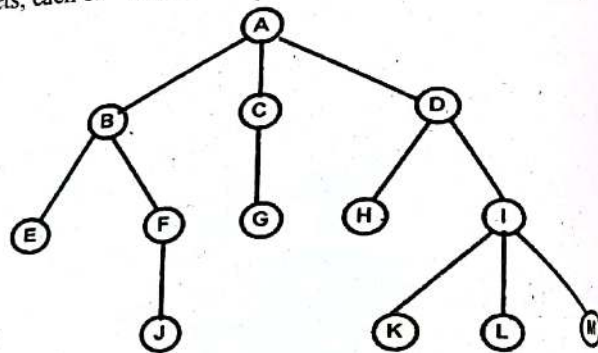


Figure 6.1: A Tree Structure

6.2 Basic Terminologies In Tree

- **Root:** Root is a special data item, which is the first in the hierarchy. In the above tree, 'A' is the root item.
- **Node:** Each item in a tree is called a node. A, B, F, G etc. are examples of node.
- **Parent:** A node that points to (one or more) other nodes is the parent of those nodes while the nodes pointed to are the children. B is parent of E, F and E, F are children of B.
- **Leaf (external nodes):** Nodes with no children are leaf nodes or external nodes or terminal nodes. In the above tree E, J, G, H, K, L, and M are terminal nodes.
- **Internal nodes:** Nodes with children are internal nodes or non-terminal nodes. Note that the root node is also an internal node. Node such as B, C, D, are non-terminal nodes.
- **Siblings (brother):** Nodes that have the same parents are siblings.

In the above tree:

- E and F are siblings of parent node B.
- K, L, and M are siblings of parent node I.
- **Descendants:** The descendants of a node consist of its children, and their children, and so on. All nodes in a tree are descendants of the root node (except the root node itself).
- **Ancestors:** The ancestors of a node consist of the parent node and the parent of that node also. The root node is the ancestor of all nodes and the root does not have any ancestor.
- **Order (degree):** Order (degree) of a node is given by the number of its children. If a node has two children, its degree is two.

In the above tree.

- degree of node A is 3.
- degree of node B is 2.
- degree of node C is 1.

Order (degree) of a tree is given by the maximum of the degrees of its nodes. In other words, if the degree of a tree is n , its nodes can have, at most, n children. In the above tree the node A has degree 3 which is the maximum degree. So the degree of the above tree is 3.

- **Edge:** The link between any two nodes is called an edge. It is directed from the parent towards its child.
- **Path:** There is a single, unique path from the root to any node. For e.g., the path from node A to J is A, B, F, and J.
- **Height:** A height of a node is equal to the maximum path length from that node to a leaf node. A leaf node has a height of 0. The height of a tree is equal to the height of the root. For e.g., the height of node B from leaf node J is 2.
- **Depth:** A depth of a node is equal to the maximum path length from root to that node. A root has a depth of 0. The depth of a tree is equal to the depth of the deepest node in the tree. It, of course, is equal to the tree height. For e.g., the depth of node J is 3.
- **Forest:** it is a set of disjoint trees. In a given tree, if you remove its node then it becomes a forest.
- **Level:** The level of a node is 0, if it is root. Otherwise, it is one more than its parent. For example:
 - Node C is at level 1
 - Node K is at level 3

6.3 Binary Tree

A binary tree is a finite set of data items which is either empty or consists of a single item called the root and the two disjoint binary trees called the left subtree and right subtree. In a binary tree the maximum degree of any node is at most two. That means, there may

be a zero degree node (usually an empty tree) or one degree node or two degree node.

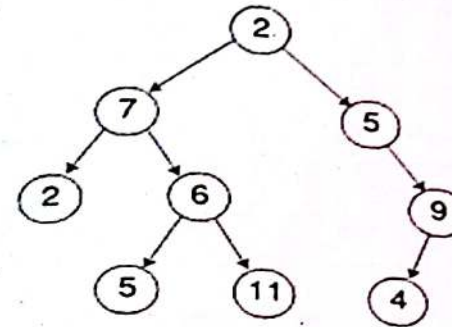


Figure 6.2: A binary tree

Some Properties of a Binary Tree

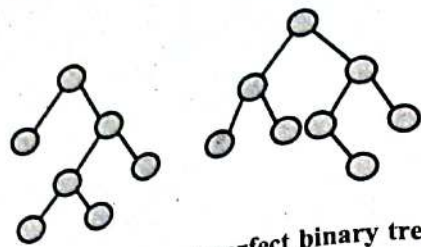
- Number of external nodes $\leq$ number of internal nodes + 1
- Number of nodes at level $i \leq 2^i$
- Number of external nodes $\leq 2^{\text{height}}$ (Here, height means height of the tree)
- Number of nodes in a tree $\leq 2^{\text{height}+1}-1$ (Here, height means height of the tree)
- $\text{height} \geq \log_2(\text{number of external nodes})$
- $\text{height} \geq \log_2(\text{number of nodes}) - 1$

6.3.1 Types of Binary Trees

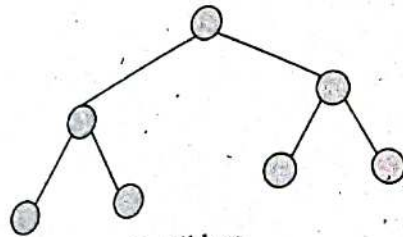
There are two types of binary trees.

A **strictly binary tree** or **full binary tree** or **proper binary tree** is a tree in which every node has zero or two children. That is, every internal node has nonempty left and right subtrees. A strictly binary tree with n leaves contains $2n - 1$ nodes.

Here the first tree is a strictly binary tree, while the second one is not.



A complete binary tree or perfect binary tree is a strictly binary tree in which all leaves are at the same depth. It has following properties.



- Every internal node has 2 children
- Height of n will have $2^{n+1} - 1$ nodes
- Height of n will have 2^n leaves
- No of nodes in level l have 2^l nodes
- No of external nodes = No of internal nodes + 1

6.4 Tree Traversals

Visiting the nodes in certain order is called traversing. The reason for visiting the nodes is to process the contents of the node according to our requirement.

In a linear structure like an array or a linked list, there are two ways of traversal: from front to end and from end to front.

These traversals can be achieved, simply, by using loops.

Tree, being a non-linear data structure, there are three different ways for traversal: preorder, inorder, and postorder traversals.

The general rules for these traversals are as follows:

1. Preorder traversal

In this traversal, the root node is visited prior to the left or right node in a subtree. The rule is

1. Visit the root node (N).
2. Visit the nodes in the left subtree in preorder (L).
3. Finally, visit the nodes in the right subtree in preorder (R).

2. Inorder Traversal

In this traversal, the root node is visited in between the left and the right node in a subtree. The rule is

1. Visit the nodes in the left subtree in inorder (L).
2. Visit the root node (N).
3. Finally, visit the nodes in the right subtree in inorder (R).

3. Postorder Traversal

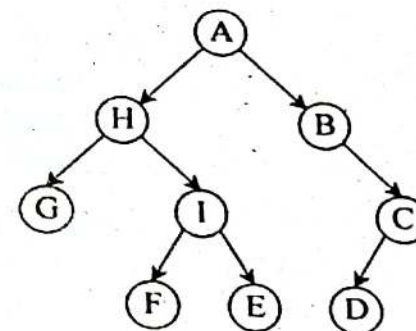
In this traversal, the root node is visited after the left and the right node in a subtree. The rule is

1. Visit the nodes in the left subtree in postorder (L).
2. Visit the nodes in the right subtree in postorder (R).
3. Finally, visit the root node (N).

Examples:

Traverse the following trees in preorder, inorder, and postorder.

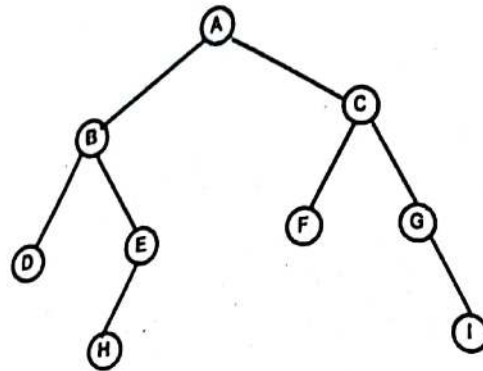
1.



Solution:

Preorder traversal: A, H, G, I, F, E, B, C, D
 Inorder traversal: G, H, F, I, E, A, B, D, C
 Postorder traversal: G, F, E, I, H, D, C, B, A

2.



Solution:

Preorder traversal: A, B, D, E, H, C, F, G, I
 Inorder traversal: D, B, H, E, A, F, C, G, I
 Postorder traversal: D, H, E, B, F, I, G, C, A

6.5 Expression Tree

An expression tree is used to represent arithmetic and logical expressions. It is built up from the simple operands and operators of an expression by placing the simple operands as the leaves of a binary tree, and the operators as the interior nodes.

For each binary operator, the left subtree contains all the simple operands and operators in the left operand of the given operator and the right subtree contains everything in the right operand.

For a unary operator, one subtree will be empty.

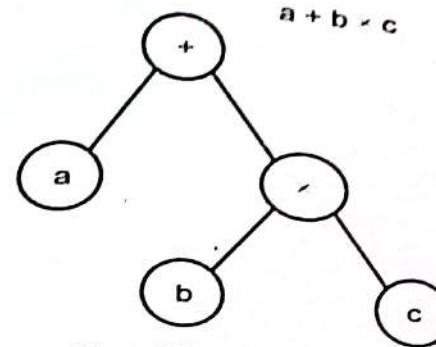


Figure 6.3: An expression tree

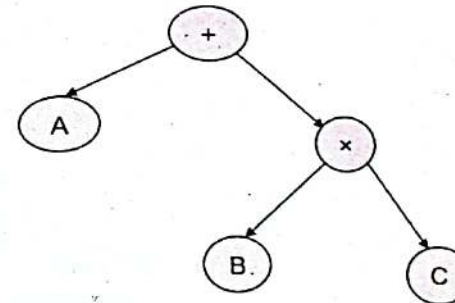
6.5.1 Expression Tree Traversals

When an expression tree is traversed in inorder, we get an infix expression. When it is traversed in preorder, we get the equivalent prefix expression. Similarly, when it is traversed in postorder, we get the equivalent postfix expression.

Examples: Create expression trees from the given expressions and traverse them.

1. $A + B \times C$

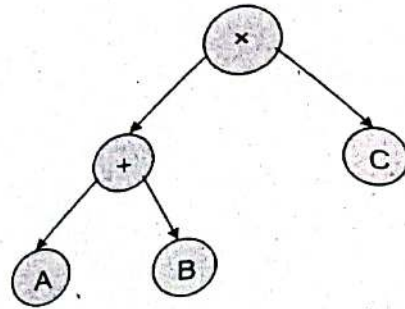
Solution:



When an expression tree is traversed in inorder, we get an infix expression. When it is traversed in preorder, we get the equivalent prefix expression. Similarly, when it is traversed in postorder, we

get the equivalent postfix expression. So, now let us check:
 The preorder traversal is: $+ A \times B C$ (equivalent to prefix expression).
 The postorder traversal is: $A B C \times +$ (equivalent to postfix expression).

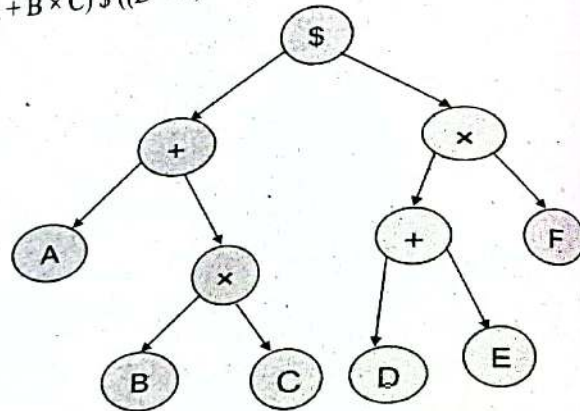
2. $(A + B) \times C$
 Solution:



The preorder traversal is: $\times + A B C$ (equivalent to prefix expression)

The postorder traversal is: $A B + C \times$ (equivalent to postfix expression)

3. $(A + B \times C) \times ((D + E) \times F)$



The preorder traversal is: $\$ + A \times B C \times + D E F$ (equivalent to prefix expression)

The postorder traversal is: $A B C \times + D E F \times \$$ (equivalent to postfix expression)

6.5.2 Evaluation of Expression Tree

Evaluation of an expression tree is done by applying the operator at the root to the values obtained by recursively evaluating left and right subtrees.

Algorithm

1. Evaluate (r)
2. If r is a leaf
return the value stored in r

Else

Let op be the operator stored in r

$r = \text{leftchild}(r)$

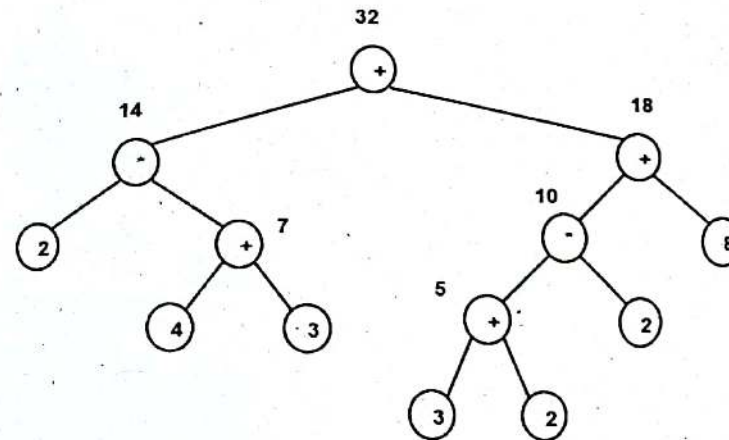
$x = \text{evaluate}(\text{LeftChild}(r))$

$y = \text{evaluate}(\text{RightChild}(r))$

return $x \text{ op } y$

End If

For example, consider the following expression tree. After evaluation, the answer is found to be 32.



6.5.3 Construction of an Expression Tree out of a Postfix Expression

Given a postfix expression, following is the algorithm to build an expression tree. The inorder and preorder traversals of this tree will give infix and prefix expressions respectively.

Algorithm

1. Make an empty stack.

2. For each character from left to right:

if the character is an operand.

- make a single node tree for this operand, push its address into the stack.

elseif it is an operator, say op

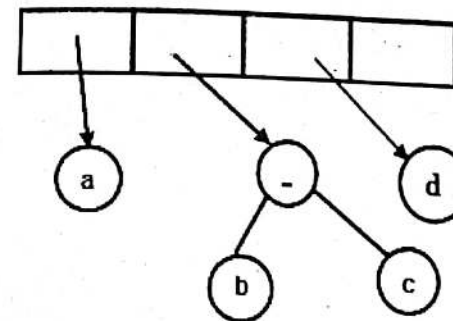
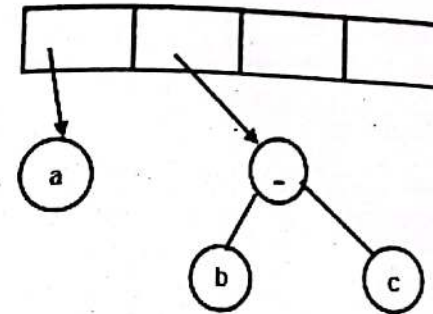
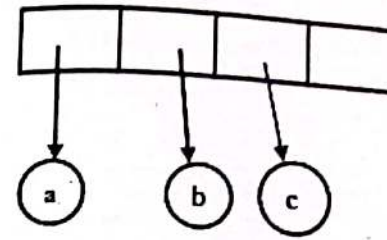
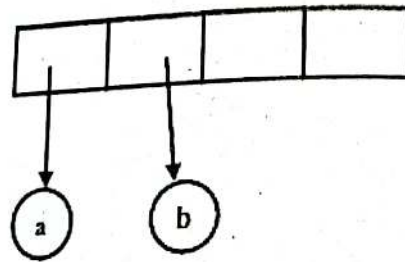
- make a single node tree for this operator
- pop the stack, make it right child of op
- pop the stack, make it left child op
- push the tree (whose root is op) into the stack

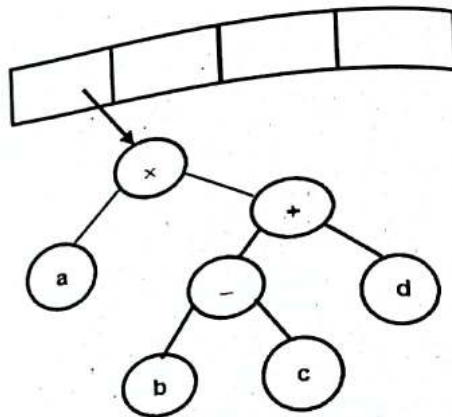
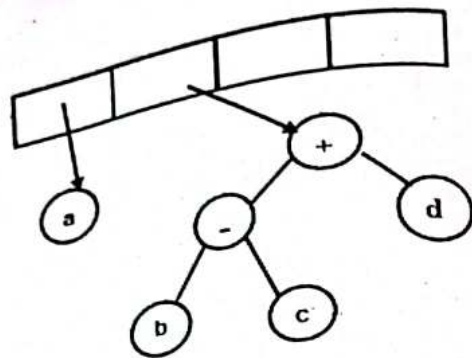
endif

3. Pop the stack to get the required expression tree.

Example: Construct an expression tree from the given postfix expression: $abc-d+x$.

Solution:





This is expression tree of given postfix expression.

6.6 Representation of Binary Tree

A binary tree can be represented in two different ways.

- Using linked list
- Using array

6.6.1 Linked Representation and Implementation

A binary tree is a collection of nodes. Each node in a binary tree has three fields, the data part, and two pointers that point to the root of left and right subtree. For a leaf the left and right pointers will

null. Also for an empty subtree, the corresponding pointers will be null. An external pointer is used to point to the root node of the tree.

```

struct bnode
{
    int info; //for storing data. Currently it is of type int.
    struct bnode *leftchild; //pointer to the root of the left subtree
    struct bnode *rightchild; //pointer to the root of the right subtree
};
    
```

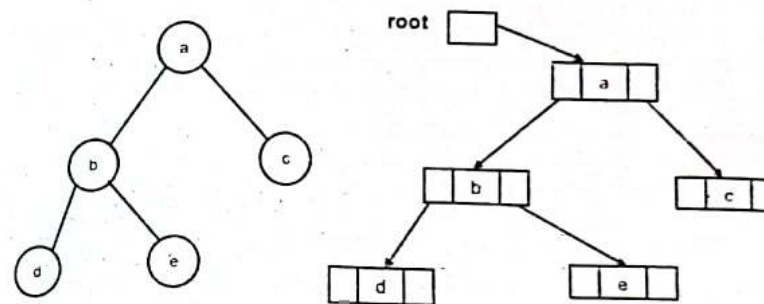


Figure 6.4: A binary tree and its linked list representation

6.6.2 Array Representation of Binary Tree

A binary tree can be represented by means of a linear array having index 'k', $0 \leq k \leq n$, where n is the number of nodes.

The root node is stored at the index 0.

Parent(k): The parent of the node at index k is given by $\text{floor}((k-1)/2)$ if k is not equal to 0

LeftChild(k): Left child of the node at k is given by $2k+1$.

RightChild(k): Right child of the node at k is given by $2k+2$.

Siblings(k): If the left child at index k is given, then its right sibling is at k+1. Similarly, if right child at index k is given, its left child is at k-1.

Some binary trees along with their sequential representation are as follows:

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	
8	
9	h
10	i
11	j
12	k
13	
14	

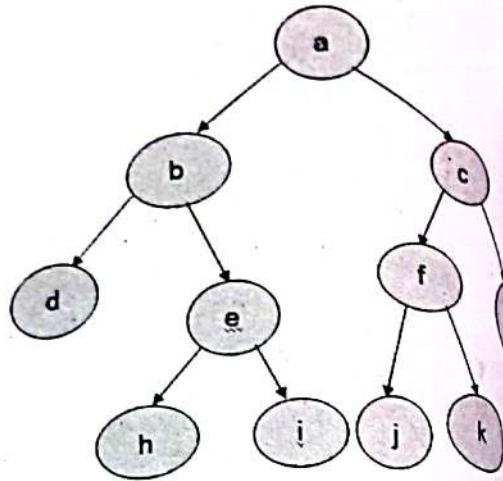


Figure 6.5: Array representation of binary tree

A sequential representation consumes more space for representing binary trees. But for representing complete binary tree, it proves to be efficient as no space is wasted. But, the amount of spaces has to be determined at first.

6.7 Binary Search Tree (BST)

A binary search tree is a binary tree which is either empty, or it contains a key and satisfies the following criteria:

- All keys (if any) in the left subtree are smaller than the key in the root.
- All keys (if any) in the right subtree are larger than the key in the root.
- The left and right subtrees should be BST.

So, we can also conclude that no two nodes in a BST have same value.

The inorder traversal of a BST yields the keys in a sorted order (ascending).

If the tree is reasonably full, searching for a key takes $O(\log_2(n))$ time, where n is the number of nodes.

Example: Create a binary search tree for given numbers: 18, 10, 26, 5, 20, 30, 4, 6, 25.

Solution:

All the keys in the left subtree are smaller than root. All the keys in right subtree are greater than key in the root.

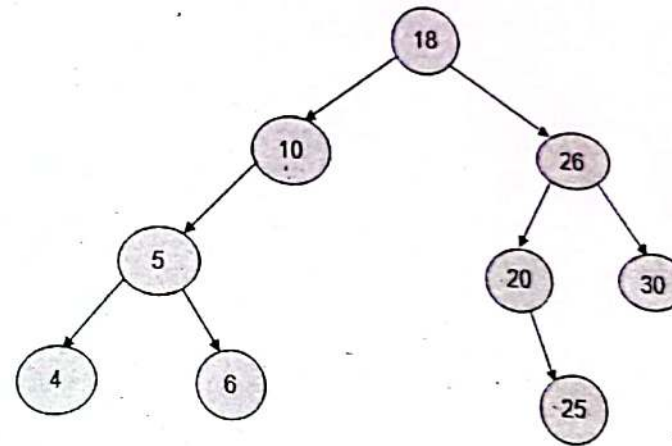


Figure 6.6: Binary search tree

6.7.1 Operations on a Binary Search Tree

- **Search(x, T):** Search for key x in the BST T . If x is found in some node n of T return a pointer to node n or NULL otherwise.
- **Insert(x, T):** Insert a new node with x in the info part in the tree T such that the property of BST is maintained.

- **Delete(x,T):** Delete a node x in the info part from the tree T such that the property of BST is maintained.
- **FindMin(T):** Returns pointer to a node having minimum key in tree T , NULL if T is empty.
- **FindMax(T):** Returns pointer to a node with maximum key in T , NULL if T is empty.

1. Searching

In BST, searching of the desired data item can be performed by branching into left or right subtree until the desired data item is found. We proceed from the root node.

Algorithm

1. If the tree is empty, then we failed to find x , so return NULL.
2. Else, compare x with the key stored in the root of T .
 - If x is same as the key in root, return address of the root.
 - If x is smaller than the key in root, search for x in the left subtree of T recursively.
 - If x is greater than the key in root, search for x in right subtree recursively.

2. Insert

In BST, we don't allow duplicate data items. So, to insert a data item having key ' K ' in a binary search tree, we must check that its key is different from those of existing data items by performing a search for data item with the same key K . If the search is unsuccessful, the data item is inserted into BST at the point the search is terminated.

Given a new item with key x , then following algorithm will insert it in BST ' T '.

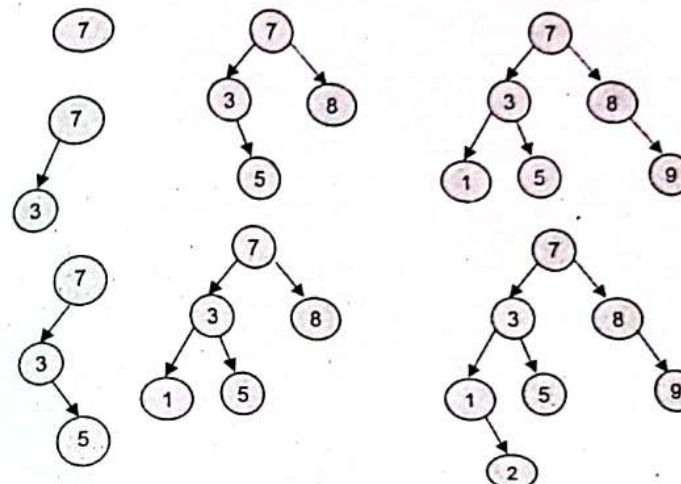
Algorithm

1. If the tree is empty, create a new node n storing the key x and make this new node n the root of the tree. Otherwise, compare x to the key stored at root R .

2. If x is identical to key in R , don't change the tree.
3. If x is smaller, insert new item into left subtree of the root recursively.
4. If x is larger, insert new item into right subtree of the root recursively.

Example: Insert the numbers 7, 3, 5, 8, 1, 9, 2 in BST.

Solution:

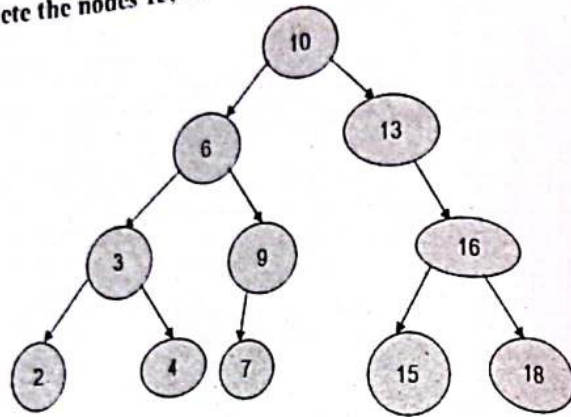


3. Deletion

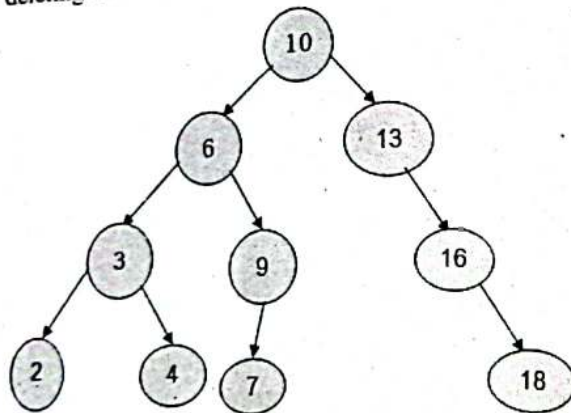
To delete a given node (key), it is searched first. If search is unsuccessful, the algorithm terminates. Otherwise, the following possibilities arise:

- If node is a leaf, it can be deleted immediately.
- If the node has one subtree, the node can be deleted by replacing it with the root of the non-empty subtree.
- If the node has two subtrees, we replace the node with the rightmost node of the left subtree T_l or the leftmost node of the right subtree T_r .

Examples:
1. Delete the nodes 15, and 13 from the given BST.

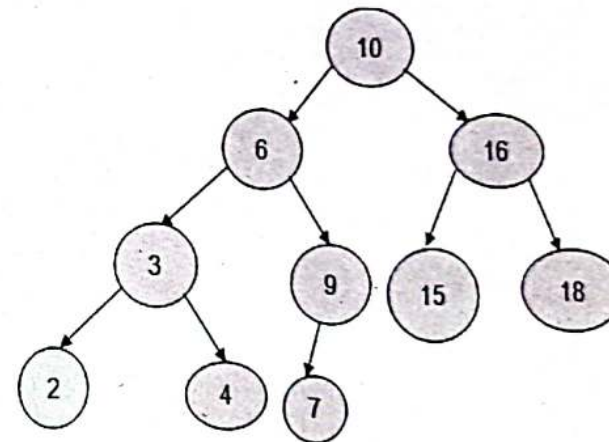
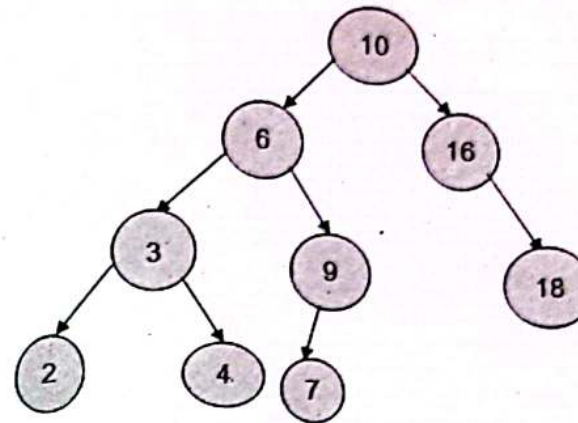


Solution:
After deleting leaf node 15



Now let's delete node with key 13 from the above tree

2. Delete root node with value 10 from given BST.

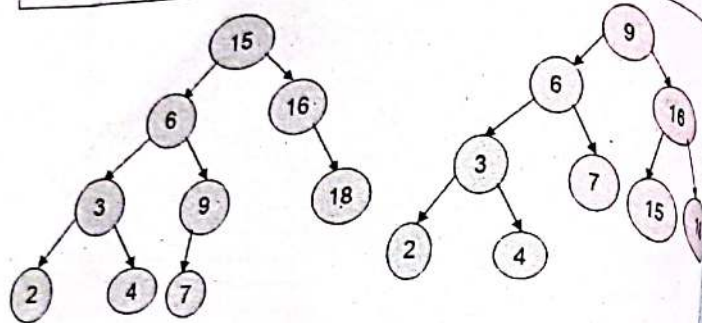


Solution:

After deleting root node 10

Leftmost node of right subtree takes its place

Rightmost node of left subtree takes its place



6.7.2 Efficiency of Binary Search Tree

Complexity of BST operations is proportional to the length of the path from the root to the node being manipulated, i.e., height of the tree.

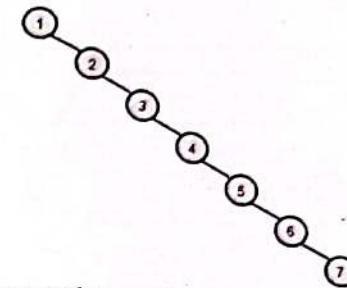
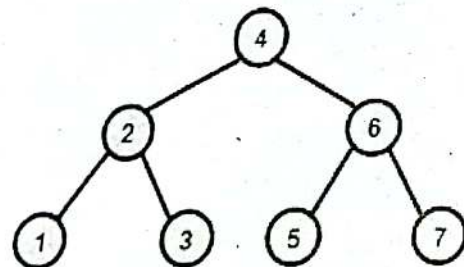
In a well balanced tree, the length of the longest path is roughly $\log n$, where n is the no of nodes in the tree.

Example: 1 million entries $\rightarrow$ longest path is $\log_2 10,48,576 = 20$.

For a thin, unbalanced tree, complexity of BST for searching operation is $O(n)$.

So, balancing is important to decrease the computational complexity.

Example:



In the first tree, the search operation requires 3 comparisons to find 7.

In the second tree, the search operation requires 7 comparisons to find 7.

6.8 Height Balanced Trees or AVL Trees

The complete binary tree has the shortest overall path length for any binary tree. The longest path in a complete binary tree with n elements is guaranteed to be no longer than $\log(n)$. If we can keep our tree complete, we're set for fast-search times but the cost is very high to maintain a complete binary tree. Instead, use height balanced binary trees: for each node, the height difference between the left and right sub trees is at most one.

A binary tree of height h is completely balanced if all the leaves occur at level h or $h-1$, and all nodes at levels lower than $h-1$ have two children.

An AVL tree is a binary search tree in which the heights of the left and right subtree of the root differ by, at most, 1 and in which the left and right subtree are again AVL trees. It is named after their inventors' G.M. Adelson, Velskii and E.M. Landis.

Example of balance tree:

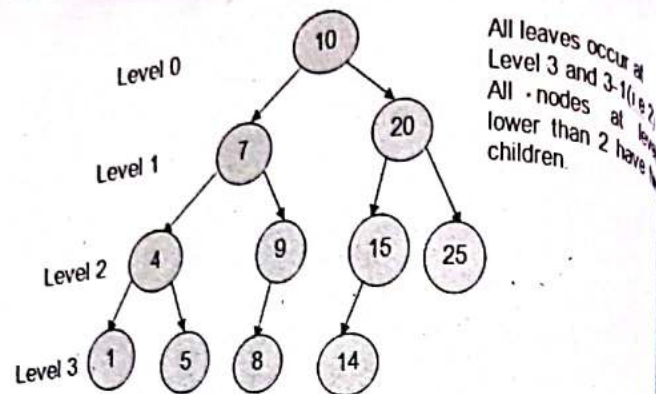
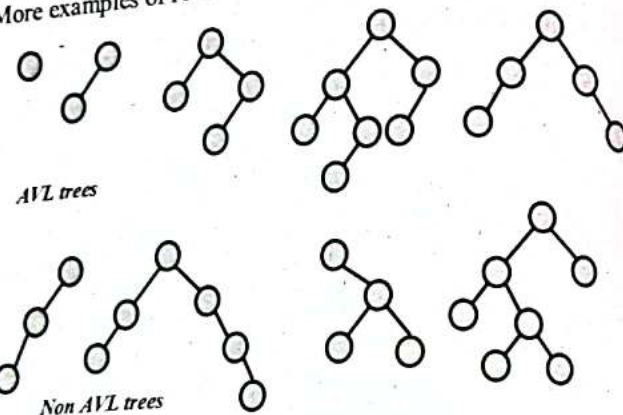


Figure 6.7: AVL tree

More examples of AVL and non-AVL trees are shown here:



Algorithm of Height Balanced (AVL) Tree

1. Insert the new node using the same algorithm as for an ordinary binary search tree.
2. Beginning with new node, calculate the difference in heights of the left or right subtrees of each node on the path leading from the new node back up the tree towards the root (node).
3. If the height balance is within the range -1 to 1 continue their checks until the root node is encountered.

4. If height balance is out of the range then an imbalance is found, perform a rotation of nodes to correct imbalance.

- i. If insertion is at left subtree of the left child of node n or insertion at right subtree of the right child of node n .
 - Perform single rotation right or single rotation left respectively.
- ii. If insertion at right subtree of the left child of node n or insertion at left subtree of the right child of node n .
 - Perform double rotation left-right or right-left respectively.

6.8.1 Insertion in AVL Tree

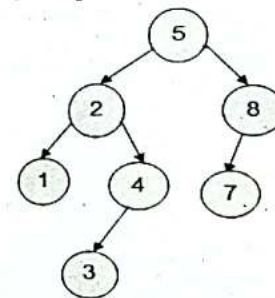
A new node is inserted in an AVL tree by using usual binary search tree algorithm. It, often, is inserted without changing the height of the subtree. Even when the height of the subtree increases, it may be shorter subtree that has grown.

But there may be cases where, a node is added to a subtree that is taller than other subtree. Then, the height difference would be 2.

If this is the case, then the property has to be restored before the insertion step is considered over.

We use 'ROTATIONS' to maintain the AVL tree properties.

Example:



AVL tree property will be violated at node 8 if we insert 6.

Cases of violation:

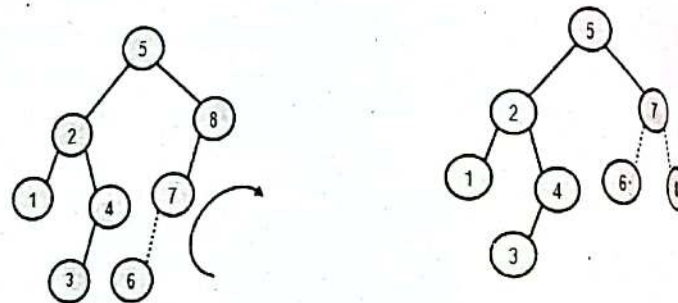
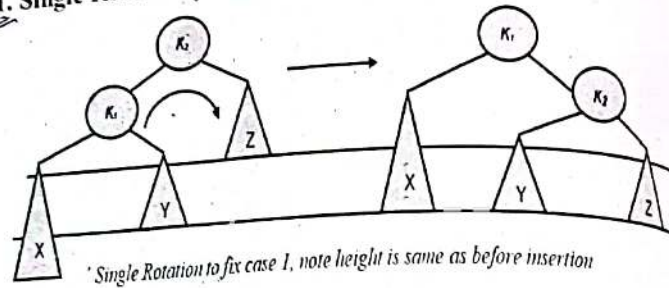
After insertion, violation might occur in four ways.

- 1) Insertion at left subtree of the left child of node n.
- 2) Insertion at right subtree of the left child of node n.
- 3) Insertion at left subtree of the right child of node n.
- 4) Insertion at right subtree of the right child of node n.

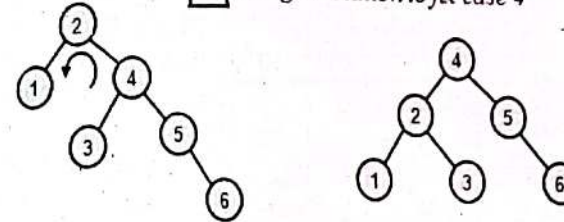
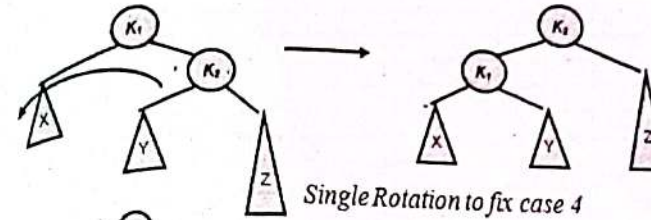
The first and fourth cases (left-left and right-right) is rebalanced using single rotation.

The second and third cases (right-left and left-right) are fixed using double rotation. This is also known as dogleg pattern.

1. Single Rotation



AVL property violated by insertion of 6, then fixed by single rotation.

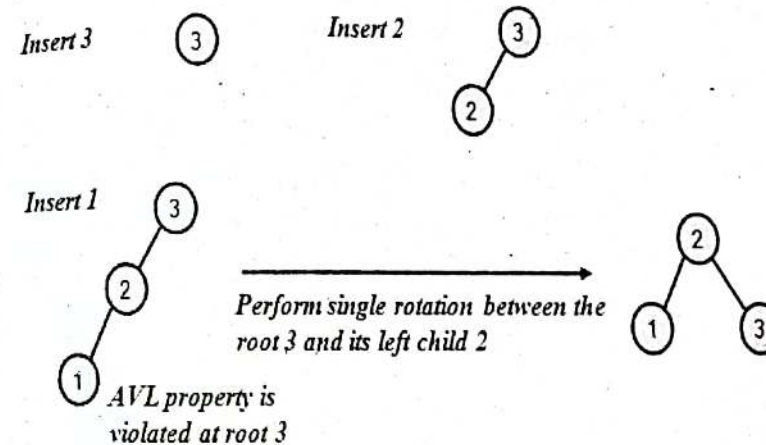


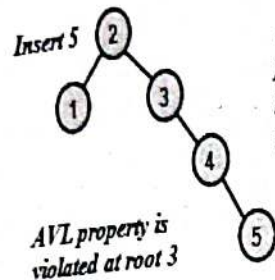
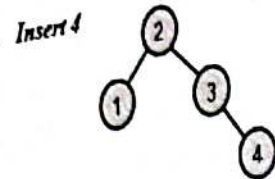
AVL property violated by insertion of 6, then fixed by single rotation.

Tracing of Single Rotation

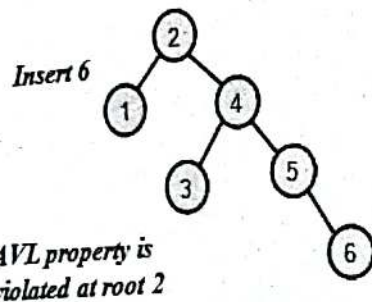
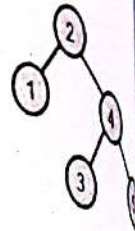
Example: Insert 3, 2, 1, 4, 5, 6, 7 in an empty AVL tree.

Solution:

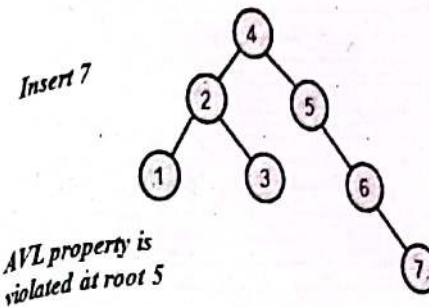
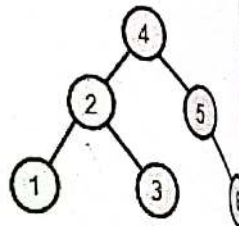




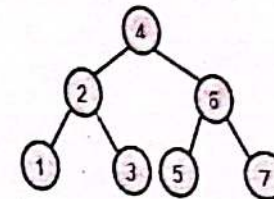
Perform single rotation between the root 3 and its right child 4



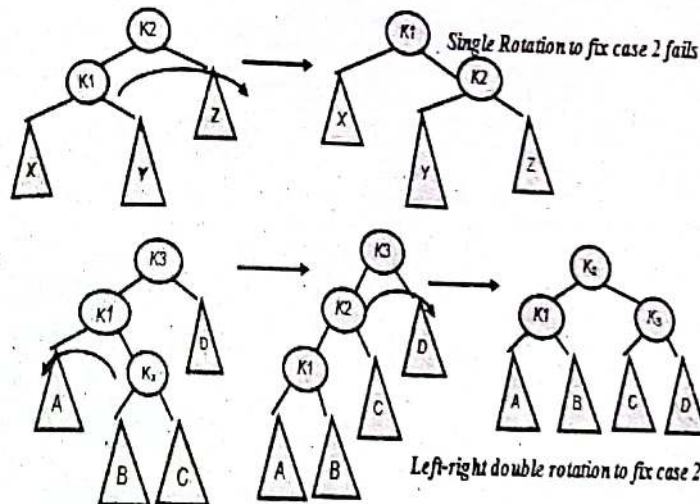
Perform single rotation between the root 2 and its right child 4

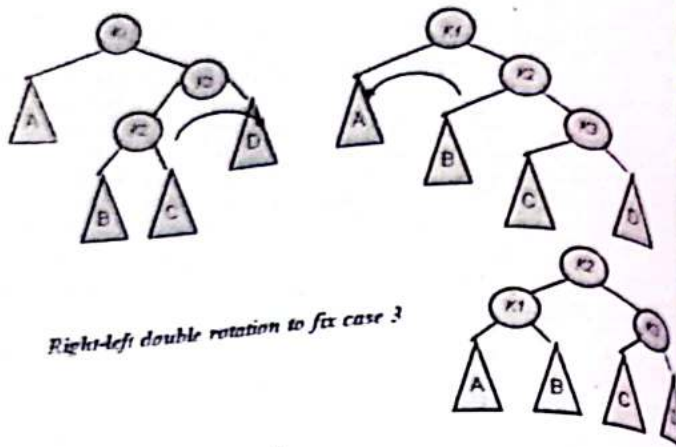


Perform single rotation between the root 5 and its right child 6



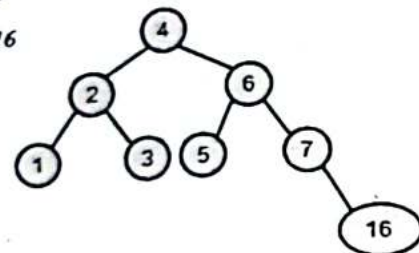
2. Double Rotation



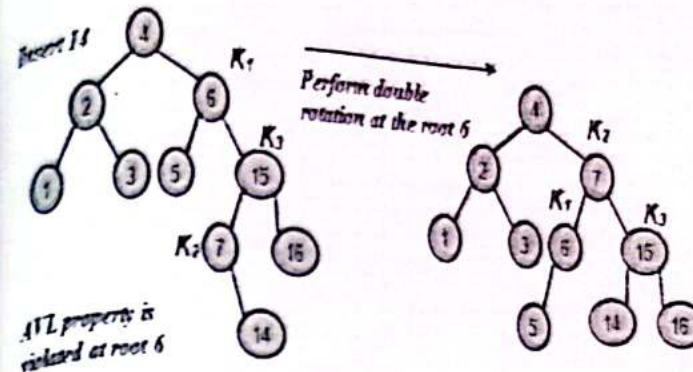
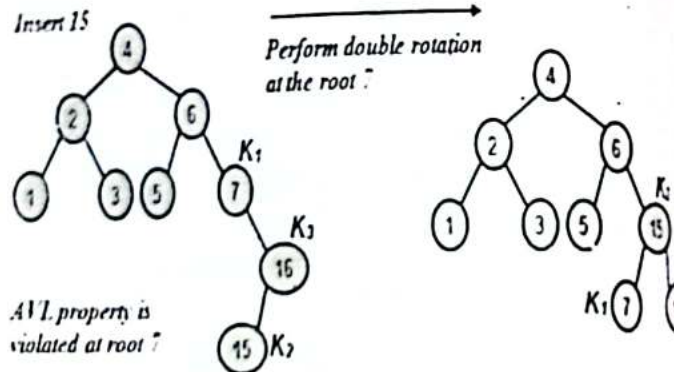


Tracing of Double Rotation
 Example: Insert 16, 15, 14, 13, 12 in the previous tree.
 Solution:

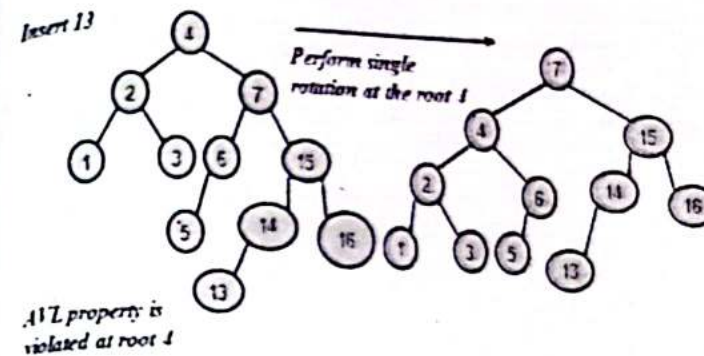
Insert 16



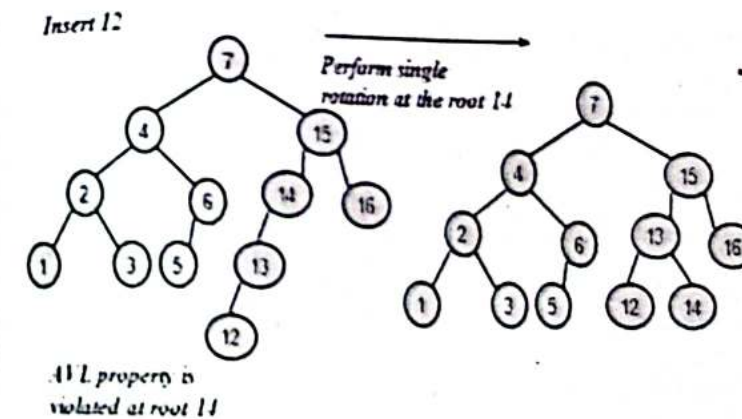
Insert 15



Insert 13



Insert 12



6.8.2 Deletion in AVL Tree

Like insertions, deletion of nodes in AVL Tree is also similar to deletion in BST. After deletion of nodes, we have to check for imbalance and correct it to rebalance the tree.

6.8.3 Efficiency of AVL Tree

- The maximum height of an AVL tree is $1.44 \times \log n$, which is an $O(\log n)$ function.
 - This means that in the worst possible case, a lookup in a large AVL tree needs no more than 44% more comparisons than a lookup in a completely balanced tree.
- Even in the worst case, AVL trees are efficient; they still have $O(\log n)$ lookup times.
- On average, for large n , AVL trees have lookup times of $(\log n) + 0.25$, which is even better than the above worst case figure, though still $O(\log n)$.
- On average, a rotation (single or double) is required in 46.5% of insertions. Only one (single or double) rotation is needed to readjust an AVL tree after an insertion throws it out of balance.

6.9 Huffman Algorithm

Lots of compression systems use Huffman algorithm for the compression of text, images, audio, data, etc. The general idea behind the Huffman algorithm is: use less number of bits to represent mostly used key.

For example:

Let's say we get four kind of signals in a system.

A	B	C	D
---	---	---	---

For such system, we might use 2-bit representation as

A	00
B	01

C	10
D	11

And we have data string as,

AAABCAABBCDDAAAABAAABABABADDAAAABC

If the number of A is 10,000, B is 234, C is 45 and D is 134 then the total storage required will be $(10,000 + 234 + 45 + 134) \times 2 = 20,826$ bits to store.

Since, the data 'A' is repeated ten thousand times, if the storage for A can be reduced, then the total space required will automatically be reduced. Therefore, in Huffman algorithm, the data (symbol) that is used most is represented by a single bit.

Now, while assigning the code word, the most frequently used symbol is assigned a code word of length '1', and the final one always gets code word with all ones.

Let's assign code word for the symbols as;

A	0
B	10
C	110
D	111

Now, for the given data string, the binary value will be,

00010110001010110111111000010000100100100111111000010110

Now, if we calculate the total space required to store the data, it will be:

$$(10,000 \times 1 + 234 \times 2 + 45 \times 3 + 134 \times 3) = 11,005 \text{ bits.}$$

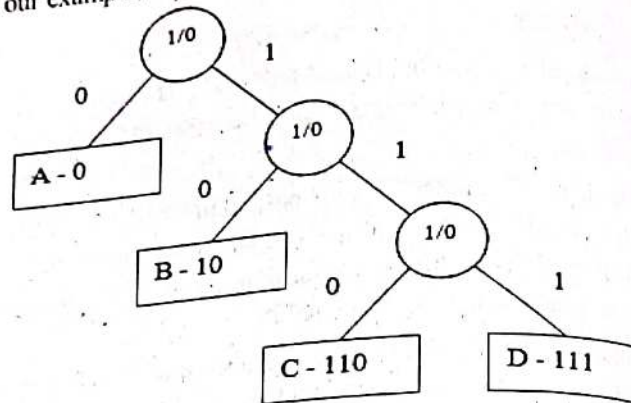
Here, we saved $20,826 - 11,005 = 9,821$ bits.

If the data to be stored is very large, then more bits will be saved.

Now the problem is how we recover the original data from compressed string;

0001011000101011011111000010000100100100111110000101
10

Here, a tree is made according to the code word assignment.
For our example, we have to make a tree like;

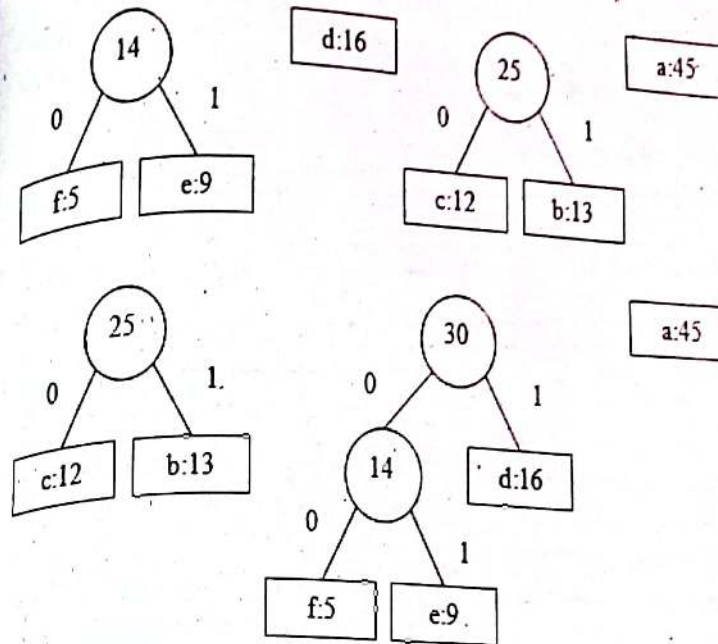
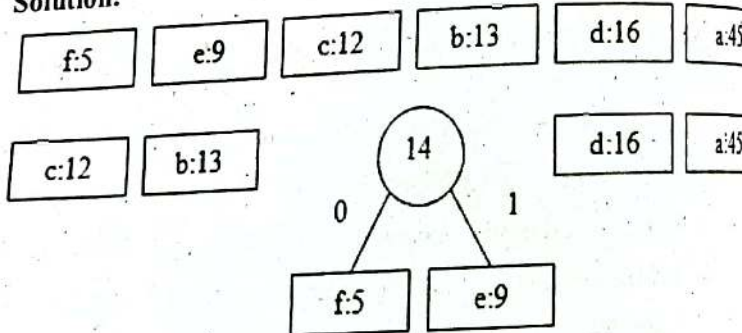


Now, to decode the encoded string, read a bit at a time. Compare with root node. If it is 0, then we are at the end decision point. So, it must be 'A'. If it was 1, then we have to read the next bit again and check. If the next bit is 0 then it is B. Otherwise, read next bit and check until we do not reach a decision point.

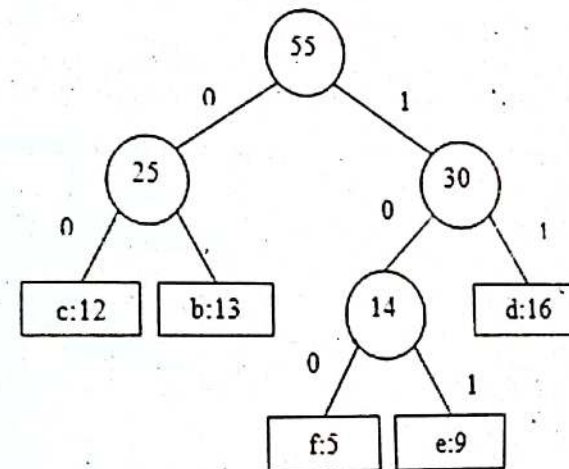
Example: Construct a Huffman code for the given symbols.

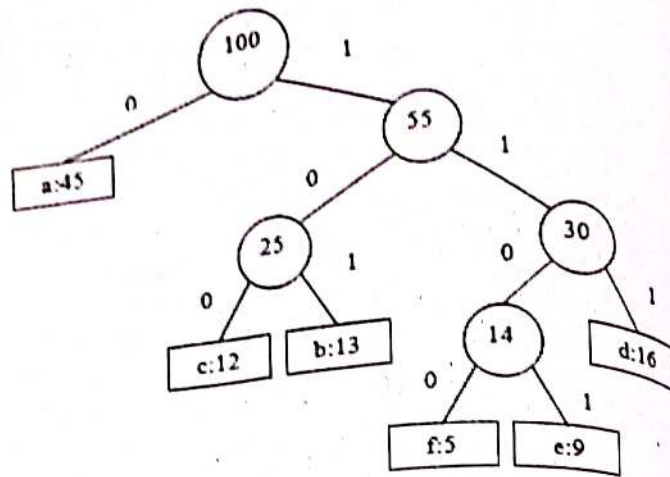
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5

Solution:



a:45





Each part shows the contents of the queue sorted into increasing order by frequency. At each step, the two trees with lowest frequencies are merged.

Leaves are shown as rectangles containing a character and its frequency. Internal nodes are shown as circles containing the sum of the frequencies of its children. An edge connecting an internal node with its children is labeled 0 if it is an edge to a left child and 1 if it is an edge to a right child.

The codeword for a letter is the sequence of labels on the edges connecting the root to the leaf for that letter.

Hence, we generate the variable-length codeword for the given symbols.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

6.10 Multiway Tree

In binary search tree, each node contains a single key and points to two sub-trees.

We may extend this concept to a general search tree in which each node contains one or more keys.

A multi-way search tree of order n is a general tree in which each node has n or fewer sub-trees and contains one fewer key than it has sub-trees.

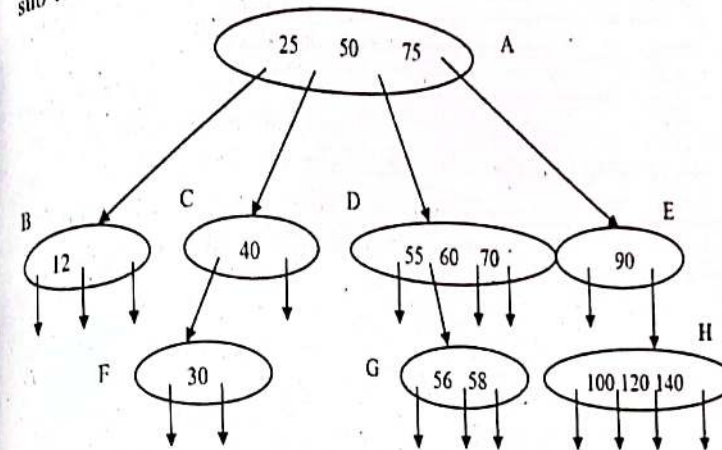


Figure 6.8: A multiway search tree of order 4

Each node can have at most 3 keys and 4 children, since the order is 4. The nodes A, D and H are called full nodes since it has the maximum no. of keys.

Balanced-Multiway Tree

A multiway search tree can be improved to minimize accesses by accomplishing the followings:

1. Make the height of the tree as small as possible.
2. No empty subtrees appear above the leaves (so that the division of keys into subsets is as efficient as possible).

3. All the leaves be on the same level (so that searches will be guaranteed to terminate with about the same number of accesses).
4. Every node (except the leaves) have at least some minimal number of children. These conditions leads us to the definition of B-Tree.

6.11 B-Tree

A B-tree of order m is an m -way search tree in which

1. All leaves are on the same level.
2. All internal nodes except the root have at most m nonempty children, and at least $(m/2)$ nonempty children.
3. The number of keys in each internal node is one less than the number of its children, and these keys partition the keys in the children in the fashion of a search tree.
4. The root has at most m children, or as few as 2 if it is not a leaf, or none if the tree consists of the root alone.

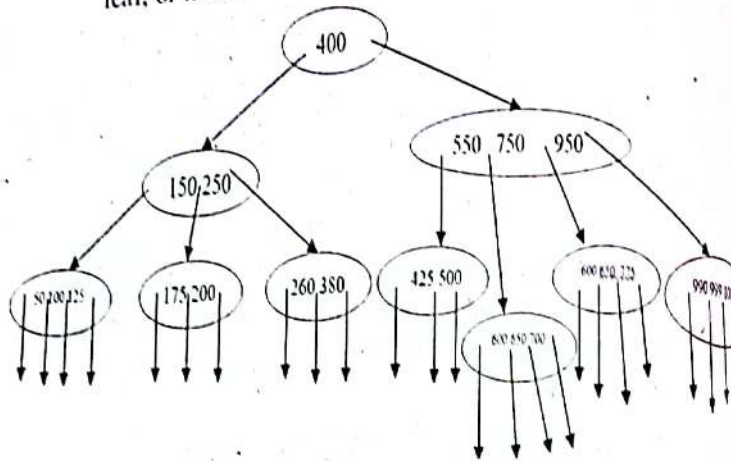


Figure 6.9 : B-Tree

6.11.1 Insertion into a B-Tree

The condition that all the leaves be on the same level forces a characteristic behavior of B-Tree. In contrast to binary search tree,

B-Trees are not allowed to grow at their leaves, instead they are forced to grow at the root. The general method of insertion is as follows.

First a search is made to see if the new key is in the tree. This search (if the key is new) will terminate in failure at a leaf. The new key is then added to the leaf node. If the node was not previously full, then the insertion is finished.

When a key is added to a full node, then the node splits into two nodes on the same level, except that the median key is not put into either of the two new nodes but is instead sent up the tree to be inserted into the parent node.

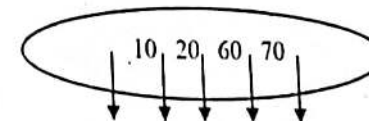
Example: Construct B-Tree of order 5 from the data set given by 10, 70, 60, 20, 110, 40, 80, 130, 100, 50, 190, 90, 180, 240 showing each insertion steps.

Solution:

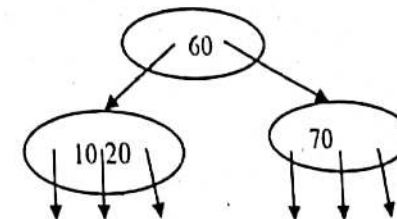
Here, order of B-Tree (m) = 5.

Maximum number of keys in a node = $m - 1 = 4$.

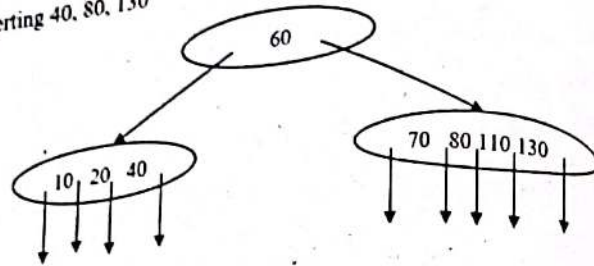
Inserting 10, 70, 60, 20



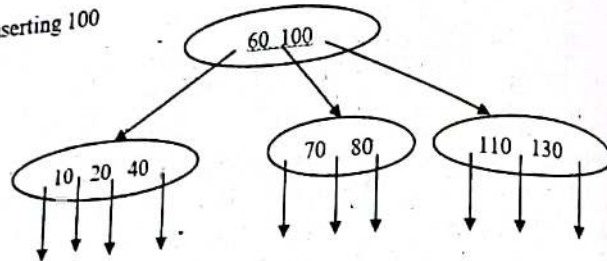
Inserting 110



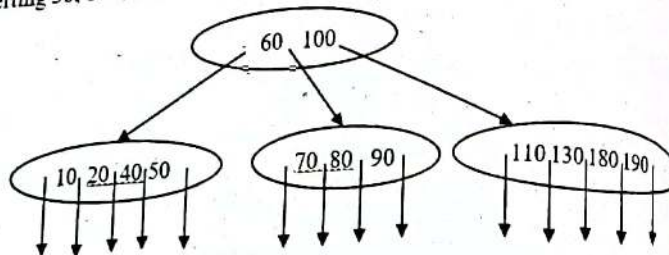
Inserting 40, 80, 130



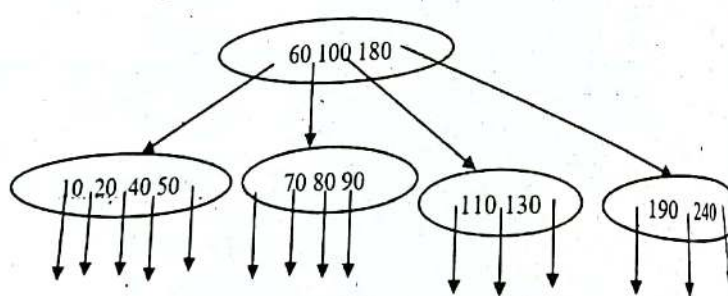
Inserting 100



Inserting 50, 190, 90, 180



Inserting 240



The first four keys will be inserted into one node as shown in the diagram. They are sorted into proper order as they are inserted. There is no room for fifth key 110, so its insertion causes the node to split into two, and the medial key, 60 moves up to enter a new node, which is a new root. Since the split nodes are now only half full, the next three keys can be inserted without any difficulty. Note these simple insertions can require rearrangement of the keys within a node. The next insertion 100, again splits a node and this time it is 100 itself that is the median key, so moves up to join 60.

Next several insertions are similar. The final insertion of 240 splits the node containing 110, 130, 180 and 190 which is again full. Hence, this node in turn splits and 180 is promoted into the root.

6.12 Red Black Tree

A red black tree is a kind of self balancing binary search tree. Each node stores an extra bit representing color, used to ensure that the tree remains approximately balanced during insertions and deletions. When the tree is modified, the new tree is rearranged and repainted to restore the coloring properties.

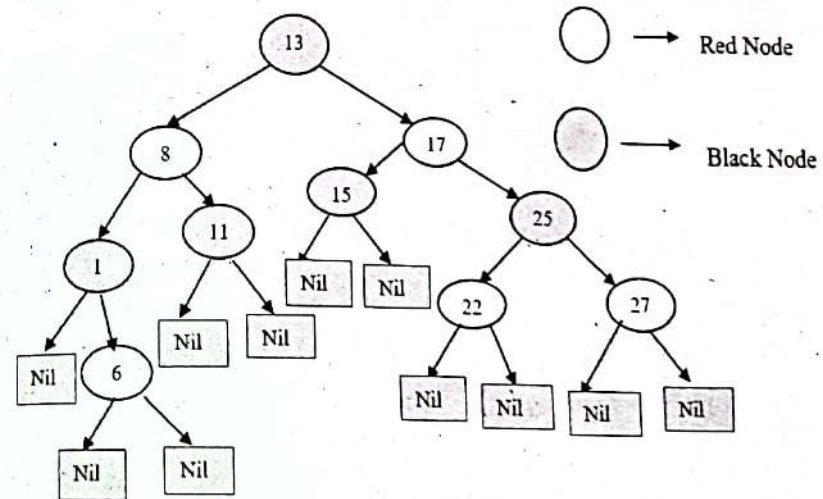


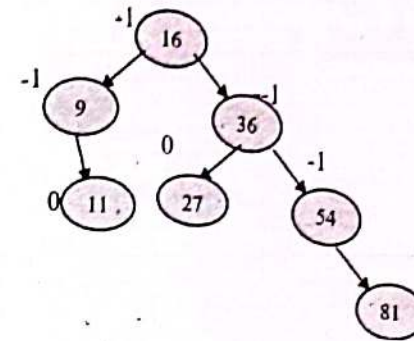
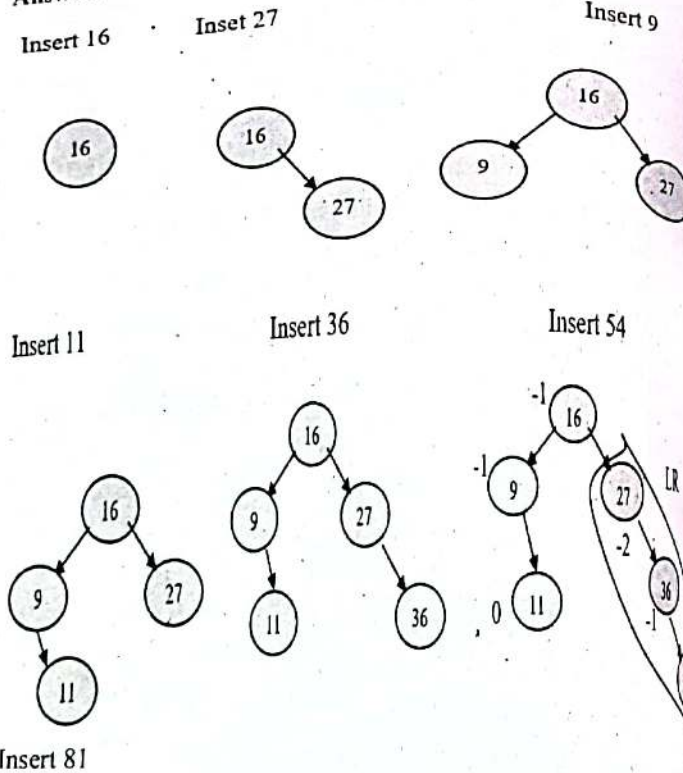
Figure 6.10: Red black tree

- Red black tree has the following red-black properties:
1. Every node is either red or black.
 2. Every leaf (NIL) is black.
 3. If a node is red, then both its children are black.
 4. Every simple path from a node to a descendant leaf contains the same number of black nodes.

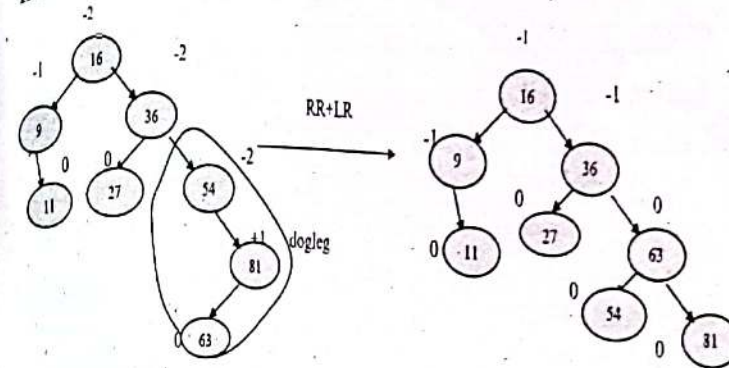
Question:

Construct AVL tree using following sequence of data: 16, 27, 9, 11, 36, 54, 81, 63, 72.

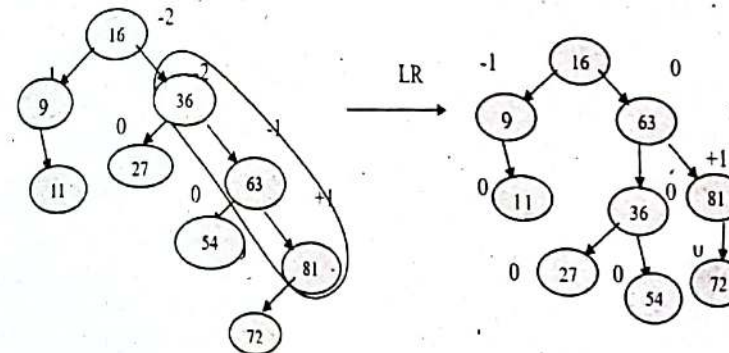
Answer:



Insert 63



Insert 72



(This is required AVL tree)

Program 1: Implementation of Binary Tree.

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
struct bst
{
    struct bst *left;
    int data;
    struct bst *right;
};
typedef struct bst node;
node *insrt(node *,int);
void pre_order(node *);
void in_order(node *);
void post_order(node *);
void search(node *,int);
node *del(node *,int);
node *findmin(node *);
node *findmax(node *);
void destroy(node *);
void internal(node *);
void external(node *);
void child1(node *);
void main()
{
    int n,num;
    node *root=NULL,*temp;
    do
```

```
    printf("\n\n1.CREATE A BINARY TREE");
    printf("\n\n2.PREORDER TRAVERSAL");
    printf("\n\n3.INORDER TRAVERSAL");
    printf("\n\n4.POSTORDER TRAVERSAL");
    printf("\n\n5.SEARCH A NODE VALUE");
    printf("\n\n6.DELETE A NODE");
    printf("\n\n7.FIND MIN NODE");
    printf("\n\n8.FIND MAX NODE");
    printf("\n\n9.DESTROY BINARY TREE");
    printf("\n\n10.INTERNAL NODES");
    printf("\n\n11.EXTERNAL NODES");
    printf("\n\n12.NODE WITH SINGLE CHILD");
    printf("\n\n13.QUIT");
    printf("\n\nEnter ur choice");
    scanf("%d",&n);
    switch(n)
    {
        case 1:
            printf("\nEnter data");
            scanf("%d",&num);
            root=insrt(root,num);
            break;
        case 2:
            if(root==NULL)
                printf("\nThe tree is empty!");
            else
```



```

        pre_order(root);
    getch();
    break;
case 3:
    if(root==NULL)
        printf("\nThe tree is empty!");
    else
        in_order(root);
    getch();
    break;
case 4:
    if(root==NULL)
        printf("\nThe tree is empty!");
    else
        post_order(root);
    getch();
    break;
case 5:
    printf("\nEnter a number to search");
    scanf("%d",&num);
    search(root,num);
    break;
case 6:
    if(root==NULL)
    {
        printf("\n Tree is empty");
        getch();
    }

```

```

    else
    {
        printf("\n Enter No. to delete:");
        scanf("%d",&num);
        root=del(root,num);
    }
    break;
case 7:
    temp=findmin(root);
    if(temp==NULL)
        printf("\nThe tree is empty!");
    else
        printf("\nThe minimum node is %d.",temp->data);
    getch();
    break;
case 8:
    temp=findmax(root);
    if(temp==NULL)
        printf("\nThe tree is empty!");
    else
        printf("\nThe maximum node is %d.",temp->data);
    getch();
    break;
case 9:
    destroy(root);

```

```

        root=NULL;
        printf("\nThe tree is now empty.");
        getch();
        break;

    case 10:
        internal(root);
        getch();
        break;

    case 11:
        external(root);
        getch();
        break;

    case 12:
        child1(root);
        getch();
        break;

    case 13:
    {
        printf("\n\nThank you for using this
        program.\nPress any key to exit.");
        getch();
        exit(1);
    }

    default:
        printf("\nInvalid choice!!!");
        getch();
    }

} while(n!=13);
}

```

```

node *insrt(node *r,int num)
{
    if(r==NULL)
    {
        r=(node *)malloc(sizeof(node));
        r->data=num;
        r->left=r->right=NULL;
    }
    else if(num<r->data)
        r->left=insrt(r->left,num);
    else if(num>r->data)
        r->right=insrt(r->right,num);
    else if(num==r->data)
    {
        printf("\nThis node already exists in the tree");
        getch();
    }
    return r;
}

void pre_order(node *root)
{
    if(root!=NULL)
    {
        printf("%d\n",root->data);
        pre_order(root->left);
        pre_order(root->right);
    }
}

```

```

}
void in_order(node *root)

```

```

{
    if(root!=NULL)
    {

```

```

        in_order(root->left);
        printf("%d\n",root->data);
        in_order(root->right);
    }
}

```

```

void post_order(node *root)

```

```

{
    if(root!=NULL)
    {

```

```

        post_order(root->left);
        post_order(root->right);
        printf("%d\n",root->data);
    }
}

```

```

void search(node *root,int num)

```

```

{
    if(root==NULL)
    {

```

```

        printf("\nSearch Unsuccessful!\nTry other value");
        getch();
        return;
    }

```

```

    else if(num==root->data)

```

```

{
    printf("\nSearch Successful!\nNumber %d is Found!",num);
    getch();
    return;
}

```

```

else if(num<root->data)

```

```

search(root->left,num);

```

```

else

```

```

search(root->right,num);
}

```

```

node *del(node *r,int num)

```

```

{
    node *temp,*root;

```

```

    root=r;

```

```

    if(root==NULL)

```

```

        return root;

```

```

    else if(num<root->data) //Search in left subtree

```

```

        root->left=del(root->left,num);

```

```

    else if(num>root->data) //Search in right subtree

```

```

        root->right=del(root->right,num);

```

```

    else if(root->left!=NULL && root->right!=NULL)

```

```

        //if both the subtrees are present

```

```

        {

```

```

            temp=findmin(root->right);

```

```

            root->data=temp->data;

```

```

            root->right=del(root->right,root->data);
        }
    }
}

```



```

//if one of the subtree is null
else
{
    temp=root;
    if(root->left==NULL)
        root=root->right;
    else if(root->right==NULL)
        root=root->left;
    free(temp);
}
return root;
}
node *findmin(node *root)
{
    node *temp;
    temp=root;
    if(temp==NULL)
        return NULL;
    if(temp->left==NULL)
        return(temp);
    else
        return(findmin(temp->left));
}
node *findmax(node *root)
{
    node *temp;
    temp=root;
    if(temp==NULL)
        return NULL;

```

```

if(temp->right==NULL)
    return(temp);
else
    return(findmax(temp->right));
}
void destroy(node *root)
{
    node *t;
    t=root;
    if(t!=NULL)
    {
        destroy(t->left);
        destroy(t->right);
        printf("\nValue %d is deleted.",t->data);
        free(t);
    }
}
void internal(node *root)
{
    if(root!=NULL)
    {
        if(root->left!=NULL || root->right!=NULL)
            printf("\n%d",root->data);
        internal(root->left);
        internal(root->right);
    }
}

```

```

void external(node *root)
{
    if(root!=NULL)
    {
        if(root->left==NULL && root->right==NULL)
            printf("\n%d",root->data);
        external(root->left);
        external(root->right);
    }
}

void child1(node *root)
{
    if(root!=NULL)
    {
        if(root->left==NULL)
        {
            if(root->right!=NULL)
                printf("\n%d",root->data);
        }
        else if(root->right==NULL)
        {
            if(root->left!=NULL)
                printf("\n%d",root->data);
        }
        child1(root->left);
        child1(root->right);
    }
}

```

Chapter 7: Sorting

7.1 Introduction

The concept of an ordered set of elements is one that has considerable impact in our daily lives. So, sorting is one of the most common ingredients of programming systems. The process of rearranging the items in a list according to some linear order is termed as sorting.

- The process of finding a telephone number in a telephone directory is simplified considerably by the fact that the names are listed in alphabetical order.
- In a library, books are shelved in a specific order, each book is assigned a specific position relative to the others and can be retrieved in a reasonable amount of time.

Stable Sort

- It is possible for two records in a list to have the same key.
- A sorting algorithm is stable if for all records i and j such that $k[i]$ equals $k[j]$, if $r[i]$ preceded $r[j]$ in the original list, $r[i]$ precedes $r[j]$ in the sorted list.
 - i.e., a stable sort keeps records with same key in the same relative order that they were in before the sort.

Original Table	
Name	Address
AAB	BBC
CDI	LKK
AAB	BBA
KKA	KSO
IEH	IEU

Sorted Table	
Name	Address
AAB	BBC
AAB	BBA
CDI	LKK
IEH	IEU
KKA	KSO

This sort is known as stable sort. In normal considerations, the data is sorted like:

AAB BBA
AAB BBC

But the stable sort keeps track of original pattern unless specified.

Types of Sorting

- **Internal sorting:** Records to be sorted are in main memory.
- **External sorting:** Records to be sorted, or some of them kept in auxiliary storage (disk/tape).

7.2 Sorting Algorithms

1. Exchange sort

Comparison based. The basic idea is to compare two elements; if out of order, swap them or move one of the elements. Example: bubble sort, quick sort.

2. Selection sort

An element is selected and is placed in its correct position. Example: selection sort, heap sort.

3. Insertion sort

Sorts by inserting an element into a sorted list. Example: insertion sort, merge sort.

7.3 Bubble sort

Basic Idea: Pass through the list sequentially several times. At each pass, each element in the list is compared to its successor and they are interchanged if they are not in proper order.

At each pass, one element will be in its proper position. In general, $A[n-i]$ will be in its place after pass i . Since each pass place a new element in its proper position, a total of $N-1$ passes are required for a list of N elements. Also, all the elements in positions greater than or equal to $N-i$ are already in proper position after pass i , so they need not be considered in succeeding passes.

C Function

```
void bubblesort(int a[], int N)
{
    int pass, j;
    for(pass=0; pass<N-1; pass++)
    {
        for(j=0; j<N-pass-1; j++)
            if(a[j]>a[j+1])
                swap(&a[j], &a[j+1]);
    }
}
```

Example: Sort the following numbers using bubble sort.
25, 57, 48, 37, 12, 92, 86, 33.

Solution:

Total no. of items $N = 8$

25 57 48 37 12 92 86 33 Interchange

25	57	48	37	12	92	86	33	No	} 1 st Pass
25	57	48	37	12	92	86	33	Yes	
25	48	57	37	12	92	86	33	Yes	
25	48	37	57	12	92	86	33	Yes	
25	48	37	12	57	92	86	33	No	
25	48	37	12	57	92	86	33	Yes	
25	48	37	12	57	86	92	33	Yes	
25	48	37	12	57	86	33	92		

25	48	37	12	57	86	33	92	No	} 2 nd Pass
25	48	37	12	57	86	33	92	Yes	
25	37	48	12	57	86	33	92	Yes	
25	37	12	48	57	86	33	92	No	
25	37	12	48	57	86	33	92	No	
25	37	12	48	57	86	33	92	Yes	
25	37	12	48	57	33	86	92		

25	37	12	48	57	33	86	92	Interchange	
25	37	12	48	57	33	86	92	No	
25	37	12	48	57	33	86	92	Yes	
25	37	12	48	57	33	86	92	No	
25	12	37	48	57	33	86	92	No	
25	12	37	48	57	33	86	92	Yes	1st Pass
25	12	37	48	33	57	86	92		
25	12	37	48	33	57	86	92	Interchange	
25	12	37	48	33	57	86	92	Yes	
25	12	37	48	33	57	86	92	No	
12	25	37	48	33	57	86	92	No	
12	25	37	48	33	57	86	92	Yes	4th Pass
12	25	37	33	48	57	86	92		
12	25	37	33	48	57	86	92	Interchange	
12	25	37	33	48	57	86	92	No	
12	25	37	33	48	57	86	92	No	
12	25	37	33	48	57	86	92	Yes	5th Pass
12	25	33	37	48	57	86	92		
12	25	33	37	48	57	86	92	Interchange	
12	25	33	37	48	57	86	92	No	
12	25	33	37	48	57	86	92	No	6th Pass
12	25	33	37	48	57	86	92		
12	25	33	37	48	57	86	92	Interchange	
12	25	33	37	48	57	86	92	No	
12	25	33	37	48	57	86	92		7th Pass

Algorithm

- Given a list A of size N, the following algorithm uses bubble sort to sort the list
 - For pass = 0 To N - 2
 - For j = 0 To N - pass - 2
 - If $A[j] > A[j + 1]$

Swap the elements $A[j]$ and $A[j + 1]$
End If

End For

End For

Efficiency

This algorithm is good for small n usually less than 100 elements.

No. of comparisons: $(n-1) + (n-2) + \dots + 2 + 1$
 $= (n-1)(n-1+1)/2$
 $= n(n-1)/2$
 $= O(n^2)$

No. of interchanges:

- This cannot be greater than no. of comparisons
- In the best case, there are no interchanges
- In the worst case, this equals no of comparisons

The average and worse case running time of bubble sort is $O(n^2)$.

It is actually the no of interchanges which takes up most time of the program's execution than the no of comparisons.

When elements are large and interchange operation is expensive, it is better to maintain an array of pointers to the elements. One can then interchange pointers rather than the elements itself.

7.4 Insertion Sort

Basic idea: Sorts a list of record by inserting new element into an existing sorted list. An initial list with only one item is considered to be sorted list. For a list of size N, N-1 passes are made, and for each pass the elements from $a[0]$ through $a[i-1]$ are sorted.

Take the element $a[i]$, find the proper place to insert $a[i]$ within 0, 1, ..., i-1 and insert $a[i]$ at that place.

To insert new item into the list:

- Search the position in the sorted sublist from last toward first.
- While searching, move elements one position right to make a room to insert $a[i]$.
- Place $a[i]$ in its proper place.

Example: Sort the following numbers using Insertion sort.
25, 57, 48, 37, 12, 92, 86, 33.

Solution:

Initially	25 57 48 37 12 92 86 33	
Pass 1	<u>25 57</u> 48 37 12 92 86 33	Insert 57
Pass 2	25 <u>48 57</u> 37 12 92 86 33	Insert 48
Pass 3	25 37 <u>48 57</u> 12 92 86 33	Insert 37
Pass 4	12 25 37 <u>48 57</u> 92 86 33	Insert 12
Pass 5	12 25 37 <u>48 57 92</u> 86 33	Insert 92
Pass 6	12 25 37 <u>48 57 86 92</u> 33	Insert 86
Pass 7	12 25 33 37 48 57 86 92	Insert 33

C Function

```
void InsertionSort(int a[], int N)
```

```
{
    int i, j;
    int hold; //the current element to insert
    for (i = 1; i < N; i++) // Insert a[i] into the sorted list
    {
        hold = a[i]; // hold the element to be inserted
        for (j = i-1; j >= 0 && a[j] > hold; j--) //Move right
            position all
        a[j+1] = a[j]; //elements greater than hold
        a[j+1] = hold; // Place hold in its proper place
    }
}
```

Efficiency

No of comparisons:

Best case: $n - 1$

Worst case: $\frac{n^2}{2} + O(n)$

Average case: $\frac{n^2}{4} + O(n)$

Hence running time of insertion sort is $O(n^2)$ in worst and average case and $O(n)$ in best case and space requirement is $O(1)$.

Advantages:

It is an excellent method whenever a list is nearly in the correct order and few items are removed from their correct locations.

Since there is no swapping, it is twice as faster than bubble sort.

Disadvantage:

It makes a large amount of shifting of sorted elements when inserting later elements.

7.5 Selection Sort

The selection sort algorithm sorts a list by selecting successive elements in order and placing into their proper sorted positions.

A list of size N require $N-1$ passes:

For each pass I ,

- Find the position of i^{th} largest (or smallest) element.
- To place the i^{th} largest (of smallest) in its proper position, swap this element with the element currently in the position of its largest (or smallest) element.

C Function

```
void SelectionSort(int a[], int N)
```

```
{
    int i, j;
    int maxpos;
    for (i = N-1; i > 0; i--) //Find the position of largest
        element from 0 to i
    {
        maxpos = 0;
        for (j = 1; j <= i; j++)
            if (a[j] > a[maxpos])
                maxpos = j;
        if (maxpos != i)
            swap(&a[maxpos], &a[i]);
        //Place the ith largest element in its place
    }
}
```

Example: Sort the following numbers using selection sort.
25, 57, 48, 37, 12, 92, 86, 33.

Solution:

Initially 25 57 48 37 12 92 86 33

Find largest between a[0] and a[7] -> 92, swap 92 with the last element 33

Pass 1 25 57 48 37 12 33 86 92

Find largest between a[0] and a[6] -> 86, since 86 is in 6th position and so is i, no interchange.

Pass 2 25 57 48 37 12 33 86 92

Find largest between a[0] and a[5] -> 57, swap with 33

Pass 3 25 33 48 37 12 57 86 92

Find largest between a[0] and a[4] -> 48, swap 48 with 12

Pass 4 25 33 12 37 48 57 86 92

Find largest between a[0] and a[3] -> 37, No swap since i = maxpos

Pass 5 25 33 12 37 48 57 86 92

Find largest between a[0] and a[2] -> 33, swap 33 with 12

Pass 6 25 12 33 37 48 57 86 92

Find largest between a[0] and a[1] -> 25, swap 12 with 25

Pass 7 12 25 33 37 48 57 86 92

Finally the list is sorted.

Efficiency

No of comparisons:

Best, average and worst case: $n(n-1)/2$

No of assignments (movements):

Best, average and worst case: $3(n-1)$, (total $n-1$ swaps)

If we include a test, to prevent interchanging an element with itself, the number of interchanges in the best case would be 0.

Hence running time of selection sort is $O(n^2)$ and additional space requirements is $O(1)$.

Advantages:

- It is the best algorithm in regard to data movement.
- An element that is in its correct final position will never be moved and only one swap is needed to place an element in its proper position.

Disadvantage:

- In case of number of comparisons, it pays no attention to the original ordering of the list. For a list that is nearly correct to begin with, selection sort is slower than insertion sort.

7.6 Divide and Conquer Sorting

- The idea of dividing a problem into smaller but similar subproblems is called divide and conquer.
- Divide and Conquer Sorting
 - Procedure Sort(list)
 - If (list has length greater than 1)
 - Partition the list into two sublists lowlist, highlist
 - Sort(lowlist)
 - Sort(highlist)
 - Combine (lowlist, highlist)
 - End If
 - End Procedure

7.6.1 Quick Sort

It is the fastest known sorting algorithms used in practice.

Basic idea:

Divide the list into two sublists such that all elements in the first list is less than some pivot key and all elements in the second list is greater than the pivot key, and finally sort the sublists independently and combine them.

Algorithm

If size of list A is greater than 1

- Pick any element v from A. This is called the pivot
- Partition the list A by placing v in some position j , such that
 - all elements before position j are less than or equal to v
 - all elements after position j are greater than or equal to v
- Recursively sort the sublists $A[0]$ through $A[j-1]$ and $A[j+1]$ through $A[N-1]$

- Return $A[0]$ through $A[j-1]$ followed by $A[j]$ (the pivot) followed by $A[j+1]$ through $A[N-1]$

Example: Sort the following numbers using quick sort.
25, 57, 48, 37, 12, 92, 86, 33.

Solution:

Initially 25 57 48 37 12 92 86 33

Choose the first element 25 as the pivot and partition the array

(12) 25 (57 48 37 92 86 33)

The first subarray is automatically sorted

12 25 (57 48 37 92 86 33)

Choose the first element 57 of the second subarray as the pivot and partition the subarray

12 25 (48 37 33) 57 (92 86)

12 25 (48 37 33) 57 (92 86)
12 25 (37 33) 48 57 (92 86)
12 25 (33) 37 48 57 (92 86)
12 25 33 37 48 57 (92 86)
12 25 33 37 48 57 (92 86)
12 25 33 37 48 57 (86) 92
12 25 33 37 48 57 86 92

Description of partition procedure

- Choose any element as the pivot, here we choose the first element as the first item in the list.
- Initialize two pointers, up and down to the upper bound (low) and lower bound (high) of the array
- While down is left of up, repeat these steps

- Move down right, skipping over elements that are smaller than or equal to pivot.
- Move up left, skipping over elements that are larger than the pivot.
- When down and up have stopped, down is pointing at a large element and up is pointing at a small element.
- If down is left of up, swap the elements at down and up (The effect is to push a large element to the right and a small element to the left).
- Swap the first element (pivot) with the element at up.
- Return up as the pivot location.

The partition procedure of quick sort for the above example is shown below.

25	57	48	37	12	92	86	33	
Choose Pivot 25								
down								up
25	57	48	37	12	92	86	33	
	down							up
25	57	48	37	12	92	86	33	
	down			up				
25	57	48	37	12	92	86	33	
down < up, so swap up and down and increment down								
		down		up				
25	12	48	37	57	92	86	33	down > up stop

	up	down						
25	12	48	37	57	92	86	33	down >= up, swap with up
(12)	25	(48	37	57	92	86	33)	
List is divided into two lists, pivot is at its proper position. First list contains only one element so automatically sorted. Now choose 48 as pivot element for second list.								
Choose pivot	down						up	
48								
12	25	48	37	57	92	86	33	
				down			up	
12	25	48	37	57	92	86	33	down < up so, swap
				down			up	
12	25	48	37	33	92	86	57	
					down		up	Move down
12	25	48	37	33	92	86	57	
				Up	down		up	Move up
12	25	48	37	33	92	86	57	
12	25	(33	37)	48	(92	86	57)	down >= up so, swap
Sublist further got divided into two sublists. Pivot 48 is at its proper position								

12	25	(33	37)	48	(92	86	57)	
		down	up					
12	25	(33	37)	48	(92	86	57)	Choose 21 as pivot
			down	up				Move down
12	25	(33	37)	48	(92	86	57)	Move up
		up	down					
12	25	(33	37)	48	(92	86	57)	down > up, so swap
12	25	33	(37)	48	(92	86	57)	37 is single list so automatically sorted
Pivot 92				down		up		
12	25	33	37	48	92	86	57	Move down
								down up
12	25	33	37	48	92	86	57	down > up
12	25	33	37	48	(57	86)	92	
Pivot 57				down	up			Move down
12	25	33	37	48	57	86	92	
					down up			
12	25	33	37	48	57	86	92	Move up
				up	down			

12	25	33	37	48	57	86	92	down >= up, so swap
12	25	33	37	48	57	(86)	92	

Sublist 86 automatically is sorted.

Method for choosing pivot

First element: Choose the first item in the list.

Random element: Choose any item from the list. Swap it with the first item to apply the algorithm.

Median: Pick three elements randomly and use their median as pivot.

Efficiency

No. of comparisons:

Average case: $O(n \log n)$

Worst case: $O(n^2)$

No of interchanges (swaps):

Average case: $O(n \log n)$

Worst case: $O(n^2)$

Hence, the time complexity of QuickSort is $O(n \log n)$ for average case and $O(n^2)$ for worst case.

7.6.2 Merge Sort

The merge sort also uses divide and conquer approach. It divides the list into sub lists. Then merge two sorted into a single list.

Algorithm

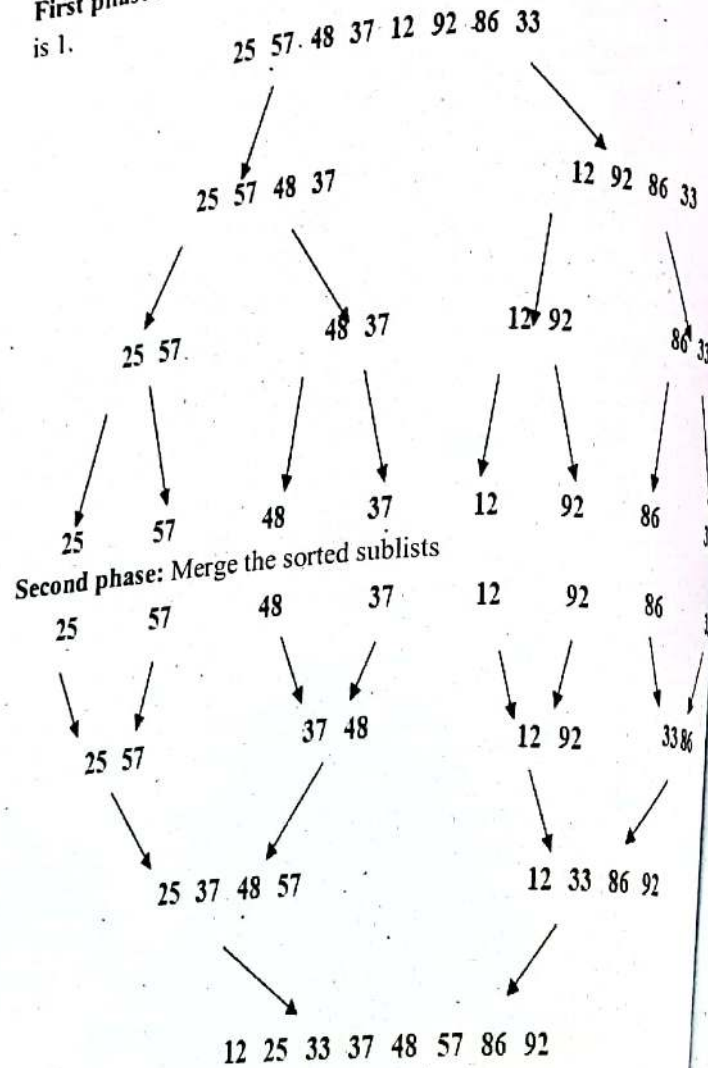
- If size of list is greater than 1
 - Divide the list into two sublists of sizes nearly equal as possible.
 - Recursively sort the sublist separately.
 - Merge the two sorted sublists into a single sorted list.

• End if

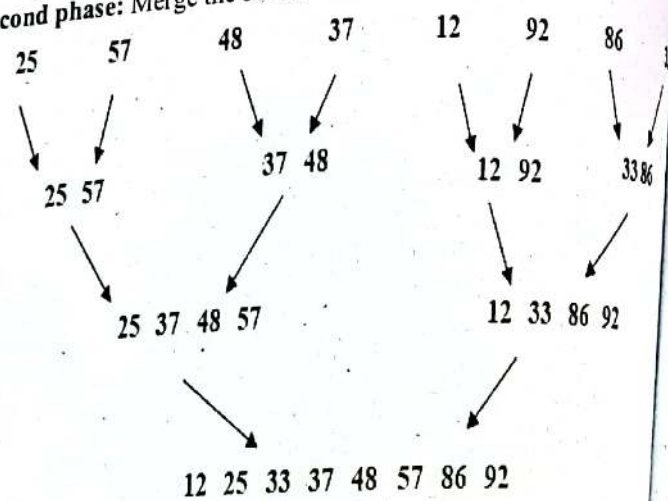
Example: Sort the following numbers using merge sort.
25, 57, 48, 37, 12, 92, 86, 33.

Solution:

First phase: Partition the list in two equal halves, until the list size is 1.



Second phase: Merge the sorted sublists



Merging Description

To simply, the algorithm merge to sorted sublist into a third list.

When finished, we copy back the third list in the original sorted halves to get the sorted list.

The basic merging algorithm takes two input sorted arrays A and B, and an output array C. We initialize the pointers Aptr, Bptr, and Cptr to point the beginning of their respective arrays. The smaller of A[Aptr] and B[Bptr] is copied to the next entry in C and appropriate pointers are advanced. When any one of the list has been finished, the remainder of the other list is copied to C.

Aptr

25	37	48	57
----	----	----	----

Bptr

12	33	86	92
----	----	----	----

Cptr

12			
----	--	--	--

Compare 25 and 12 and insert minimum (12) into array C and move Bptr and Cptr

Aptr

25	37	48	57
----	----	----	----

Bptr

12	33	86	92
----	----	----	----

Cptr

12				
----	--	--	--	--

Compare 25 and 33 and insert minimum (25) into array C and move Aptr and Cptr

Aptr			
25	37	48	57
Bptr			
12	33	86	92
Cptr			
12	25		

Compare 37 and 33 and insert minimum (33) into array C and move Bptr and Cptr and so on.
In this way the two lists are merged.

Efficiency

No. of comparisons:

For all cases the number of comparisons is to be $O(n \times \log n)$, the constant term is different for different cases. On average, it requires fewer than $n \times \log n - n + 1$ comparisons.

No. of assignments:

For our implementation, it is twice the no of comparisons, merging in the temporary array and copying back to the original array which is still $O(n \times \log n)$.

Space complexity:

In contrast to other sorting algorithms, Merge sort requires $O(n)$ extra space for the temporary memory used while merging. Algorithm has been developed for performing in-place merge in $O(n)$ time, but this would increase the number of assignments.

If recursive version is used, additional space is required for the implicit stack, which is $O(\log n)$. Hence, Space complexity for merge sort is $O(n)$.

Notes on merge sort

Even though the worst-case running-time of Merge Sort is $O(n \log n)$, it is not an algorithm of choice for sorting contiguous lists. Merge Sort can prove superior over other sorting algorithms when used with linked lists.

7.7 Binary Tree Sort

General idea is to create a binary search tree and access the elements either in LVR and RVL for ascending and descending order.

But, in case of imbalanced tree (right skewed and left skewed), the search time goes approximately n^2 . Therefore, to minimize the search time, AVL trees are maintained. This will increase performance up to $n \log n$. Still, BST requires some time to search and retrieve the data. After deletion of elements, there are some burden to maintain the BST property. It mean, the tree is accessed 2 times. To minimize the time for retrieval, heap is created. In heap sort, the heap creation takes time, but the retrieval takes no time.

7.7.1 Heap Sort

The heap sort algorithm sorts by representing its input as a heap in the array

- Two phases in sorting
 - First phase: Converts the array representation of the tree into a heap.
 - Second phase: Repeatedly moves the largest element to the last position by swapping the first element with the last element and adjusts the heap property at each stage in the remaining elements.

First phase

- The entries in the array being sorted are interpreted as a binary tree in array implementation.
- Tree with only one node automatically satisfies the heap property. So, we don't need to worry about any of the leaves.

3. Start from the level above the leaf nodes, and work on backward towards the root.

Second phase

1. Note that the root (the first element of the array) has the largest key.
2. Repeat these steps until the size of heap becomes 1
 - Move the largest key at root to the last position of the heap, replacing an entry x currently at the last position.
 - Decrease a counter i that keeps track of the size of the heap, thereby excluding the largest key from further sorting.
 - The element x may not belong to the root of the heap, so insert x into the proper position to restore the heap property.

Adjusting the heap properties

When the last element, x , is replaced by the largest element at the root, a hole is created at the root and the heap size becomes smaller by 1. We must move x somewhere to restore the heap property.

1. First we look if x can be placed in the hole, by looking at the two children of that hole.
2. If x belongs to the hole, then we put x there and we are done.
3. Else we slide the larger of the two children to the hole, thus pushing the hole down one level.
4. We repeat this process on the subtree until x can be placed in the hole or there are no children.
5. Thus, our action is to place x in its correct spot along a path from the root containing minimum children.

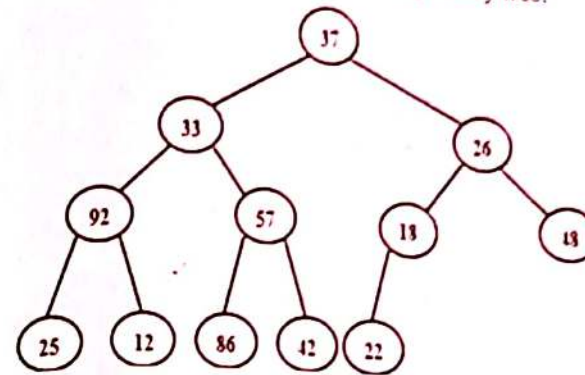
Example: Sort the following numbers using heap sort: 37, 33, 26, 92, 57, 18, 48, 25, 12, 86, 42, 22.

Solution:

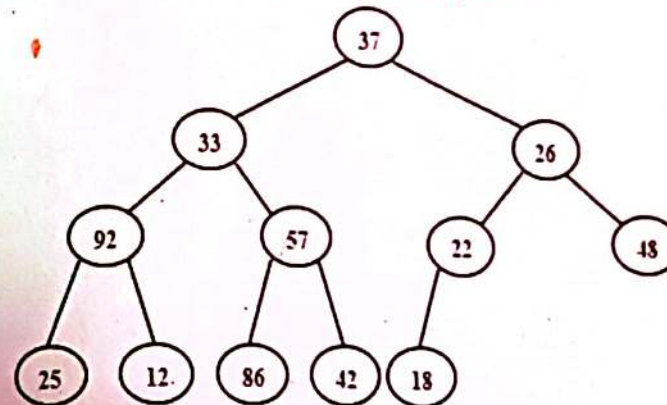
First phase:

37 33 26 92 57 18 48 25 12 86 42 22

This array can be represented as following binary tree:

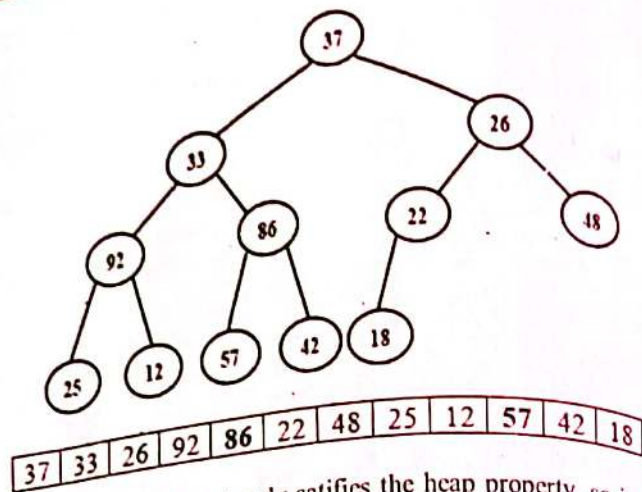


This is not a heap. So, adjust to convert it to max heap as follows:
Leave the leaf nodes. Adjust heap property at 18:

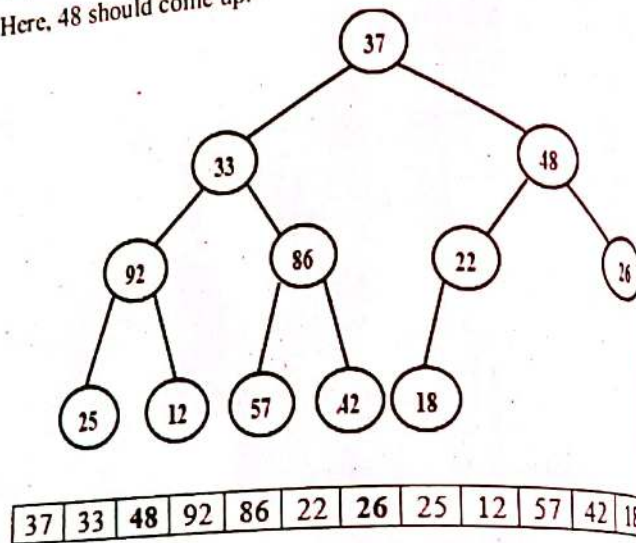


37 33 26 92 57 22 48 25 12 86 42 18

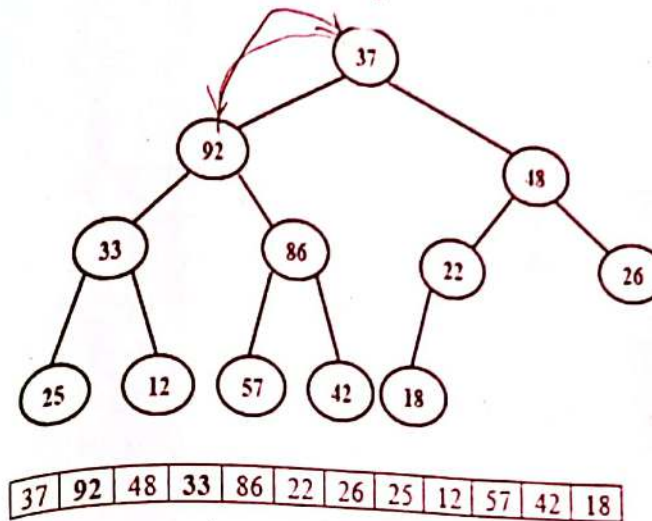
Adjust heap property at 57:



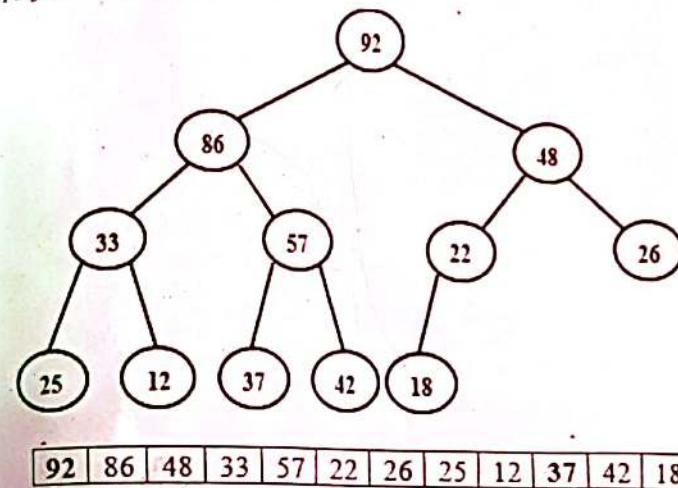
At 92 the subtree already satisfies the heap property, so it will remain as it is.
Adjust heap property at 26:
Here, 48 should come up.



Adjust heap property at 33:



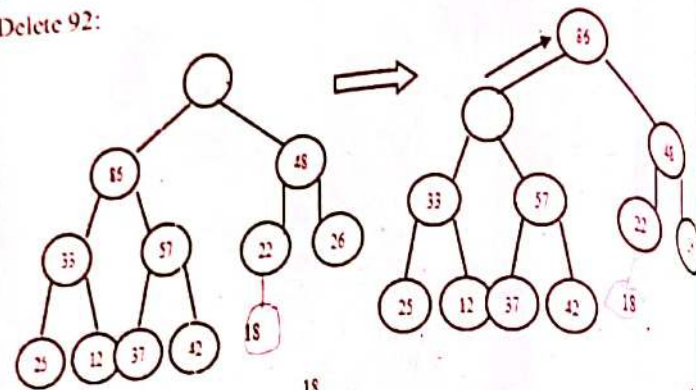
Adjust heap property at 37:



Second phase:

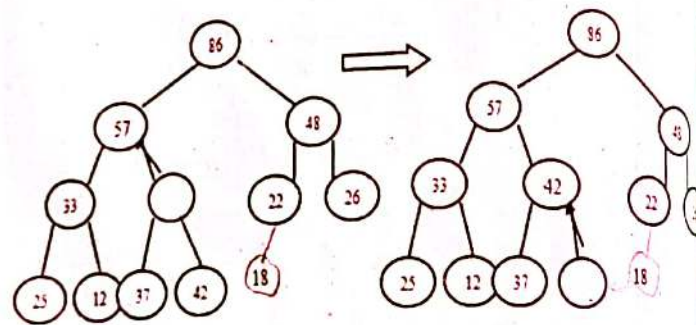
Move the largest element, 92, to the last position of the heap, thus replacing 18. Now, we must find a way to put 18 in its proper position, thus restoring the heap property.

Delete 92:



86	48	33	57	22	26	25	12	37	42	92
0	1	2	3	4	5	6	7	8	9	10

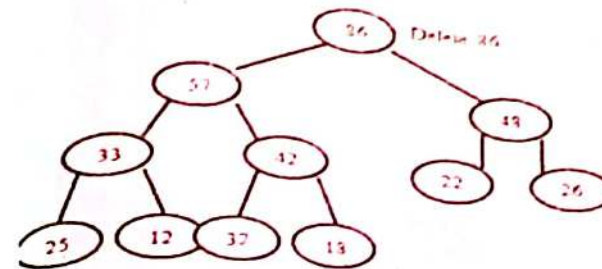
86	48	33	57	22	26	25	12	37	42
0	1	2	3	4	5	6	7	8	9



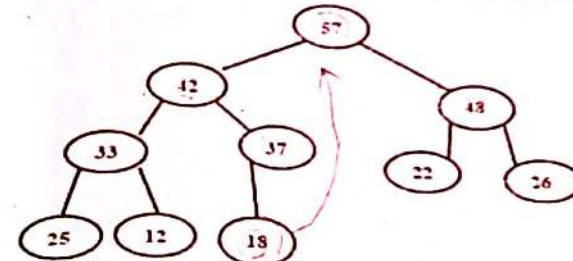
86	57	48	33	22	26	25	12	37	42
0	1	2	3	4	5	6	7	8	9

86	57	48	33	42	22	26	25	12	37
0	1	2	3	4	5	6	7	8	9

Delete 86:

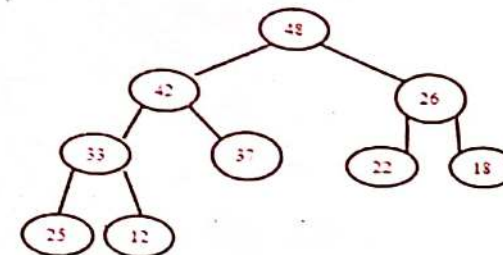


86	57	48	33	42	22	26	25	12	37	18	92
0	1	2	3	4	5	6	7	8	9	10	11



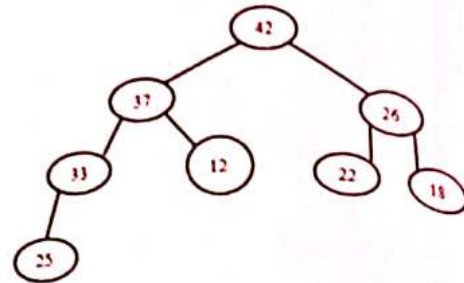
57	42	48	33	37	22	26	25	12	18	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 57



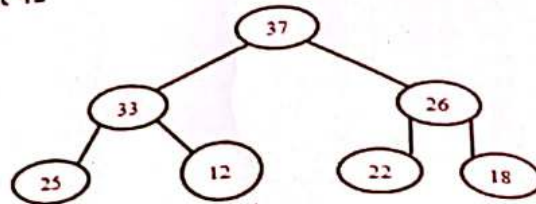
48	42	26	33	37	22	18	25	12	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 48



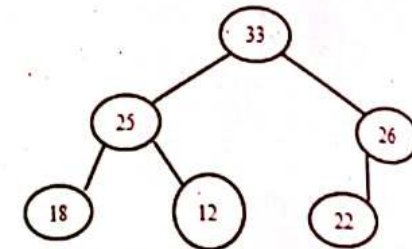
42	37	26	33	12	22	18	25	48	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 42



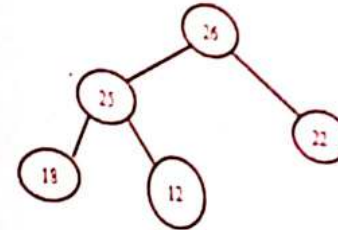
37	33	26	25	12	22	18	42	48	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 37



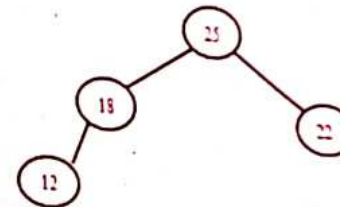
33	25	26	18	12	22	37	42	48	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 33



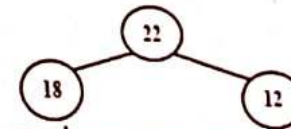
26	25	22	18	12	33	37	42	48	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 26



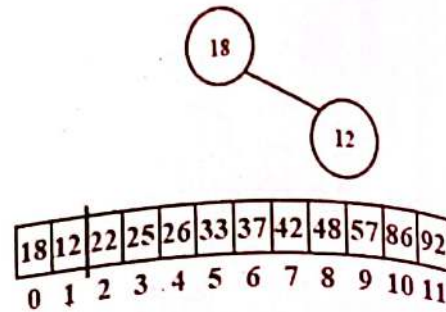
25	18	22	12	26	33	37	42	48	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

Delete 25

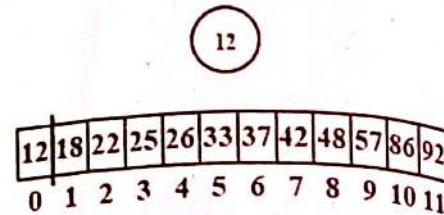


22	18	12	25	26	33	37	42	48	57	86	92
0	1	2	3	4	5	6	7	8	9	10	11

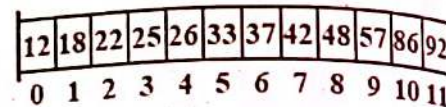
Delete 22



Delete 18



Delete 12



Finally, the data are sorted.

Efficiency

- No of comparisons and assignments
 - Worst-case: $O(n \log n)$
 - Average case: $O(n \log n)$
- Hence time complexity of heap sort is $O(n \log n)$ for both worst case and average case and space complexity is $O(1)$. In average case, it is not as efficient as quick sort, however, it is far superior to quick sort in worse case. Generally, heap sort is used for large amount of data.

7.8 Heap as Priority Queue

A priority queue is a data structure with the following primitive operations

- Insert an item.
- Remove the item having the largest (or smallest) key.

Implementations:

- Use a sorted contiguous list, removal takes $O(1)$ but insertion takes $O(n)$.
- Use an unsorted list, insertion takes $O(1)$ but removal takes $O(n)$.

Efficiency of Priority Queue

Consider the properties of heap:

- The item with largest key is on the top and can be removed immediately. However it will take time $O(\log n)$ to restore the heap property for remaining keys.
- For insertion we shift the new item from down to up which also takes $O(\log n)$.

Hence, implementation of a priority queue as a heap proves advantageous for large n

- It efficiently represents in contiguous storage and is guaranteed to require only logarithmic time for both insertions and deletions.

7.9 Shell Sort

Significant improvement on simple insertion sort can be achieved by using shell sort (or diminishing increment sort). This method separates original file into subfiles. These subfiles contain every k^{th} element of the original file. The value of k is called an increment. Example: If $k=5$, then subfile consists of $x[0]$, $x[5]$, $x[10]$, ... is first sorted.

After the first k subfiles are sorted (usually by simply insertion), a new smaller value of k is chosen and the file is again partitioned into a new set of subfiles. Each of these larger subfiles is sorted and the process is repeated yet again, until eventually the value of the k is set to 1.

The decreasing sequence of increments can:

Either be fixed at the start of the entire process. The last value must be 1

Or take the first increment to $hk = \text{floor}(N/2)$ and $hk = \text{floor}(hk/2)$ until $hk = 1$. $hk =$ subsequent increment.

Example: Sort the following numbers using shell sort:
81, 94, 11, 96, 12, 35, 17, 95, 28, 58, 41, 75, 15.

Solution:

Let $hk = 13/2 = 6$, so here increment is 6. The shell sort will be sub divided into 6 sub files.

Sub files	Sorted sub files
81 17 15	15 17 81
94 95	94 95
11 28	11 28
96 58	58 96
12 41	12 41
35 75	35 75

After 1st iteration, the list will look like:

15 94 11 58 12 35 17 95 28 96 41 75 81

Now, in second iteration $hk = \text{floor}(hk/2) = \text{floor}(6/2) = 3$

Subfiles	Sorted subfiles
15 58 17 96 81	15 17 58 81 96
94 12 95 41	12 41 94 95
11 35 28 75	11 28 35 75

After 2nd iteration, the list will be as follows:

15 12 11 17 41 28 58 94 35 81 95 75 96

Now, $hk = \text{floor}(hk/2) = 3/2 = 1$

15 12 11 17 41 28 58 94 35 81 95 75 96

Now, using simple insertion sort we get,

15 12 11 17 41 28 58 94 35 81 95 75 96

12 15 11 17 41 28 58 94 35 81 95 75 96

11 12 15 17 41 28 58 94 35 81 95 75 96

11 12 15 17 41 28 58 94 35 81 95 75 96

11 12 15 17 41 28 58 94 35 81 95 75 96

11 12 15 17 28 41 58 94 35 81 95 75 96

11 12 15 17 28 41 58 94 35 81 95 75 96

11 12 15 17 28 41 58 94 35 81 95 75 96

11 12 15 17 35 28 41 58 94 81 95 75 96

11 12 15 17 35 28 41 58 81 94 95 75 96

11 12 15 17 35 28 41 58 81 94 95 75 96

11 12 15 17 35 28 41 58 75 81 94 95 96

11 12 15 17 35 28 41 58 75 81 94 95 96

Hence, the list is finally sorted.

Efficiency

Worse case: $O(n^2)$

Average case: $O(n(\log n)^2)$

7.10 Radix Sort

The sorting is based on the values of the actual digits in the positional representations of the numbers being sorted.

Process

Beginning with the least-significant digit and ending with the most-significant digit, perform the following action.

Take each number in the order in which it appears in the file and place it into one of the ten queues, depending on the value of the digit currently being processed.

Then restore each queue to the original file starting with the queue of numbers with a 0 digit and ending with the queue of numbers with a 9 digit.

When these actions have been performed for each digit, starting with the least significant digit and ending with the most significant, the file is sorted.

Example: Sort the following numbers using radix sort:

64, 8, 216, 512, 27, 729, 0, 1, 343, 125

Solution:

Initially: 64, 8, 216, 512, 27, 729, 0, 1, 343, 125

Now, place all the numbers in appropriate queue.

First Pass

	0	1	512	343	64	125	216	27	8	729
no%10	0	1	2	3	4	5	6	7	8	9

Second Pass

	8		729							
	1	216	27							
	0	512	125		343		64			
(no/10)%10	0	1	2	3	4	5	6	7	8	9

Third Pass

	64									
	27									
	8									
	1									
	0	125	216	343		512		729		
(no/100)%10	0	1	2	3	4	5	6	7	8	9

Finally, we have: 0, 1, 8, 27, 64, 125, 216, 343, 512, 729.

Here, the no. of passes equals maximum number of digits in the given numbers to be sorted.

Efficiency

Efficiency of radix sort is $O(n \cdot \log n)$.

7.11 Comparison Table

Algorithms	Worse Case	Average Case
Bubble Sort	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n^2)$	$O(n \cdot \log n)$
Insertion Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \cdot \log n)$	$O(n \cdot \log n)$
Heap Sort	$O(n \cdot \log n)$	$O(n \cdot \log n)$
Radix Sort	$O(n \cdot \log n)$	$O(n \cdot \log n)$

7.12 Selecting a Sort Algorithm

Algorithms	Comments
Bubble Sort	Good for small n usually less than 100
Quick Sort	Excellent for virtual memory environment
Insertion Sort	Good for almost sorted records
Selection Sort	Good for partially sorted data and small 'n'
Merge Sort	Good for external file sorting
Heap Sort	As efficient as quick sort in average case and far superior to quick sort in the worse case
Radix Sort	Good when number of digits(letters) are less

Question:

Sort the following number using radix sort: 141, 456, 342, 789, 90, 2, 870, 192, 938 and 243. [2075 Bhadra]

Answer:

Initially: 141, 456, 342, 789, 90, 2, 870, 192, 938 and 243

Now, place all the numbers in appropriate queue.

First Pass									
no % 10									
		192							
		2							
870									
90	141	342	243			456		936	789
0	1	2	3	4	5	6	7	8	9

Second Pass

				243					
				342					
								192	
2			938	141	456		870	789	90
0	1	2	3	4	5	6	7	8	9

Third Pass

90	192								
2	141	243	342	456			789	870	938
0	1	2	3	4	5	6	7	8	9

So, sorted numbers are : 2, 90, 141, 192, 243, 342, 456, 789, 870 and 938.

Program 1: Implementation of Merge Sort.

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<string.h>
#define n 10
void msort(int x[], int temp[], int left, int right);
void merge(int x[], int temp[], int left, int mid, int right);
void main()
{
    int x[n],temp[n],i;
    printf("\n***SELECTION SORT***\n\n");
    printf("\nEnter the 10 numbers\n");
    for (i=0;i<n;i++)
```

```

    {
        scanf("%d",&x[i]);
    }

    msort(x,temp,0,n-1);
    printf("\n\nThe sorted result is:");
    for(i=0;i<n;i++)
    {
        printf("\n%d",x[i]);
    }

    getch();
}

void msort(int x[], int temp[], int left, int right)
{
    int mid;
    if(left<right)
    {
        mid=(right+left)/2;
        msort(x,temp,left,mid);
        msort(x,temp,mid+1,right);
        merge(x,temp,left,mid+1,right);
    }
}

void merge(int x[], int temp[], int left, int mid, int right)
{
    int i,lend,noelement,tmpos;
    lend=mid-1;
    tmpos=left;
    noelement=right-left+1;

```

```

while((left<=lend)&&(mid<=right))
{
    if(x[left]<=x[mid])
    {
        temp[tmpos]=x[left];
        tmpos++;
        left++;
    }
    else
    {
        temp[tmpos]=x[mid];
        tmpos++;
        mid++;
    }
}

while(left<=lend)
{
    temp[tmpos]=x[left];
    left++;
    tmpos++;
}

while(mid<=right)
{
    temp[tmpos]=x[mid];
    tmpos++;
    mid++;
}

```

```
void interchange(int f,int s)
```

```
{
```

```
    int temp;
```

```
    temp=a[f];
```

```
    a[f]=a[s];
```

```
    a[s]=temp;
```

```
}
```

Chapter 8: Searching

8.1 Introduction

Searching is a process of finding an element within the list of elements stored in any order. Searching is divided into two categories. They are:

- Linear (sequential) searching
- Binary searching

Linear searching is the basic and simple method of searching. Binary searching takes some less time to search an element from the sorted list of element. So we can say that binary searching method is more efficient than linear searching. Only drawback of binary searching is that it works on the sorted list whereas there is no prerequisite for linear searching.

8.2 Linear or Sequential Searching

In linear search, we access each element of an array one by one sequentially and see whether it is desired element or not. A search will be unsuccessful if all the elements are accessed and the desired element is not found. Therefore, linear search can be defined as the technique which traverses the array sequentially to locate the given item.

Algorithm

Let 'a' be the linear array with 'n' elements, and 'item' is an element to be searched. The algorithm finds the location 'loc' of item in 'a' or give the failure message.

1. Read the item to be searched.
2. [Initialize counter] set loc = -1, j = 0.
3. [Search for item]

Repeat while j < n

 If a[j] = item


```

        Set loc = j
        Break
    Else
        Set j = j+1
    Wend
4. [Successful] if loc >= 0
    Print the searched value's (item's) position (loc).
5. [Unsuccessful] else
    Print the searching item is not found.
6. END

```

Efficiency

The number of comparisons of keys done in sequential search of a list of length n is

- Unsuccessful search: n comparisons
- Successful search, best case: 1 comparison
- Successful search, worst case: n comparisons
- Successful search, average case: $(n + 1)/2$ comparisons
- In any case, the number of comparison is $O(n)$

8.3 Binary Searching

Binary search is an extremely efficient algorithm. This search technique searches the given item in minimum possible comparisons. To do the binary search, first we have to sort the array elements. The logic behind this technique is given below.

- First find the middle element of the array.
- Compare the middle element with an item.
- There are three cases.
 - If it is a desired element then search is successful.
 - If it is less than the desired item then search only in the first half of the array.

c) If it is greater than the desired item, search in the second half of the array.

4. Repeat the same steps until an element is found or search area is exhausted.

In this way, at each step we reduce the length of the list to be searched by half.

Requirements

- The list must be ordered.
- Rapid random access is required, so we cannot use binary search for a linked list.

Algorithm

Let 'a' be the array of size 'Maxsize' and 'LB'. 'UB' and 'mid' are variables to denote first, last and middle location of a segment. This algorithm finds the location 'Loc' of 'item' in array 'a' or return fail (sets loc=NULL).

- [Initialize segment variables]


```

set beg = LB, end = UB and mid = int((beg + end)/2)

```
- Repeat steps 3 and 4 while $beg \leq end$ and $a[mid] \neq item$
- if $item < a[mid]$, then


```

                set end = mid - 1
            
```
- Else


```

                set beg = mid + 1
            
```
- [End if]
- set $mid = int((beg + end)/2)$
- if $a[mid] = item$ then


```

                set loc = mid
                print the value and its position
            
```
- else

set loc=NULL
print search unsuccessful

[End if]
6. END

Efficiency

- In all cases, the number of comparisons is proportional to n .
- Hence, number of comparisons in binary search is $O(\log n)$, where n is the number of items in the list.
- Obviously binary search is faster than sequential search, but there is an extra overhead in maintaining the list ordered.
- For small lists, better to use sequential search.
- Binary search is best suited for lists that are constructed and sorted once, and then repeatedly searched.

8.4 Hashing

"Derive a number from the key information given to you and use that number to access all information related to the key". This is the basic principle of hashing. This way, we can access any record in a specific time without making any comparison, irrespective of the number of record in the file.

Hashing is a scheme of searching that will use some function say hash to directly find out the location of the record in a constant search time, no matter where the record is in the file. In other words, the process of mapping large amount of data into a smaller table is called hashing.

This searching scheme is easy to program compared to trees. But it is difficult to expand, since it is based on arrays.

Hash Table

A hash table is simply an array that is address via a hash function. It is a data structure made up of

- A table of some fixed size to hold a collection of records each uniquely identified by some key.

- A function called hash function that is used to generate index values in the table.

Hash Function

The basic idea in hashing is the transformation of a key into the corresponding location in the hash table. This is done by a hash function. A hash function can be defined as a function that takes key as input and transforms it into a hash table index usually denoted by H .

Some popular hash functions are:

- Folding
- Mid square method
- Division method

Hash of Key

Let H be a hash function and k is a key then $H(k)$ is called hash-of-key. The hash-of-key is the index at which a record with the key values k must be kept.

Store and Retrieve Operation using a Hash Function

The result of a hash function tells us where to look for a particular element in order to retrieve, modify or delete the element. We could use the following algorithm for retrieving a record.

retrieve(key)

```
{  
    int location;  
    location = hash(key); //The result of hash function gives the location  
    record = info[location]; //Get the record from the info array  
}
```

We must put each new record into the correct slot according to the hash function when you insert a record. The algorithm for inserting a record follows:


```
insert(key)
```

```
int location;
```

```
location = hash(key); //hash function gives the location loc.
```

```
info[location] = record; //insert record into info array at array  
//position loc.
```

8.4.1 Popular Hashing Functions

We have, in general the following methods:

- Folding method
- Mid-square method
- Division remainder method

The two principle criteria in selection of a good hash function are:

1. It must be very easy and quick to compute hash value.
2. It must minimize collision.

1. Folding Method

In this method, we chop the key into two parts and then add them.

Let the key be 5421 (key of four digits).

$$\text{Hash}(5421) = 54 + 21 = 75$$

So, 75 is the index at which we should store or retrieve record with key 5421.

2. Mid-Square Method

In this method, the key is squared. We define the hash function in this case as

$$\text{Hash}(\text{key}) = p;$$

Where p is obtained by deleting digits from both ends of $(\text{key})^2$. We emphasize that the same positions of $(\text{key})^2$ must be used for all the keys.

The following calculations are performed:

key	= 5421	1825
$(\text{key})^2$	= 29387241	3330625
Leaving 3 digits from the last and then taking two preceding digits.		
Hash(key)	= 87	30

3. Division Remainder Method

Convert the key to an integer, divide by the size of the index range and take the remainder as the result.

Example, $\text{hash}(\text{key}) = \text{hash}(1029)$

Let 100 be the table size (index range)

$$\text{Then, } \text{hash}(1029) = 1029 \% 100 = 29$$

29 is the hash value (i.e., index of the array).

To get a good distribution of indices, prime number makes the best table size.

8.5 Collision Resolution Techniques

There are a finite number of indices in a table. But there are large numbers of keys so it is clearly impossible to get two different indexes for two distinct keys. A situation in which two different keys k_1 and k_2 hash to the same index of the table i.e., $h(k_1) = h(k_2)$, it is called **collision** or **hash clash**.

When a collision occurs, alternative locations in the table are tried until an empty location is found. Looking for the next available position is called probing.

The different types of collision resolution techniques are as follows:

- Linear probing
- Chaining
- Quadratic probing
- Double hashing
- Rehashing

8.5.1 Linear Probing

A simple approach to resolve collision is to store the colliding element in the next available space. This technique is known as linear probing.

Key = 32 to be added
 $h(32) = 6$

Must probe two more times

0	1	2	3	4	5	6	7	8	9	10	11	12
				4	1	4	5	32				

Example: Insert the keys 20, 15, 89, 18, 49, 48, 79, 21 using linear probing. Consider the hash table of size 10.

Solution:

The hash function is: $h(\text{key}) = \text{key} \% 10$ (where 10 is the table size)

Let's insert following keys in the hash table: 20, 15, 89, 18, 49, 48, 79, 21.

9		9		9	89	9	89
8		8		8		8	18
7		7		7		7	
6		6		6		6	
5		5	15	5	15	5	15
4		4		4		4	
3		3		3		3	
2		2		2		2	
1		1		1		1	
0	20	0	20	0	20	0	20
	$20\%10=0$		$15\%10=5$		$89\%10=9$		$18\%10=8$

9	89 (collision)	9	89 (find next)	9	89 (collision)	9	89
8	18	8	18 (collision)	8	18	8	18
7		7		7		7	
6		6		6		6	
5	15	5	15	5	15	5	15
4		4		4		4	21
3		3		3	79	3	79 (find next)
2		2	48	2	48 (find next)	2	48 (find next)
1	49	1	49 (find next)	1	49 (find next)	1	49 (collision)
0	20 (find next)	0	20 (find next)	0	20 (find next)	0	20
	$49\%10=9$		$48\%10=8$		$79\%10=9$		$21\%10=1$

Disadvantages of linear probing:

One main disadvantage of linear probing is that, records tend to cluster, that is, the records appear next to one another when the table is about half full.

Clustering occurs when a hash function is biased towards the placement of keys into a given region within the storage space. When the linear method is used to resolve collisions, this clustering problem is compounded, because keys that collide are loaded relatively close to the initial collision point.

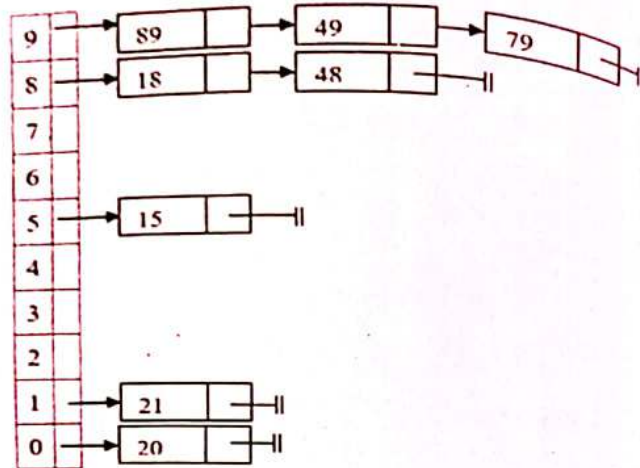
8.5.2 Chaining

It is another technique to deal with collision. In this method, for each location in the table, we keep a linked list of records that hash to the same index. The hash table contains pointers to linked list nodes; we can view these as the head pointers. Each time a record is inserted, it is added to the list at the location given by the hash function. When a collision occurs, the record is simply added to the list at the collision site.

Example: Insert the keys 20, 15, 89, 18, 49, 48, 79, 21 using chaining method. Consider the hash table of size 10.

Solution:

The hash function is: $h(\text{key}) = \text{key} \% 10$ (where 10 is the table size)
For the above keys and hash function, chaining yields;



8.5.3 Quadratic Probing

This method makes an attempt to correct the problem of clustering with linear probing. It forces the problem key to move quickly a considerable distance from the initial collision. When a key is hashed and if collision occurs for key, this method probes the table location at

$(h(k) + 1^2) \% \text{table-size}$,

$(h(k) + 2^2) \% \text{table-size}$,

$(h(k) + 3^2) \% \text{table-size}$, and so on.

That is, the first rehash adds 1 to the hash value. The second rehash adds 4, the third adds 9 and so on.

It reduces primary clustering but suffers from secondary clustering: keys that hash to some initial slot will probe the same alternative cells.

There is no guarantee to finding an empty location once the table gets more than half full.

Example: Insert the keys 20, 15, 89, 18, 49, 48, 79, 21 using quadratic probing method. Consider the hash table of size 10.

Solution:

The hash function is: $h(\text{key}) = \text{key} \% 10$ (where 10 is the table size)
For the above keys and hash function, quadratic probing yields

9	9	9 89	9 89
8	8	8	8 18
7	7	7	7
6	6	6	6
5	5 15	5 15	5 15
4	4	4	4
3	3	3	3
2	2	2	2
1	1	1	1
0 20	0 20	0 20	0 20
20%10=0	15%10=5	89%10=9	18%10=8

9	$89 ((49+1) \% 10)$	9	$89 ((48+4) \% 10)$	9	$89 ((79-1) \% 10)$	9	89
8	18	8	$18 ((48+1) \% 10)$	8	$18 ((79-16) \% 10)$	8	18
7		7		7		7	
6		6		6		6	
5	15	5	15	5	$15 ((79-25) \% 10)$	5	15
4		4		4	79	4	79
3	49	3	49	3	$49 ((79-9) \% 10)$	3	49
2		2	48	2	48	2	48
1		1		1		1	21
0	$20 ((49+4) \% 10)$	0	20	0	$20 ((79-4) \% 10)$	0	20
	$49 \% 10 = 9$		$48 \% 10 = 8$		$79 \% 10 = 9$		$21 \% 10 = 1$

8.5.4 Double Hashing

In this method, when a collision occurs, a new hash function is defined.

$$H_2(\text{key}) = R - (\text{key} \% R)$$

Where R is the nearest prime number from its table size.

8.5.5 Rehashing

If at any stage the hash table become almost full (when packing density is more than 70%), then it will be difficult to find the free slot which will increase execution time. If the hash function produces collisions, we create a new hash table of double size. We may use the old hash value on input to a rehash function and compute a new hash value. For rehashing with linear probing, we can use the rehash function as:

$$(\text{old hash value} + \text{constant}) \% \text{array-size}$$

Where constant and array size are relatively prime, i.e., the largest number that divides both of them is 1. For eg, given 100 slots array, we may use constant in rehash function:

$$(\text{old hash value} + 3) \% 100.$$

Questions:

1. Insert following keys: 30, 25, 79, 19, 48, 28, 21 and 44 in hash table using linear probing method where $h(\text{key}) = \text{key} \% 10$. [2076 Bhadra]

Answer:

To find the proper position for above keys using given hash function,

$30 \% 10 = 0$. So, insert 30 in 0 location.

$25 \% 10 = 5$. So, insert 25 in 5 location.

$79 \% 10 = 9$. So, insert 79 in 9 location.

$19 \% 10 = 9$ (collision). So, insert 19 in 1 location.

$48 \% 10 = 8$. So, insert 48 in 8 location.

$28 \% 10 = 8$ (collision). So, insert 28 in 2 location.

$21 \% 10 = 1$ (collision). So, insert 21 in 3 location.

$44 \% 10 = 4$. So, insert 44 in 4 location.

9	79
8	48
7	
6	
5	25
4	44
3	21
2	28
1	19
0	30

Hence, we insert the given keys in an array of size 10.

2. Draw hash table obtained from double hashing with hash functions: $h_1(K) = K \bmod 11$ and $h_2(K) = K \bmod 9$ for the given keys: 76, 36, 47, 49, 21, 65 with table size 11. Use $h_p(K, i) = (h_1(K) + i \times h_2(K)) \bmod m$. [2075 Bhadra]

Answer:

Here, $h_p(K, i) = (h_1(K) + i \times h_2(K)) \bmod m$ [where $m = 11$]

Initially, we take $i = 0$ to find the proper position for above keys. If collision occurs, we have to take $i = 1, 2, 3$ and so on.

$h_p(K, 0) = h_1(K) = h_1(76) = 76 \bmod 11 = 10$. So, insert 76 in 10 location.

Similarly, $h_1(36) = 36 \bmod 11 = 3$. So, insert 36 in 3 location.

$h_1(47) = 47 \bmod 11 = 3$ (collision).

now, $h_2(47) = 47 \bmod 9 = 2$.

$h(47, 1) = (3 + 1 \times 2) \bmod 11 = 5$. So, insert 47 in location 5. [take $i = 1$ for first collision]

$h_1(49) = 49 \bmod 11 = 5$ (collision).

now, $h_2(49) = 49 \bmod 9 = 4$.

$h(49, 1) = (5 + 1 \times 4) \bmod 11 = 9$. So, insert 49 in location 9. [take $i = 1$ for first collision]
 $h_1(21) = 21 \bmod 11 = 10$ (collision).
 now, $h_2(21) = 21 \bmod 9 = 3$.
 $h(21, 1) = (10 + 1 \times 3) \bmod 11 = 2$. So, insert 21 in location 2. [take $i = 1$ for first collision]
 $h_1(65) = 65 \bmod 11 = 10$ (collision).
 now, $h_2(65) = 65 \bmod 9 = 2$.
 $h(65, 1) = (10 + 1 \times 2) \bmod 11 = 1$. So, insert 65 in location 1. [take $i = 1$ for first collision]

10	76
9	49
8	
7	
6	
5	47
4	
3	36
2	21
1	65
0	

Hence, we insert the given keys in an array of size 11.

Program 1: Implementation of Linear Search.

```

#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<string.h>
    
```

```

#define n 10
void linear(int a[]);
void main()
{
    int a[n], i;
    printf("\n***LINEAR SEARCHING***\n\n");
    printf("\nEnter the 10 numbers\n");
    for(i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }

    linear(a);
}

void linear(int a[])
{
    int i, loc=-1, ch;
    printf("\n\nEnter the number to search\n");
    scanf("%d", &ch);
    for(i=0; i<n; i++)
    {
        if(a[i]==ch)
        {
            loc=i+1;
            break;
        }
    }

    if(loc>=0)
    
```

```

    {
        printf("\n%d was found in %d position",ch,loc);
    }
else
    {
        printf("\n%d was not found",ch);
    }
    getch();
}

```

Program 2: Implementation of Binary Search.

```

#include<stdio.h>
#include<conio.h>
#include<ctype.h>
#include<string.h>
#include<process.h>
#include<stdlib.h>
#define n 10
void insertionsort(int a[],int y);
void binary(int a[],int ch,int left,int mid,int right);
void main()
{
    int a[n],i,ch;
    printf("\n***BINARY SEARCHING***\n\n");
    printf("\nEnter the 10 numbers\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
}

```

```

    }
    insertionsort(a,n);
    printf("\nEnter the number to search\n");
    scanf("%d",&ch);
    binary(a,ch,0,n/2,n);
    printf("\n%d was not found",ch);
    getch();
}

```

```

void insertionsort(int a[],int y)
{
    int j,i,hold;
    for(i=0;i<y;i++)
    {
        hold=a[i];
        for(j=i-1;j>=0&& a[j]>hold;j--)
        {
            a[j+1]=a[j];
            a[j]=hold;
        }
    }
}

void binary(int a[],int ch,int left,int mid,int right)
{
    int loc;
    if((mid-1)>=left&&((mid-1)<right))
    {
        if(ch==a[mid-1])

```



```

    {
        loc=mid;
        printf("\n%d was found in %d position",ch,loc);
        getch();
        exit(1);
    }
    if(ch<a[mid-1])
        binary(a,ch,0,mid/2,mid);
    if(ch>a[mid-1])
        binary(a,ch,mid,(n+mid+1)/2,n);
}
}

```

Chapter 9: Growth Functions

9.1 Introduction

The function that derives the running time of an algorithm and its memory space requirements for a given set of input is referred as algorithm complexity.

9.2 Asymptotic Notation

Asymptotic notation is the most simple and easiest way of describing the running time of an algorithm. Asymptotic notation describes the time complexity in terms of three common measures: Best case, worst case, and average case.

The three most common asymptotic notations are:

- Big-Oh Notation
- Omega Notation
- Theta Notation

9.3 Algorithm Performance

The performance of an algorithm is obtained by totaling the number of occurrences of each operation when running the algorithm.

The performance of algorithm is evaluated as a function of the input size 'n' and is said to be considered as modulo a multiplicative constant.

The following notations are commonly used in performance analysis and used to characterize the complexity of an algorithm:

Theta Notation (θ -notation) – Tightly Bound

Big-Oh Notation (O-notation) – Upper Bound

Omega Notation (Ω -notation) – Lower Bound

1. Theta Notation (Θ -Notation)

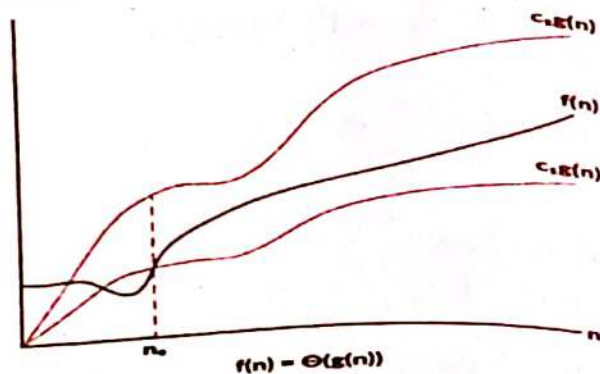


Figure 9.1: Theta notation

Theta notation bounds a function within constant factors.

Mathematically, $f(n) = \Theta(g(n))$ if there exists positive constants n_0 , C_1 and C_2 such that to the right of n_0 , the value of $f(n)$ always lies between $C_1g(n)$ and $C_2g(n)$.

2. Big-Oh Notation (O -Notation)

Big-oh is the formal method of expressing the upper bound of an algorithm's running time.

Mathematically, $f(n) = O(g(n))$ if there exists positive constants n_0 and C such that to the right of n_0 , the value of $f(n)$ always lies on or below $Cg(n)$.

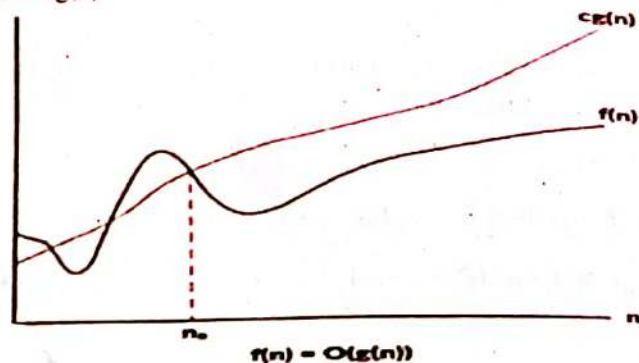


Figure 9.2: Big-oh notation

Examples

1. Given that $f(n) = 10n+5$ and $g(n) = n$. Prove that $f(n)$ is $O(g(n))$.

Solution:

Here, $f(n) = O(g(n)) = O(n)$

To show $f(n)$ is $O(g(n))$, we must show constants C and K such that $f(n) \leq Cg(n)$ for all $n \geq K$.

or, $10n+5 \leq Cn$ for all $n \geq K$.

We are allowed to choose C and K to be integer, we want as long as they are positive. They can be as big as we want but they cannot be the function of n .

Let $C=15$, then we need to show, $10n+5 \leq 15n$

Solving for n ,

$10n+5 \leq 15n$

or, $5 \leq 5n$

or, $1 \leq n$

So, $f(n) = 10n+5 \leq 15g(n)$ for all $n \geq 1$.

Hence, we have shown $f(n)$ is $O(g(n))$.

2. Given that $f(n) = 3n^2 + 4n + 1$. Show $f(n)$ is $O(n^2)$.

Solution:

We know, $4n \leq 4n^2$ for all $n \geq 1$.

and $1 \leq n^2$ for all $n \geq 1$.

$3n^2 + 4n + 1 \leq 3n^2 + 4n^2 + n^2$ for all $n \geq 1$.

$\leq 8n^2$ for all $n \geq 1$.

Hence, $f(n) \leq 8n^2$ for all $n \geq 1$.

Therefore, $f(n)$ is $O(n^2)$. [where $C = 8, K = 1$]

3. Omega Notation (Ω -Notation)

This notation gives the lower bound for a function to within constant factor.

Mathematically, $f(n) = \Omega(g(n))$ if there exists positive constants n_0 and C such that to the right of n_0 the value of $f(n)$ always lie on or above $Cg(n)$.

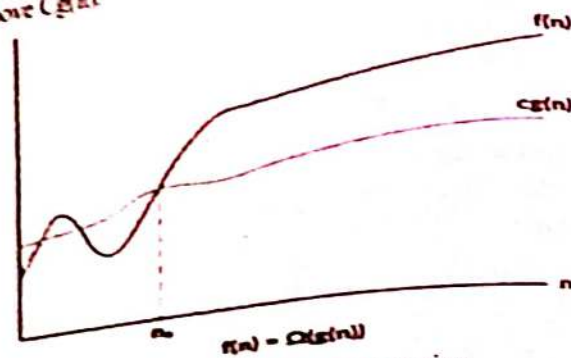


Figure 9.3: Omega notation

Chapter 10: Graphs

10.1 Introduction

A graph is a data structure that describes a binary relation between elements and has a webby looking graphical representation. Graph plays a significant role in solving a rich class of problems.

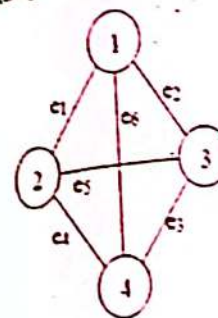
A graph $G = (V, E)$ consists of a finite set of non-empty set of vertices V and set of edges E .

$$V = \{v_1, v_2, \dots, v_n\}$$

$$E = \{e_1, e_2, \dots, e_m\}$$

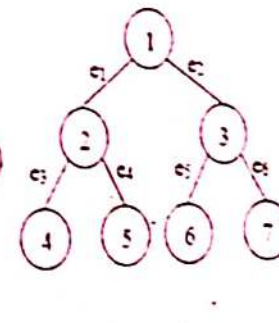
Each edge e is a pair (v, w) where $v, w \in V$. The edge is also called vw .

Example of graphs:



G_1

$V(G_1) = \{1, 2, 3, 4\}$
 $E(G_1) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$
 $e_1 = (1, 2)$
 $e_2 = (1, 3)$
 $e_3 = (1, 4)$
 $e_4 = (2, 3)$
 $e_5 = (2, 4)$
 $e_6 = (3, 4)$



G_2

$V(G_2) = \{1, 2, 3, 4, 5, 6, 7\}$
 $E(G_2) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$
 $e_1 = (1, 2)$
 $e_2 = (1, 3)$
 $e_3 = (2, 4)$
 $e_4 = (2, 5)$
 $e_5 = (3, 6)$
 $e_6 = (3, 7)$



G_3

$V(G_3) = \{1, 2, 3\}$
 $E(G_3) = \{e_1, e_2, e_3\}$
 $e_1 = (1, 2)$
 $e_2 = (2, 3)$
 $e_3 = (1, 3)$

3. Omega Notation (Ω -Notation)

This notation gives the lower bound for a function to within constant factor.

Mathematically, $f(n) = \Omega(g(n))$ if there exists positive constants n_0 and C such that to the right of n_0 the value of $f(n)$ always lie on or above $Cg(n)$.

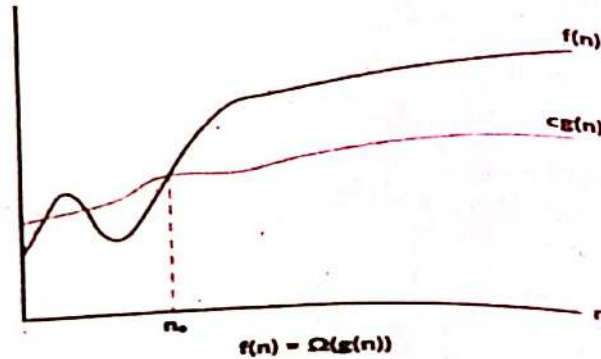


Figure 9.3: Omega notation

Chapter 10: Graphs

10.1 Introduction

A graph is a data structure that describes a binary relation between elements and has a webby looking graphical representation. Graph plays a significant role in solving a rich class of problems.

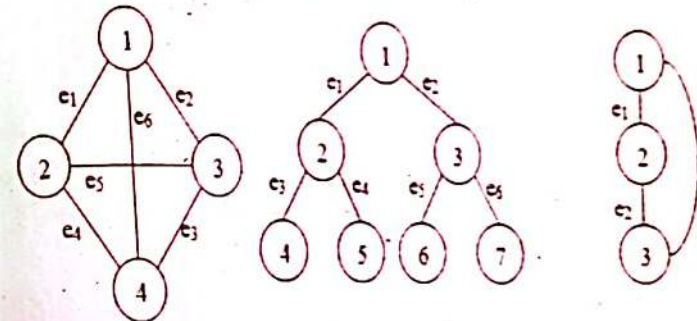
A graph $G = (V, E)$ consists of a finite set of non-empty set of vertices V and set of edges E .

$$V = \{v_1, v_2, \dots, v_n\}$$

$$E = \{e_1, e_2, \dots, e_n\}$$

Each edge e is a pair (v, w) where $v, w \in V$. The edge is also called arc.

Example of graphs:



G_1

$V(G_1) = \{1, 2, 3, 4\}$
 $E(G_1) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$
 $e_1 = (1, 2)$
 $e_2 = (1, 3)$
 $e_3 = (3, 4)$
 $e_4 = (2, 4)$
 $e_5 = (2, 3)$
 $e_6 = (1, 4)$

G_2

$V(G_2) = \{1, 2, 3, 4, 5, 6, 7\}$
 $E(G_2) = \{e_1, e_2, e_3, e_4, e_5, e_6\}$
 $e_1 = (1, 2)$
 $e_2 = (1, 3)$
 $e_3 = (2, 4)$
 $e_4 = (2, 5)$
 $e_5 = (3, 6)$
 $e_6 = (3, 7)$

G_3

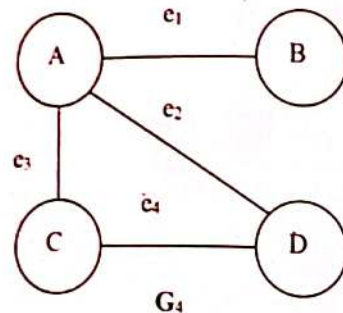
$V(G_3) = \{1, 2, 3\}$
 $E(G_3) = \{e_1, e_2, e_3\}$
 $e_1 = (1, 2)$
 $e_2 = (2, 3)$
 $e_3 = (1, 3)$

The vertices are represented by points or circles and the edges are line segments connecting the vertices. If the graph is directed, then the line segments have arrow heads indicating the direction.

10.2 Additional Terminologies in Graph

1. Adjacent and Incident

Two vertices i and j are called adjacent if there is an edge between the two. In the figure below (G_4), the vertices adjacent to vertex A are B , C , and D . The adjacent vertices of C are A and D .



If $e(i, j)$ is an edge on $E(G)$, then we say that the edge $e(i, j)$ is incident on vertices i and j . Eg. in the above figure, e_4 is incident on C and D . If (i, j) is directed edge, then i is adjacent to j and j is adjacent from i .

2. Path

A path is a sequence of distinct vertices, each adjacent to the next. For e.g. the sequence of vertices e_1, e_3, e_4 (i.e. (B, A) , (A, C) , (C, D)) of the above graph form a path from B to D . The length of the path is the number of edges in the path. So, the path from B to D has length equal to three.

In a weighted graph, the cost of a path is the sum of the costs of its edges. Loops have path length of 1.

3. Cycle

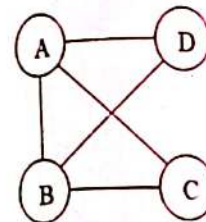
A cycle is a path containing at least three vertices such that the last vertex on the path is adjacent to the first. In the above graph G_4 , A, C, D, A is a cycle.

4. Connected

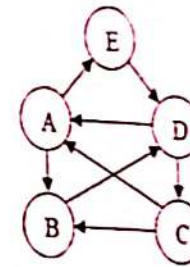
Any graph is connected provided there exists a path (directed or undirected) between any two nodes. A digraph is said to be strongly connected if, for any two vertices there is a direct path from i to j .

A digraph is said to be weakly connected if, for any two vertices i and j , there is a direct path from i to j or j to i . or A weakly-connected graph is a directed graph for which its underlying undirected graph is connected.

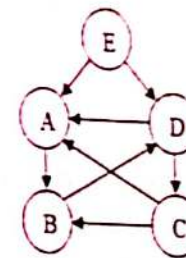
Example:



Connected



Strongly Connected



Weakly Connected

5. Complete graph

A complete graph is a graph in which there is an edge between every pair of vertices.

6. Trees and Graph

A tree is a connected graph with no cycle. A tree has $|E| = |V| - 1$ edges. Since, it is connected, there is a path between any two vertices.

10.3 Directed Graphs

If every edge (i, j) , in $E(G)$ of a graph G is marked by a direction from i to j , then the graph is called directed graph. It is often called diagraph.

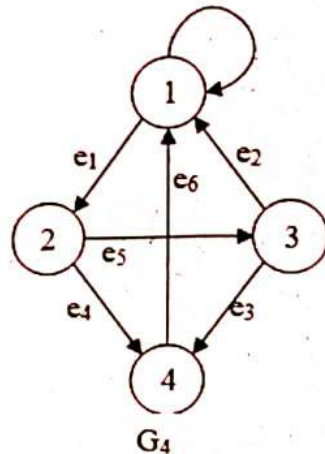


Figure 10.1: Directed graph

In edge $e = (i, j)$, we say, e leaves i and enters j . In diagraphs, self loops are allowed. The indegree of a vertex v is the number of edge entering v . The outdegree of a vertex v is the number of edges leaving v . The sum of the indegrees of all the vertices in a graph equals to the sum of outdegree of all the vertices.

$$\sum_{i=0}^n d_{in}(v_i) = \sum_{i=0}^n d_{out}(v_i) \quad n = \text{no. of vertices}$$

10.4 Undirected Graph

If the directions are not marked for any edge, the graph is called undirected graph. The graphs G_1, G_2, G_3 are undirected graphs. In an undirected graph, we say that an edge $e = (u, v)$ is incident on u and v (u and v are connected).

Undirected graph don't have self loops. Incidence is a symmetric relation i.e. if $e = (u, v)$

Then u is a neighbor of v and vice-versa. The degree of a vertex, $d(v)$, is the total number of edges incident on it.

$$\sum_{i=0}^n d(v_i) = 2|E|$$

The sum of the degrees of all the vertices in a graph equals twice the number of edges.

10.5 Weighted Graphs

A graph is said to be weighted graph if every edge in the graph is assigned some weight or value. The weight is a positive value that may represent the cost of moving along the edge, distance between the vertices etc.

The two vertices with no edge (path) between them can be thought of having an edge (path) with weight infinite.

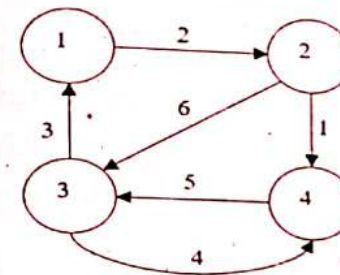


Figure 10.2: Weighted graph

10.6 Representation of Graph

A graph can be represented in many ways. Some of them are described in this section.

1. Adjacency Matrix

Adjacency matrix A for a graph $G = (V, E)$ with n vertices, is $n \times n$ matrix, such that

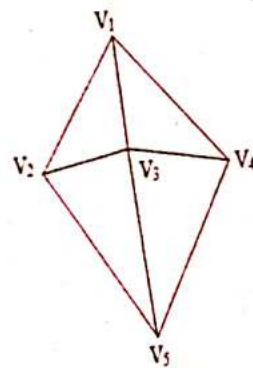
$A_{ij} = 1$, if there is an edge from v_i to v_j

$A_{ij} = 0$, if there is no such edge

We can also write,

$$A(i, j) = \begin{cases} 1 & \text{if and only if } (v_i, v_j) \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

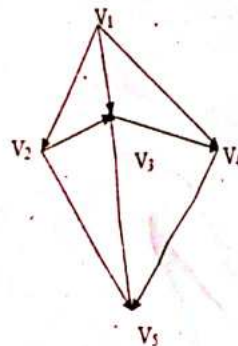
Example: An adjacency matrix for the following undirected graph is



$A =$

	V_1	V_2	V_3	V_4	V_5
V_1	0	1	1	1	0
V_2	1	0	1	0	0
V_3	1	1	0	1	1
V_4	1	0	1	0	1
V_5	0	1	1	1	0

If the graph is directed, then the adjacency matrix will be as follows. The number in each row tells the outdegree of that vertex.



$A =$

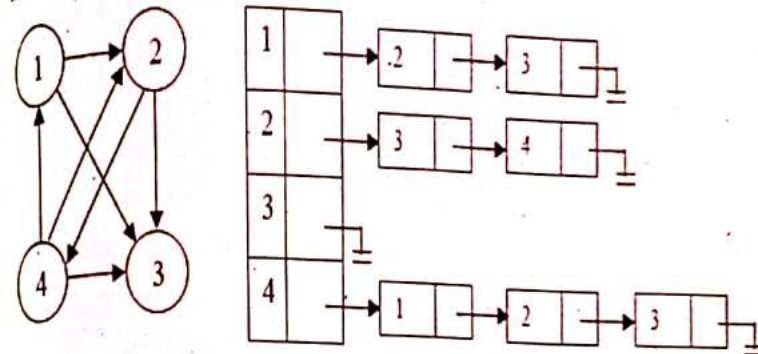
	V_1	V_2	V_3	V_4	V_5
V_1	0	1	1	1	0
V_2	0	0	1	0	0
V_3	0	0	0	1	0
V_4	0	0	0	0	1
V_5	0	0	1	0	0

In this representation we require n^2 bits to represent a graph with n nodes. It is a simple way to represent a graph, but it has following disadvantages:

- it takes $O(n^2)$ space.
- it takes $O(n^2)$ time to solve most of the problems.

2. Adjacency List Representation

The n rows of an adjacency matrix can be represented as n linked lists. This is one list for each node in a graph. Each list will contain adjacent nodes. Each node has two fields, Vertex and Link. The vertex field of a node will contain the nodes that are adjacent to the node p .



In above figure, the index of the array represents a vertex and each element in its linked list represents the other vertices that form an edge with the vertex.

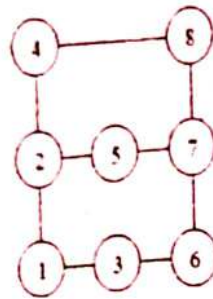
10.7 Graph Traversals

A graph traversal means visiting all the nodes of the graph. Basically, there are two methods of graph traversals.

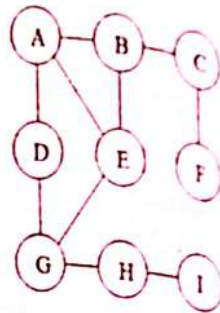
1. Breadth first traversal
2. Depth first traversal

10.7.1 Breadth First Traversal

The breadth first traversal begins with a given vertex and then it next visits all the vertices adjacent to v , putting the vertices adjacent to these in a waiting list to be traversed after all vertices adjacent to v have visited. Breadth first traversal uses queue.



Starting from 1 Breadth First Traversal
yields 1, 2, 3, 4, 5, 6, 8, 7



Starting from A Breadth First Traversal
yields A, B, D, E, C, G, F, H, I

Algorithm

void BFTTraversal(Graph G)

```

{
    for each vertex v in G
        visited[v] = false;
    for each vertex v in G
        if(visited[v]==false)
        {
            enqueue(V, Q);
            do
            {
                v=dequeue(q);
                visited[v]=true;
                visited(v);
                for each vertex w adjacent to v
                if(visited[w]==false)
                    enqueue(w, q);
            } while(isempty(Q)==false)
        }
}

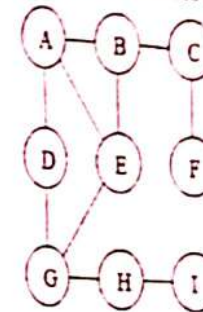
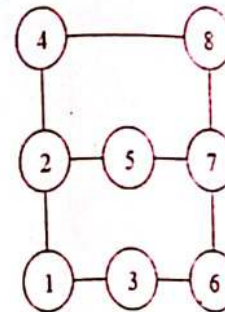
```

10.7.2 Depth First Traversal

The depth first traversal algorithm is roughly analogous to preorder tree traversal, the algorithm follows:

- Suppose that the traversal has just visited a vertex v , and let $w_0, w_1, \dots, w_k$ be the vertices adjacent to v
- Next, visit w_0 and keep $w_1, \dots, w_k$ waiting.
- After visiting w_0 , we traverse all the vertices to which it is adjacent before returning to traverse $w_1, \dots, w_k$.

Depth first traversal uses stack to store the adjacent vertices but one.



For above 2 figures, depth first traversal yields:

Adjacency list

Adjacency list

1	2, 3	A	B, D, E
2	1, 4, 5	B	A, C, E
3	1, 6	C	B, F
4	2, 8	D	A, G
5	2, 7	E	A, B, G
6	3, 7	F	C
7	5, 6, 8	G	D, E, H
8	4, 7	H	G, I
		I	H

DF Traversal:	DF Traversal:
1, 2, 4, 8, 7, 5, 6, 3	A, B, C, F, E, G, D, H, I

Algorithm

```
void DFTraversal(Graph G)
```

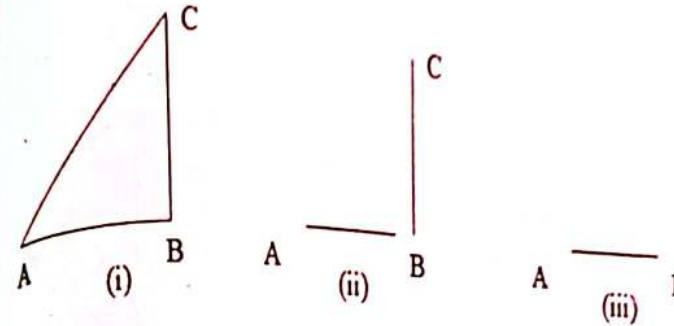
```
{
  for each vertex v in G
    Visited[v]=false;
  for each vertex v in G
    If(visited[v]==false)
      Traverse(v);
}
```

```
void Traverse(Vertex v)
```

```
{
  Visited[v]=true;
  Visited(v);
  For each vertex w adjacent to v
  {
    If(visited[w]==false)
      Traverse(w);
  }
}
```

10.8 Subgraph

Let H be a graph with vertex set $V(H)$ and edge set $E(H)$ and, similarly, let G be a graph with vertex set $V(G)$ and edge set $E(G)$. Then H is said to be a subgraph of G, written as $H \leq G$ if $V(H) \leq V(G)$ and $E(H) \leq E(G)$.

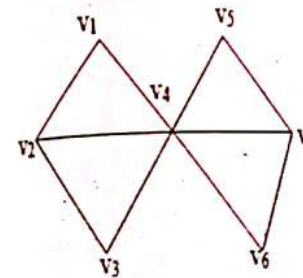


Here, second and third graph are subgraph of the first.

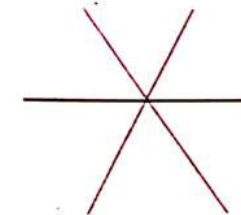
10.9 Spanning Tree

Let a graph $G=(V,E)$ be a graph. If T is a subgraph of G and contains all the vertices but no cycles/circuits, then 'T' is said to be spanning tree of G.

In other words, the spanning tree of an undirected graph G is the tree formed from graph edges which connects all the vertices of G.

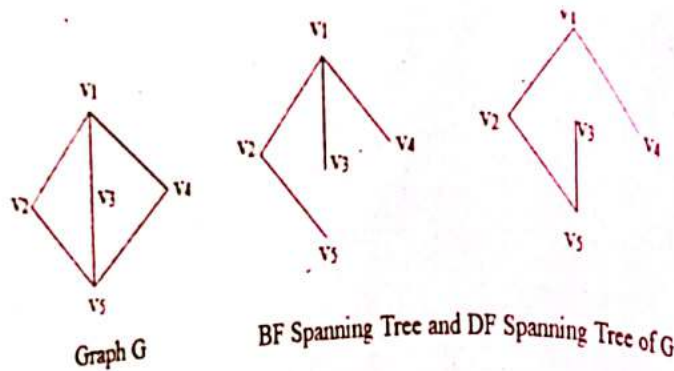


Undirected Graph G



Spanning Tree of G

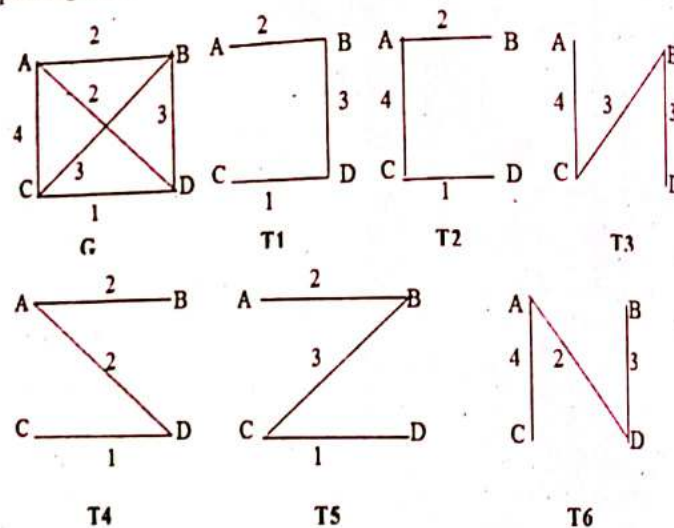
For a given graph $G=(V, E)$, if Breadth-First search and Depth-First search call is made, then the resultant tree that is generated is the spanning tree of the graph G. Thus, by making a Depth-First search on G, we get the Depth-First spanning tree. Similarly, on applying Breadth-First search on graph 'G' we get Breadth First spanning tree.



10.10 Minimum Spanning Tree

The cost of a spanning tree of a weighted, undirected graph is the sum of the costs (weights) of the edges in tree.

The minimum spanning tree of a weighted undirected graph is the spanning tree that connects all the vertices in G at lowest cost. So, if T is the minimum spanning tree of graph G and T' is any other spanning tree of G then, $W(T) \leq W(T')$.



Here, T_1 to T_6 are the spanning trees of G. Among them, T_4 has minimum cost (i.e., 5). So, T_4 is the minimum spanning tree of graph G.

There are several algorithms available to determine the minimal spanning tree of a given weighted graph.

10.10.1 Kruskal's Algorithm

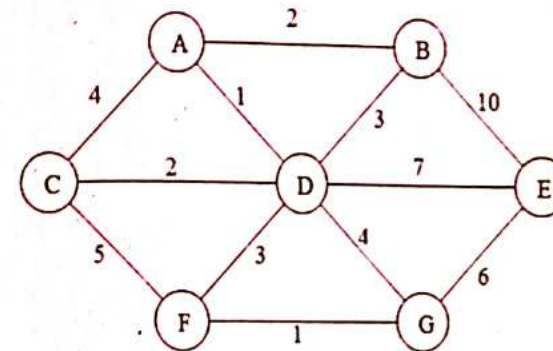
To determine minimum spanning tree, consider a graph G with n vertices.

Step 1: List all the edges of the graph G with increasing weights.

Step 2: Proceed sequentially to select one edge at a time joining n vertices of G such that no cycle is formed until n-1 edges are selected.

Step 3: Draw the n-1 edges that were selected forming a minimal spanning tree T of G.

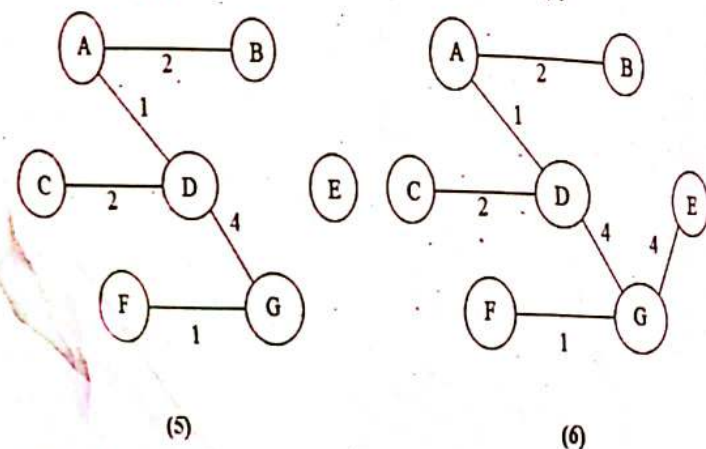
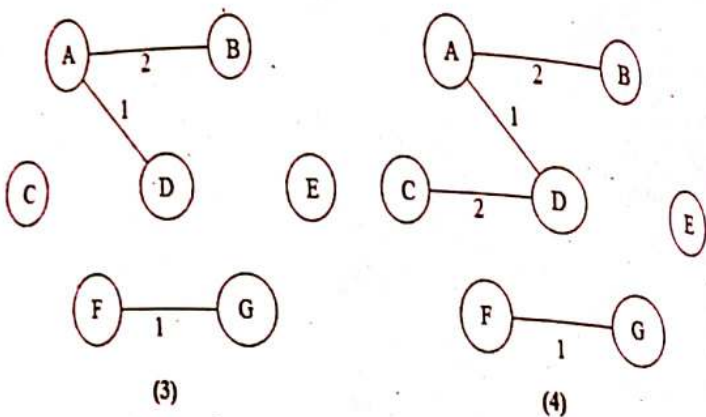
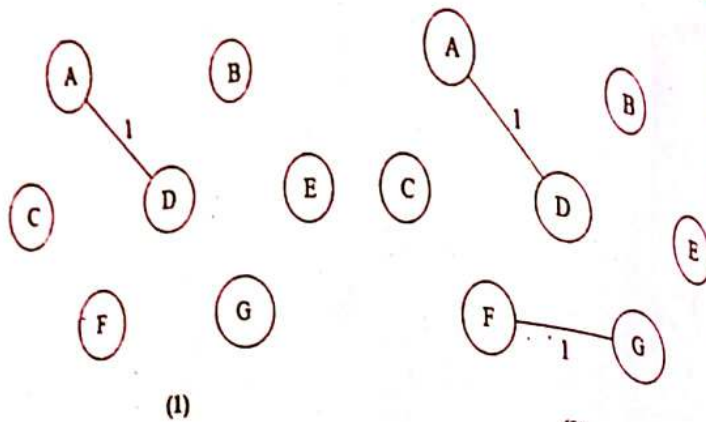
Example: Determine minimum spanning tree for the given graph using Kruskal's algorithm.



Solution:

We have following edges in the given graph:

Edges:	AD	FG	AB	CD	BD	AC	DG	CF	EG	DE	DF	BE
Weights	1	1	2	2	3	4	4	5	6	7	8	10

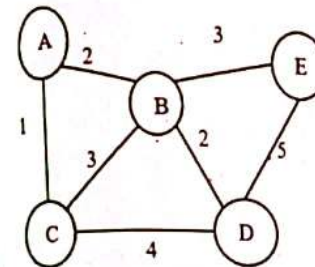


Hence, the minimum spanning tree is generated.

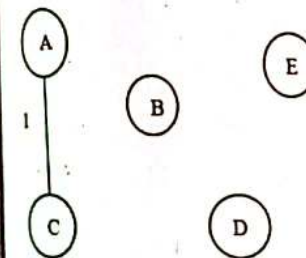
10.10.2 Round Robin Algorithm

This method provides better performance when the number of edges is low. Initially each node is considered to be a partial tree. Each partial tree is maintained in a queue Q. Priority queue is associated with each partial tree, which contains all the arcs ordered by their weights.

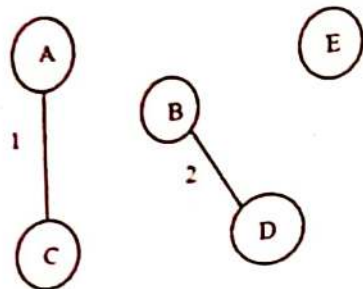
The algorithm proceeds by removing a partial tree, T1, from the front of Q, finding the minimum weight arc a in T1; deleting from Q, the tree T2, at the other end of arc a; combining T1 and T2 into a single new tree T3 and at the same time combining priority queues of T1 and T2 and adding T3 at the rear of priority queue. This continues until Q contains a single tree, the minimum spanning tree.



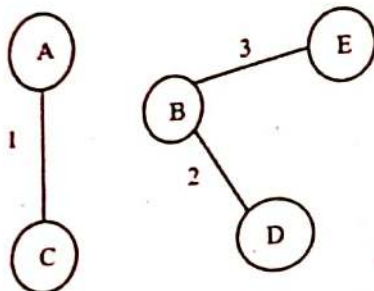
Q	Priority queue
{A}	1, 2
{B}	2, 2, 3, 3
{C}	1, 3, 4
{D}	2, 4, 5
{E}	3, 5



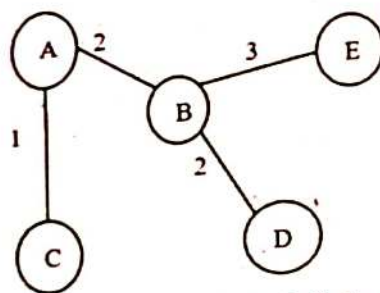
Q	Priority queue
{B}	2, 2, 3, 3
{D}	2, 4, 5
{E}	3, 5
{A, C}	2, 3, 4



Q	Priority queue
{E}	3, 5
{A, C}	2, 3, 4
{B, D}	2, 3, 3, 4, 5



Q	Priority queue
{A, C}	2, 3, 4
{E, B, D}	2, 3, 4, 5, 5



Q	Priority queue
{A, C, E, B, D}	2, 3, 4, 4, 5, 5

Only 1 partial tree is left in the queue, which is the required minimum spanning tree.

10.11 Shortest Path Algorithm

There are many problems that can be modeled using graph with weight assigned to their edges. Eg, we may set up the basic graph model by representing cities by vertices and flights by edges. Problems involving distances can be modeled by assigning distances between cities to the edges. Problems involving flight time can be modeled by assigning flight times to edges. Problem involving fares to the edges.

Thus, we can model airplane or other mass transit routes by graphs and use shortest path algorithm to compute the best route between two points.

Similarly, if the vertices represent computers; the edges represent a link between computers; and the costs represent communication costs, delay costs, then we can use the shortest-path algorithm to find the cheapest way to send electronic news, data from one computer to a set of other computers.

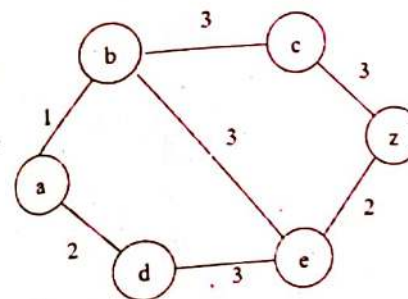
Basically, there are three types of shortest path problems:

Single path: Given two vertices, s and d , find the shortest path from s to d and its length (weights).

Single source: Given a vertex, s find the shortest path to all other vertices.

All pairs: Find the shortest path from all pair of vertices.

Let source vertex be a . We will find the shortest path to all the other vertices.



Vertex	Cost	Path
b	1	a, b
c	4	a, b, c
d	2	a, d
e	4	a, b, e
z	6	a, b, e, z

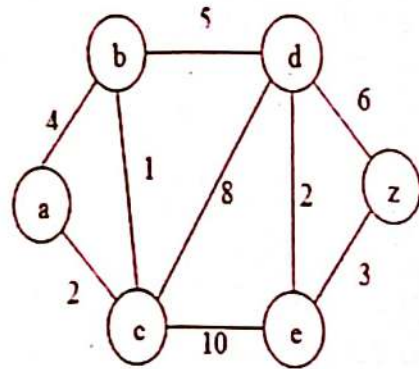
10.11.1 Dijkstra's Algorithm to find the Shortest Path

1. Mark all the vertices as unknown.
2. For each vertex v keep a distance d_v from source vertex s to v initially set to infinity except for s which is set to $d_s = 0$.

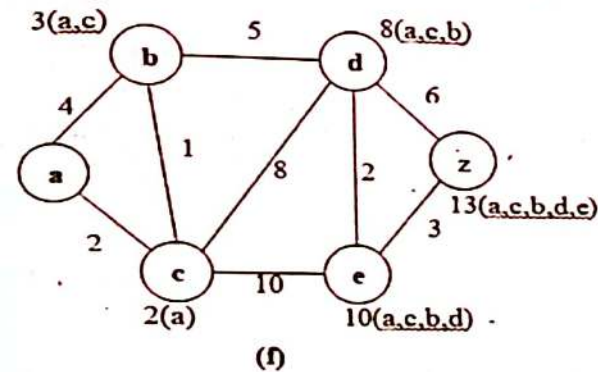
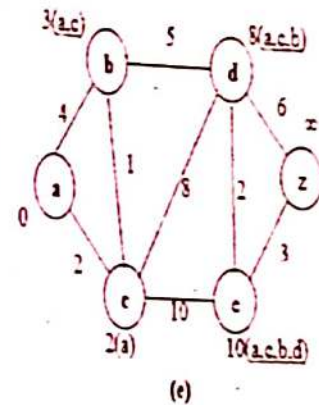
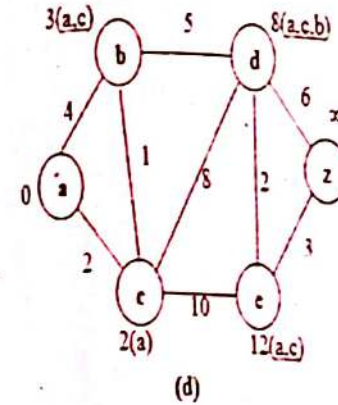
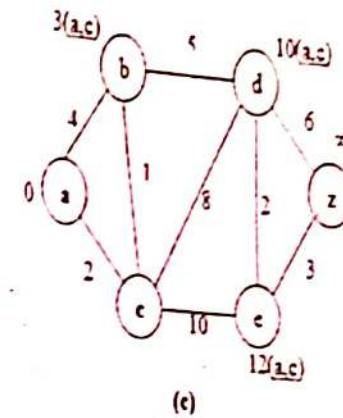
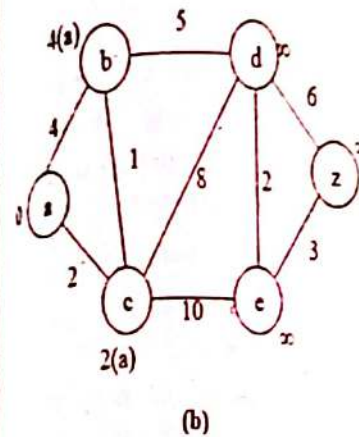
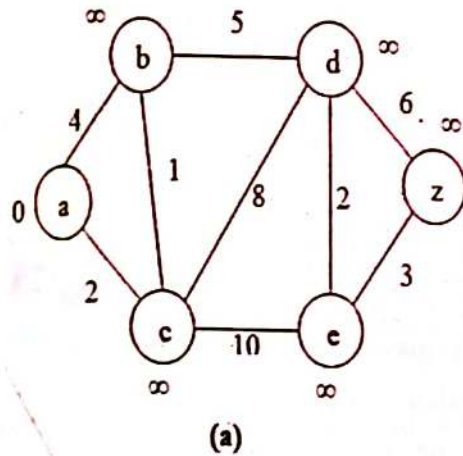
these steps until all vertices are known

- Select a vertex v , which has the smallest d_v among all the unknown vertices
- Mark v as known
- for each vertex w adjacent to v
If w is unknown and $d_v + \text{cost}(v, w) < d_w$,
update d_w to $d_v + \text{cost}(v, w)$

Example: Find the shortest path for the given graph with source node 'a' using Dijkstra's algorithm.



Solution:



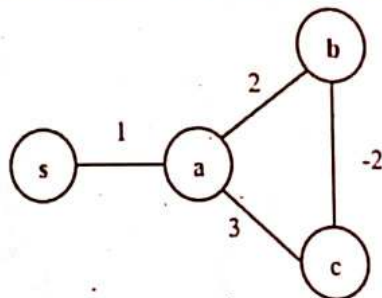
Hence, the shortest path from node 'a' to node 'z' is a-c-b-d-e-z.

10.11.2 Greedy Algorithm

One of the simplest approaches that often leads to a solution of an optimization problem. This approach selects the best choice at each step, instead of considering all sequences of steps that may lead to an optimal solution. Algorithm that make what seems to be the "best" choice at each step are called greedy algorithm.

Greedy algorithms tend to find the solution very fast. These algorithms choose the best option available for current state only (locally optimal). It lacks fore-sight seeing feature.

The search for minimum path between two nodes in shortest path algorithm is also a greedy algorithm. We just search for the edge for which the sum of weight and path up to that node is minimal. The shortest path also could not guarantee that the result found is the shortest path in case of negatively weighted graphs.



If the shortest path algorithm is applied in given graph (to go to node 'b' from node 's'), then the algorithm will choose edge with length '2' at node 'a' (s, a, b). And hence the minimum path will be of length '3'. But in this case, the path (s, a, c, b) has length '2' only. Therefore, the previous prediction was wrong.

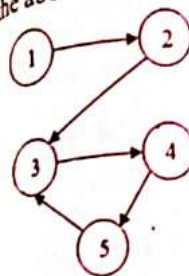
Greedy algorithms produce good solutions on some mathematical problems, but not on others.

10.12 Transitive Closure

In all shortest path, we noticed that every time we have to determine whether there is a path from vertex i to j. This can be accomplished by observing a matrix, say d^n . The path exists if and only if the entry

does not have the value infinity. In matrix d^n , we consider only path (not the weight). Thus, d^k can be regarded as Boolean matrix.

If there is a path from i to j, then $(i, j)^{th}$ entry in d^k is 1 (which means true) otherwise the value is 0. If 'A' is the adjacency matrix of graph 'G', then the transitive closure is defined to be a Boolean matrix with the above property.



Directed Graph

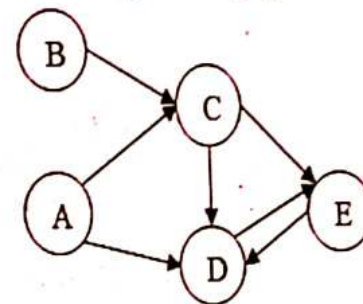
$$A^+ = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

A^+ Matrix has the following property:

- 1 if there is a path from i to j
- 0 otherwise

The matrix A^+ is called the transitive closure. The Transitive closure helps to determine which pair of vertices in a digraph are connected by a path. It takes $O(n^4)$ time complexity.

Let's consider the following directed graph.



Now, let adj2 is a matrix that stores information about the existence of path of length 2. The adj2 matrix (path matrix of length 2) can be represented as:

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	0
E	0	0	0	0	1

Similarly, adj3 can be represented as:

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	0	1
E	0	0	0	1	0

And adj4 can be represented as:

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	0
E	0	0	0	0	1

To calculate the path matrix, we just multiply the matrix itself. For example, to calculate path matrix 3, the matrix of adj1 is multiplied 3 times itself. While multiplying, the addition operation is replaced by '||' operation and multiplication is replaced by '&&' operation. To find at least one path between two nodes, we can check adj1, adj2, adj3, ..., adj_{maxnodes-1}. If any of the adjacency matrix give true value for node 'i' and 'j', then there exists at least a path between

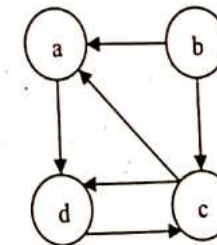
nodes 'i' and 'j'. To store the adjacency values for path, we use a path matrix as:

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	1
E	0	0	0	1	1

The length of path can be 1, 2, 3 or 4... and there is no termination of the order of the length. Let 'm' is the highest order of length and 'n' is the no. of total nodes. If the graph supports $m > n$, then at least one of the node is visited twice or more times in the path. It means, we can discard the cycling path from the node to the same node. The removal process is repeated until no node is repeated in the path. This ensures that the paths that exist in the graph be less than or equal to n. The final matrix path in which no path exists that has length greater than 'n' is known as transitive closure of the matrix adj.

10.13 Warshall's Algorithm

Transitive closure method of finding path between nodes is quite inefficient because it processes the adjacency matrix lots of times. An efficient method for calculating transitive closure is by Warshall's algorithm whose time complexity is only $O(n^3)$.



We first find the matrices d_0, d_1, d_2, d_3 , and d_4 . The matrix d_4 is the transitive closure.

$$d_0 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad d_1 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

d_1 has 1 as its $(i,j)^{th}$ entry if there is a path from v_i to v_j that has v_1 as an interior vertex.

$$d_2 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad d_3 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

d_2 has 1 as its $(i,j)^{th}$ entry if there is a path from v_i to v_j that has v_1 and v_2 as interior vertex. d_3 has 1 as its $(i,j)^{th}$ entry if there is a path from v_i to v_j that has v_1, v_2 and v_3 as interior vertex.

$$d_4 = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Finally, d_4 has 1 as its $(i,j)^{th}$ entry if there is a path from v_i to v_j that has v_1, v_2, v_3 and v_4 as interior vertex. The matrix d_4 is the transitive closure.

Algorithm

procedure warshall ($M_R : n \times n$ zero one matrix)

{

$$D = M_R$$

for $k = 1$ to n

begin

for $i = 1$ to n

begin

for $j = 1$ to n

$$d_{ij} = d_{ij} \vee (d_{ik} \wedge d_{kj})$$

end

$D = [d_{ij}]$ is the transitive closure.

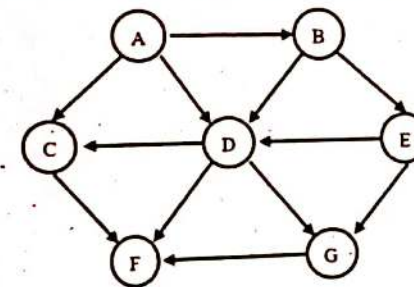
10.14 Topological Sorting

A topological sort is an ordering of vertices in a directed acyclic graph (graph with no cycle) graph (dag), such that if there is a path from v_i to v_j , then v_i appears before v_j in the ordering. A topological sort of a graph can be viewed as an ordering of its vertices along a horizontal line so that all directed edges go from left to right.

Applications:

Consider the course available at university as the vertices of a directed graph; where there is an edge from one course to another if the first is a prerequisite for the second.

A topological ordering is then a listing of all the courses such that all prerequisite for a course appear before it does.



G_1

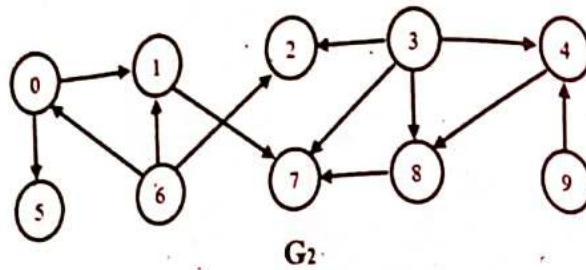


Figure 10.4: Directed graph with no directed cycle

Algorithms to find out topological ordering:

1. Breadth First Ordering
2. Depth First Ordering

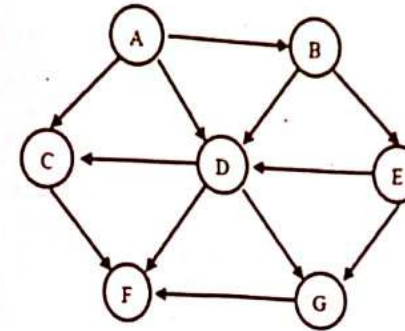
10.14.1 Breadth First Ordering

In breadth first topological ordering of a directed graph with no cycles, we start by finding the vertices that should be first in the topological order and then apply the fact that every vertex must come before its successors in the topological order. The vertices that come first are that are not successors of any other vertex.

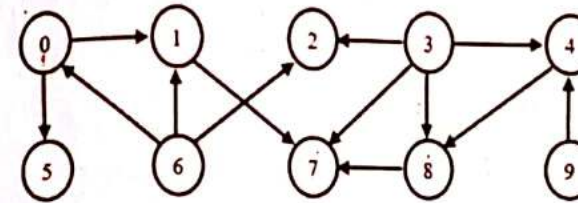
Determining the breadth first topological ordering:

1. Fill the information from the graph into the table (vertices, predecessors and predecessor count).
2. Visit the vertex that have predecessor count zero (0).
3. Decrement the predecessor count value by 1 where ever the visited vertex appears in the adjacent predecessor vertices column.
4. Repeat steps 2 and 3 until all the vertices has been visited.

Example: Determine the breadth first topological ordering for the following graphs G_1 and G_2 .



G_1



G_2

Solution:

From the graph G_1 we get the following information:

Vertex	Adjacent predecessor vertices	Predecessor count
A	-	0
B	A	1 0
C	A, D	2 1 0
D	A, B, E	3 2 1 0
E	B	1 0
F	C, D, G	3 2 1 0
G	D, E	2 1 0

The breadth first topological ordering gives:

A	B	E	D	C	G	F
---	---	---	---	---	---	---

From the graph G_2 we get following information:

Vertex	Adjacent predecessor vertices	Predecessor count
0	6	1 0
1	0, 6	2 1 0
2	3, 6	2 1 0
3	-	0
4	3, 9	2 1 0
5	0	1 0
6	-	0
7	1, 3, 8	3 2 1 0
8	3, 4	2 1 0
9	-	0

The breadth first topological ordering gives:

3	6	9	0	2	4	1	5	8	7
---	---	---	---	---	---	---	---	---	---

10.14.2 Depth First Ordering

Start by finding a vertex that has no successors and place it last in the order.

After, we have, by recursion placed all the successors of a vertex into the topological order, then place the vertex itself in the order (before any of its successors).

Determining the depth first topological ordering:

1. Fill the information from the graph into the table (vertices, successors and successor count).
2. Push all the vertices into stack and visit the vertex that is on the top.
3. Decrement the successor count value by 1 wherever the visited vertex appears in the adjacent successor vertices column.
4. Repeat steps 2 and 3 until all the vertices has been visited.

Example: Determine the depth first topological ordering for the above graphs G_1 and G_2 .

Solution:

From the graph G_1 , we get the following information:

Vertex	Adjacent successor vertices	Successor count
A	B, C, D	3 2 1 0
B	D, E	2 1 0
C	F	1 0
D	C, F, G	3 2 1 0
E	D, G	2 1 0
F	-	0
G	F	1 0

The depth first topological ordering gives:

A	B	E	D	C	G	F
---	---	---	---	---	---	---

From the graph G_2 , we get the following information:

The breadth first topological ordering gives:

A	B	E	D	C	G	F
---	---	---	---	---	---	---

From the graph G_2 we get following information:

Vertex	Adjacent predecessor vertices	Predecessor count
0	6	1 0
1	0, 6	2 1 0
2	3, 6	2 1 0
3	-	0
4	3, 9	2 1 0
5	0	1 0
6	-	0
7	1, 3, 8	3 2 1 0
8	3, 4	2 1 0
9	-	0

The breadth first topological ordering gives:

3	6	9	0	2	4	1	5	8	7
---	---	---	---	---	---	---	---	---	---

10.14.2 Depth First Ordering

Start by finding a vertex that has no successors and place it last in the order.

After, we have, by recursion placed all the successors of a vertex into the topological order, then place the vertex itself in the order (before any of its successors).

Determining the depth first topological ordering:

1. Fill the information from the graph into the table (vertices, successors and successor count).
2. Push all the vertices into stack and visit the vertex that is on the top.
3. Decrement the successor count value by 1 wherever the visited vertex appears in the adjacent successor vertices column.
4. Repeat steps 2 and 3 until all the vertices has been visited.

Example: Determine the depth first topological ordering for the above graphs G_1 and G_2 .

Solution:

From the graph G_1 , we get the following information:

Vertex	Adjacent successor vertices	Successor count
A	B, C, D	3 2 1 0
B	D, E	2 1 0
C	F	1 0
D	C, F, G	3 2 1 0
E	D, G	2 1 0
F	-	0
G	F	1 0

The depth first topological ordering gives:

A	B	E	D	C	G	F
---	---	---	---	---	---	---

From the graph G_2 , we get the following information:

Vertex	Adjacent successor vertices	Successor count
0	1, 5	2 + 0
1	7	1 + 0
2	-	0
3	2, 4, 7, 8	4 + 3 + 2 + 0
4	8	1 + 0
5	-	0
6	0, 1, 2	3 + 2 + 0
7	-	0
8	7	1 + 0
9	4	1 + 0

The depth first topological ordering gives:

9	6	3	4	8	2	0	5	1	7
---	---	---	---	---	---	---	---	---	---

References

1. Y. Langsam, M. J. Augenstein, and A. M. Tanenbaum, "Data Structures Using C and C++", Second Edition, Prentice Hall.
2. N. Upadhyay, "The Design and Analysis of Algorithm", S. K. Kataria and Sons.
3. E. Horowitz, S. Sahni, and D. Mehta, "Fundamentals of Data Structures in C++", W.H. Freeman and Company, 1995.
4. A. V. Aho, J. E. Hopcroft, and J. D. Ullman, "Data Structures and Algorithms", Addison-Wesley, 1983.
5. J. P. Tremblay, and P.G. Sorenson, "An Introduction to Data Structures with Application".
6. R. L. Kruse, B. P. Leung, and C. L. Tondo, "Data Structures and Program Design in C", Prentice Hall, 1991.
7. M. A. Weiss, "Data Structures and Algorithm Analysis in C", Second Edition, Benjamin-Cummings, 1997.
8. G. S. Baluja, "Data Structures Through C", Dhanpat Rai and Co.
9. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, "Introduction to Algorithms", PHI.